

Unified Checkout

REST API

Developer Guide

© 2025. Cybersource Corporation. All rights reserved.

Cybersource Corporation (Cybersource) furnishes this document and the software described in this document under the applicable agreement between the reader of this document (You) and Cybersource (Agreement). You may use this document and/or software only in accordance with the terms of the Agreement. Except as expressly set forth in the Agreement, the information contained in this document is subject to change without notice and therefore should not be interpreted in any way as a guarantee or warranty by Cybersource. Cybersource assumes no responsibility or liability for any errors that may appear in this document. The copyrighted software that accompanies this document is licensed to You for use only in strict accordance with the Agreement. You should read the Agreement carefully before using the software. Except as permitted by the Agreement, You may not reproduce any part of this document, store this document in a retrieval system, or transmit this document, in any form or by any means, electronic, mechanical, recording, or otherwise, without the prior written consent of Cybersource.

Restricted Rights Legends

For Government or defense agencies: Use, duplication, or disclosure by the Government or defense agencies is subject to restrictions as set forth the Rights in Technical Data and Computer Software clause at DFARS 252.227-7013 and in similar clauses in the FAR and NASA FAR Supplement.

For civilian agencies: Use, reproduction, or disclosure is subject to restrictions set forth in subparagraphs (a) through (d) of the Commercial Computer Software Restricted Rights clause at 52.227-19 and the limitations set forth in Cybersource Corporation's standard commercial agreement for this software. Unpublished rights reserved under the copyright laws of the United States.

Trademarks

Authorize.net and The Power of Payment are registered trademarks of Cybersource Corporation. Cybersource and Cybersource Decision Manager are trademarks and/or service marks of Cybersource Corporation. Visa, Visa International, Cybersource, the Visa logo, the Cybersource logo, and 3-D Secure are the registered trademarks of Visa International in the United States and other countries. All other trademarks, service marks, registered marks, or registered service marks are the property of their respective owners.

Version: 25.12.01

Contents

Unified Checkout Developer Guide.....	6
Recent Revisions to This Document.....	7
Introduction to Unified Checkout.....	12
Unified Checkout Flow.....	14
Enabling Unified Checkout in the Business Center.....	16
Server-Side Set Up.....	17
Capture Context.....	17
Client-Side Set Up.....	22
Loading the JavaScript Library and Invoking the Accept Function.....	22
JavaScript Example: Initializing the SDK.....	23
JavaScript Example: Displaying the Button List.....	24
JavaScript Example: Client-Defined Trigger for Click to Pay or PAN Entry.....	24
JavaScript Example: Processing a Payment.....	25
Complete Integration Examples.....	27
JavaScript Example: Setting Up with Full Sidebar.....	27
JavaScript Example: Setting Up with the Embedded Component.....	28
Capture Context API.....	30
Features.....	41
Client Version History.....	45
Requesting the Capture Context.....	47
Required Fields for Requesting the Capture Context.....	48
REST Example: Requesting the Capture Context.....	48
Validating the Capture Context.....	54
Transient Tokens.....	56
Transient Token Format.....	56
Example: Transient Token Format.....	56
Token Verification.....	59
Dual-Branded Cards.....	59
Authorizations with a Transient Token.....	60
REST Example: Requesting an Authorization with a Transient Token.....	61
Payment Details API.....	63
REST Example: Retrieving Transient Token Payment Details.....	64

JavaScript API Reference.....	65
Class: Accept.....	65
Class: AcceptError.....	67
Class: UnifiedPayments.....	70
Unified Checkout Configuration.....	74
Webhooks Support.....	75
Enable Digital Payments.....	79
Enrolling in Apple Pay.....	79
Preparing a Device for Testing Apple Pay on Unified Checkout.....	80
Enabling Click to Pay.....	81
Enrolling in Google Pay.....	87
Managing Google Pay Authentication Types.....	87
Enrolling in Echeck/ACH Services.....	88
Enrolling in Alternative Payment Methods.....	89
Manage Permissions.....	90
Managing Permissions as a Direct Merchant.....	91
Managing Permissions as a Portfolio Administrator.....	92
Test Your Unified Checkout Configuration.....	93
Test Payment Details.....	94
Payment Methods.....	98
Cards.....	99
Online Bank Transfers.....	103
Bancontact.....	105
DragonPay.....	106
iDeal.....	107
MyBank.....	108
Multibanco.....	109
Przelewy24 P24.....	110
Tink Pay By Bank.....	111
Verify Status for Online Bank Transfers (PPRO Enabled).....	112
Verify Status for Online Bank Transfers (Tink Pay By Bank).....	113
Handle Responses.....	114
Buy Now, Pay Later.....	115
Afterpay.....	116
Handle Responses.....	118

Captures.....	118
Post-Pay Reference Payments.....	119
Konbini.....	120
Verify Status for Post-Pay Reference (PPRO Enabled).....	122
Handle Responses.....	122
Unified Checkout Appendix.....	124
Unified Checkout UI.....	125
Apple Pay UI.....	125
Click to Pay UI.....	127
Google Pay UI.....	128
Manual Payment Entry UI.....	129
Pay with Bank Account UI.....	131
Paze UI.....	134
JSON Web Tokens.....	136
Unified Checkout Integration Examples.....	137
Unified Checkout with Complete Mandate.....	137
Unified Checkout with Sale.....	139
Unified Checkout with Sale and Decision Manager.....	142
Unified Checkout without Service Orchestration.....	144
Echeck Payment with a Transient Token.....	146
Example: Unified Checkout Complete Capture Context.....	148
Supported Countries for Digital Payments.....	153
Supported Countries for Digital Payments A-D.....	153
Supported Countries for Digital Payments E-K.....	155
Supported Countries for Digital Payments L-R.....	158
Supported Countries for Digital Payments S-Z.....	161
Supported Locales.....	164
Reason Codes.....	167
Unified Checkout Field Reference.....	168
Test Authentication	185
Click to Pay UI Guidelines.....	187
Click to Pay UI Examples.....	190
VISA Platform Connect: Specifications and Conditions for Resellers/Partners.....	196

Unified Checkout Developer Guide

This section describes how to use this guide and where to find further information.

Audience and Purpose

This document is written for merchants who want to enable Unified Checkout so that they can accept payments on their e-commerce page. Merchants use the Unified Checkout JavaScript SDK to place a digital button widget on their e-commerce site, allowing Cybersource to capture payment data on their behalf.

Conventions

This statement is used in this document:



Important: An *Important* statement contains information essential to successfully completing a task or learning a concept.

Related Documentation

Visit the [Cybersource documentation hub](#) to find additional technical documentation.

Customer Support

For support information about any service, visit the Support Center:

<http://support.visaacceptance.com>

Recent Revisions to This Document

25.12.01

Server-Side Set Up

Added information about supported browsers. See [Capture Context \(on page 17\)](#).

Capture Context

Added support for up to **clientVersion** 0.34. See [Client Version History \(on page 45\)](#).

Transient Tokens

Added information about Cartes Bancaires dual-branded cards. See [Dual-Branded Cards \(on page 59\)](#).

Payment Methods

Added information about Konbini. See [Konbini \(on page 120\)](#).

Appendix

Added missing fields to the field reference. See [Unified Checkout Field Reference \(on page 168\)](#).

Test Your Unified Checkout Configuration

Added test payment information for testing Visa Click to Pay authentication within the Click to Pay flow and outside of the flow. See [Test Your Unified Checkout Configuration \(on page 93\)](#).

25.11.02

Capture Context

Added a complete capture context request example with all possible fields. See [Example: Unified Checkout Complete Capture Context \(on page 148\)](#).

Updated the capture context examples for **clientVersion** 0.31. See these topics:

- [Capture Context \(on page 17\)](#)
- [Requesting the Capture Context \(on page 47\)](#)
- [Unified Checkout Integration Examples \(on page 137\)](#)

Appendix

Added information about available capture context fields. See [Unified Checkout Field Reference \(on page 168\)](#).

Added information about testing your authentication method. See [Test Authentication \(on page 185\)](#).

Added information about customizing the Click to Pay UI. See [Click to Pay UI Guidelines \(on page 187\)](#).

25.11.01

Capture Context

Updated the list of allowed card networks. See [Capture Context API \(on page 30\)](#).

Payment Methods

Added information about Bancontact, Dragonpay and MyBank. See these topics:

- [Bancontact \(on page 105\)](#)
- [DragonPay \(on page 106\)](#)
- [MyBank \(on page 108\)](#)

25.10.02

Capture Context

Added support for **clientVersion** [0.30](#). See [Client Version History \(on page 45\)](#).

Payment Methods

Added information about Tink Pay By Bank. See [Tink Pay By Bank \(on page 111\)](#).

25.10.01

JavaScript API Reference

Added properties to [Class: AcceptError \(on page 67\)](#)

Unified Checkout Appendix

Added missing reason codes. See [Reason Codes \(on page 167\)](#).

Updated the **clientVersion** field value to [0.30](#) in examples.

Added Pakistan locales. See [Supported Locales \(on page 164\)](#).

Payment Methods

Added information about handling responses for Online Bank Transfers and Buy Now, Pay Later. See [Handle Responses for Online Bank Transfers \(on page 114\)](#) and [Handle Responses for Buy Now, Pay Later \(on page 118\)](#).

25.08.02

Added more information about JSON Web Tokens. See [JSON Web Tokens \(on page 136\)](#).

25.08.01

Client-Side Setup

Added information for creating a trigger. See [JavaScript Example: Client-Defined Trigger for Click to Pay or PAN Entry \(on page 24\)](#).

Capture Context

Added PayPak as a payment method. See [Capture Context API \(on page 30\)](#).

Added information for client version 0.28. See [Client Version History \(on page 45\)](#).

Payment Methods

Added information for alternative payment methods. See [Payment Methods \(on page 98\)](#).

25.05.01

Capture Context

Added information for the complete mandate. See [Capture Context API \(on page 30\)](#).

Added information on the available features and the fields specific to each feature to the Capture Context API. See [Capture Context API \(on page 30\)](#).

Added Paze to the list of allowed payment types. See [Capture Context API \(on page 30\)](#).

Added capture context validation. See [Validating the Capture Context \(on page 54\)](#).

Unified Checkout Appendix

Added a client version history and the features included in each version. See [Client Version History \(on page 45\)](#).

Added Unified Checkout payment examples. See [Unified Checkout Integration Examples \(on page 137\)](#).

Updated the list of supported languages. See [Supported Locales \(on page 164\)](#).

Client-Side Set Up

Updated the examples for loading the JavaScript library:

- [JavaScript Example: Initializing the SDK \(on page 23\)](#)
- [JavaScript Example: Displaying the Button List \(on page 24\)](#)
- [JavaScript Example: Processing a Payment \(on page 25\).](#)

Click to Pay

Updated the list of supported countries to include Bulgaria, Greece, Japan, Romania, Slovenia, Thailand, and Vietnam. See [Supported Countries for Digital Payments \(on page 153\)](#).

25.03.02

Added information about choosing your preferred card number prefix length and supported scenarios. See the Include Card Prefix sections of these topics:

- [Capture Context \(on page 17\)](#)
- [Capture Context API \(on page 30\)](#)

25.03.01

Replaced *check* digital payment method with *eCheck* digital payment method throughout [Capture Context \(on page 17\)](#).

Corrected the example code in [REST Example: Requesting the Capture Context \(on page 48\)](#).

Added information on using customer authentication with Click to Pay for Visa transactions. See [Click to Pay Customer Authentication \(on page 85\)](#).

25.02.01

Added optional setup parameters to control the types of credentials that Google Pay on Unified Checkout receives from Google. See [Managing Google Pay Authentication Types \(on page 87\)](#).

Added test card numbers. See [Test Payment Details \(on page 94\)](#).

25.01.01

Updated the Important note about China UnionPay cards that do not have CVNs. See [Capture Context \(on page 17\)](#).

Added steps for enrolling in Apple Pay. See [Preparing a Device for Testing Apple Pay on Unified Checkout \(on page 80\)](#).

24.12.01

Capture Context

Updated the capture context request sections to include the most recent client version and allowed payment types. See [Capture Context \(on page 17\)](#) and [Capture Context API \(on page 30\)](#).

Dual-Branded and Co-Badged Cards

Added information about dual-branded cards. See [Dual-Branded Cards \(on page 59\)](#).

24.10.01

Added information about China UnionPay cards that do not have a card verification number (CVN) and expiration date. See the Important note in [Capture Context \(on page 17\)](#).

24.09.01

Added a note about transient token expiration. See [Authorizations with a Transient Token \(on page 60\)](#).

Added the Unified Checkout JavaScript Reference. See [JavaScript API Reference \(on page 65\)](#).

Introduction to Unified Checkout

Unified Checkout provides a single interface with which you can accept numerous types of digital payments. Unified Checkout calls other follow-on services such as Payments, Decision Manager, Payer Authentication, and Token Management Service (TMS).

Unified Checkout consists of a server-side component and a client-side JavaScript library.

The server-side component authenticates your merchant identity and instructs the system to act within your payment environment. The response contains limited-use public keys. The keys are for end-to-end encryption and contain merchant-specific payment information that drives the interaction of the application. The client-side JavaScript library dynamically and securely places digital payment options onto your e-commerce page.

The provided JavaScript library enables you to securely accept many payment options within your e-commerce environment. Unified Checkout can be embedded seamlessly into your existing webpage, simplifying payment acceptance.

When a customer selects a payment method from the button widget, Unified Checkout handles all of the interactions with the digital payment that was chosen. Unified Checkout is also able to orchestrate requests for follow-on services such as Payments, Decision Manager, Payer Authentication, and TMS before it provides a response to your e-commerce system.

The figure below shows Unified Checkout with customer checkout payment options.

Button Widget

Your checkout
page's UX

9:41

Your basket

Unisex Cotton Hoodie
Navy / Large

£55.00

Subtotal: £55.00

Shipping: FREE

VAT (20%): £11.00

Total: £66.00

Checkout with card

Pay by Bank

Apple Pay

Google Pay

VISA

Unified
Checkout

9:41

CHECKOUT

CONTACT DETAILS
example@email.com
+44 7412 112233

PAYMENT DETAILS:
e.....e@email.com

VISA 5555

VISA 5678

★ Reward points with this card

Enter card manually

Continue

Your checkout
page's UX

9:41

FASHION

Thank you for shopping with us!

YOUR ORDER #3505

SHIPPING TO:
1 Sheldon Square, Paddington, London W2 6TT

PAYMENT METHOD:
VISA 5555

Continue shopping

For examples of different payment method UIs through Unified Checkout, see [Unified Checkout UI \(on page 125\)](#).



Important: Each request that you send to Cybersource requires header information. For information about constructing the headers for your request, see the [Getting Started with REST Developer Guide](#).

Unified Checkout Flow

To integrate Unified Checkout into your platform, you must follow several integration steps. This section gives a high-level overview of how to integrate and launch Unified Checkout on your webpage and process a transaction. You can find the detailed specifications of the APIs later in this document.

The integration flow consists of four events:

1. You send a server-to-server API request for a capture context. This request is fully authenticated and returns a JSON Web Token (JWT) that is necessary in order to invoke the frontend JavaScript library. For information on setting up the server side, see [Server-Side Set Up \(on page 17\)](#).
2. You invoke the Unified Checkout JavaScript library using the JWT response from the capture context request. For information on setting up the client side, see [Client-Side Set Up \(on page 22\)](#).
3. You process the payment through the Unified Checkout complete mandate feature and interpret the response of the transaction flow that is displayed on your browser.



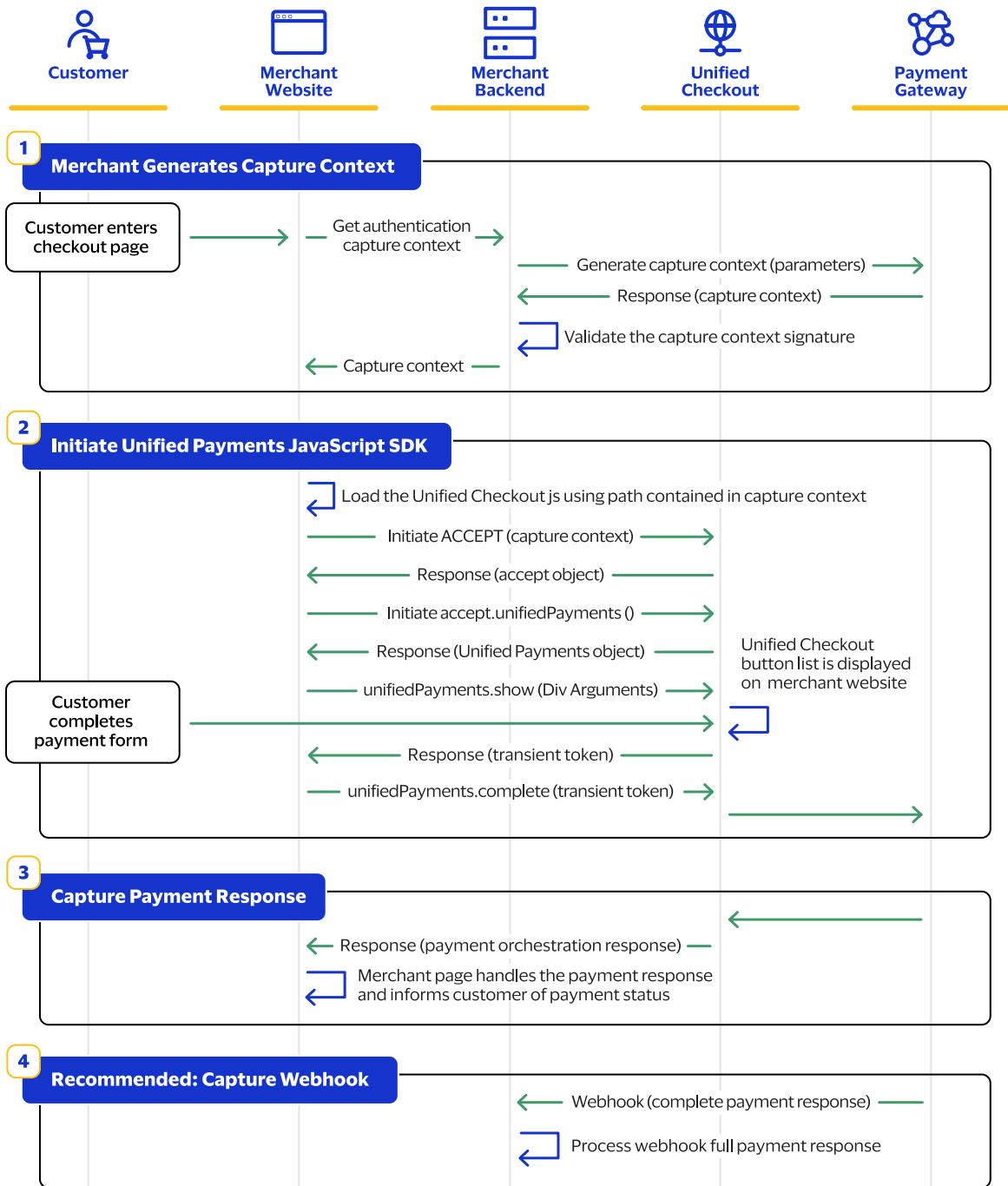
Important: If you do not use the complete mandate for Unified Checkout, you must request follow-on services using a transient token to process the payment.

4. **Optional:** Cybersource recommends that you capture the full payment response through webhook notifications.

Information that is captured by Unified Checkout, including the billing and shipping address, can be retrieved using the payment details API.

The figure below shows the Unified Checkout payment flow.

Unified Checkout Payment Flow



For more information on the specific APIs referenced, see these topics:

- [Capture Context API \(on page 30\)](#)
- [Payment Details API \(on page 63\)](#)

Enabling Unified Checkout in the Business Center

To begin using Unified Checkout, you must first ensure that your merchant ID (MID) is configured to use the service and that any payment methods you intend to use are properly set up.

1. Log in to the Business Center:

Test URL: <https://businesscentertest.cybersource.com/ebc2>

Production URL: <https://businesscenter.cybersource.com>

If you are unable to access this page, contact your sales representative.

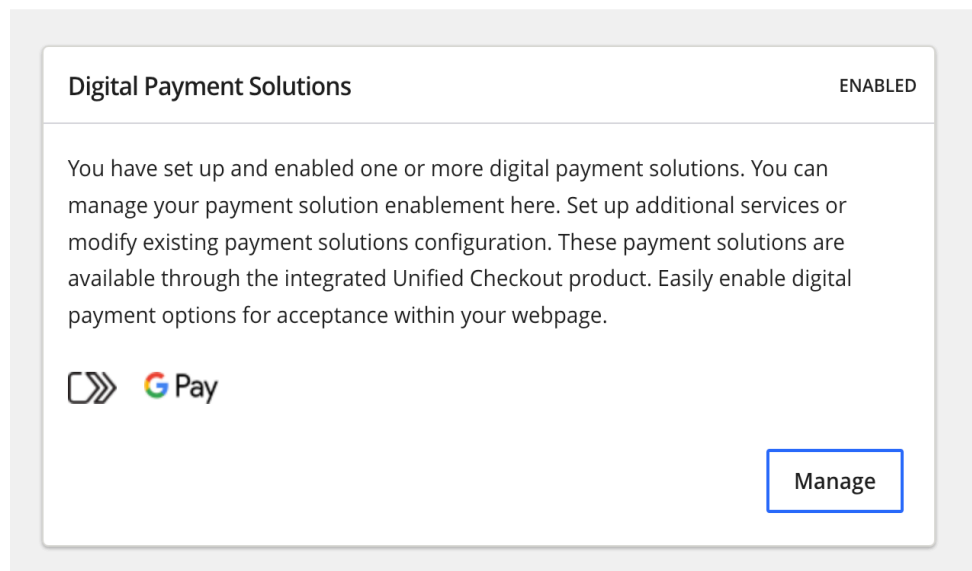
2. In the Business Center, go to the left navigation panel and choose **Payment Configuration > Unified Checkout**.
3. You can configure various payment methods such as Apple Pay, Click to Pay, and Google Pay. Click **Manage** and follow the instructions for your selected payment methods. When payment methods are enabled, they appear on the payment configuration page.



Important: You must configure payment methods you want to use for each transacting MID.

Payment Configuration

Unified Checkout



4. Click **Manage** to edit your existing payment method configurations or enroll in new payment methods as they are released.

Server-Side Set Up

This section contains the information you need to set up your server. Initializing Unified Checkout within your webpage begins with a server-to-server call to the sessions API. This step authenticates your merchant credentials, and establishes how the Unified Checkout frontend components will function. The sessions API request contains parameters that define how Unified Checkout performs.

The server-side component provides this information:

- A transaction-specific public key is used by the customer's browser to protect the transaction.
- An authenticated context description package that manages the payment experience on the client side. It includes available payment options such as card networks, payment interface styling, and payment methods.

The functions are compiled in a JSON Web Token (JWT) object referred to as the *capture context*. For information JSON Web Tokens, see [JSON Web Tokens \(on page 136\)](#).

Capture Context

This section contains the information you need to set up your server. Initializing Unified Checkout within your webpage begins with a server-to-server call to the sessions API. This step authenticates your merchant credentials, and establishes how the frontend components will function. The sessions API request contains parameters that define how Unified Checkout performs.

The server-side component provides this information:

- A transaction-specific public key is used by the customer's browser to protect the transaction.
- An authenticated context description package that manages the payment experience on the client side. It includes available payment options such as card networks, payment interface styling, and payment methods.

The functions are compiled in a JSON Web Token (JWT) object referred to as the *capture context*.

For information on JWTs see [JSON Web Tokens \(on page 136\)](#).

The capture context request is a signed JSON Web Token (JWT) that includes all of the merchant-specific parameters. This request tells the frontend JavaScript library how to behave within your payment experience. The request provides authentication, one-time keys, the target origin to the Unified Checkout integration in addition to allowed card networks and payment types. The capture context request includes these elements:

- **allowedCardNetworks**
- **allowedPaymentTypes**
- **clientVersion**
- **targetOrigin**
- **completeMandate**

Browser Support

Unified Checkout supports these browser versions:

- Safari 16
- Firefox 121
- Google Chrome/Chium-based browsers 118
- Microsoft Edge 118

Capture Context Example

Use the **targetOrigins** and the **allowedPaymentTypes** fields to define the target origin and the accepted digital payment methods in your capture context. Use the **completeMandate** to orchestrate follow-on services such as Payments, Decision Manager, Payer Authentication, and TMS. For example:

```
{
  "targetOrigins" : [ "https://test.com" ],
  "clientVersion": "0.34",
  "buttonType": "CHECKOUT_AND_CONTINUE",
  "allowedCardNetworks": [
    "VISA",
    "MASTERCARD"
  ],
  "allowedPaymentTypes": [
    "PANENTRY",
    "CLICKTOPAY",
    "APPLEPAY",
    "GOOGLEPAY"
  ],
  "completeMandate": {
    "type": "CAPTURE",
    "decisionManager": true,
    "consumerAuthentication": true,
    "tms": {
```

```

        "tokenCreate": true,
        "tokenTypes": [
            "customer",
            "paymentInstrument",
            "instrumentIdentifier",
            "shippingAddress"
        ]
    },
    "country": "US",
    "locale": "en_US",
    "captureMandate": {
        "billingType": "FULL",
        "requestEmail": true,
        "requestPhone": true,
        "requestShipping": true,
        "shipToCountries": [
            "US",
            "GB"
        ],
        "showAcceptedNetworkIcons": true
    },
    "data": {
        "orderInformation": {
            "billTo": {
                "country": "US",
                "firstName": "NEW",
                "lastName": "Test",
                "phoneNumber": "1234567890",
                "address2": "Desk M3-5573",
                "address1": "901 Metro Center Blvd",
                "buildingNumber": "150",
                "postalCode": "94404",
                "locality": "Foster City",
                "administrativeArea": "CA",
                "email": "test@example.com"
            },
            "shipTo": {
                "country": "US",
                "firstName": "NEW",
                "lastName": "Test",
                "address2": "Desk M3-5573",
                "address1": "901 Metro Center Blvd",
                "buildingNumber": "150",
                "postalCode": "94404",
                "locality": "Foster City",
                "administrativeArea": "CA"
            }
        }
    }
}

```

```
    "amountDetails": {  
      "totalAmount": "13.00",  
      "currency": "USD"  
    },  
    "clientReferenceInformation": {  
      "code": "TAGX001"  
    }  
  }  
}
```

Card Entry Form

This diagram shows how elements of the capture context request appear in the card entry form.

Anatomy of a Manual Card Entry Form

```
{
  "targetOrigins" : [ "https://the-up-demo.appspot.com" ],
  "clientVersion" : "0.19",
  "allowedCardNetworks" : [ "VISA", "MASTERCARD", "AMEX" ],
  "allowedPaymentTypes" : [ "CLICKTOPAY" ],
  "country" : "US",
  "locale" : "en_US",
  "captureMandate" : {
    "billingType" : "FULL",
    "requestEmail" : true,
    "requestPhone" : true,
    "requestShipping" : true,
    "shipToCountries" : [ "US", "GB" ],
    "showAcceptedNetworkIcons" : true
  },
  "orderInformation" : {
    "amountDetails" : {
      "totalAmount" : "1.01",
      "currency" : "USD"
    },
    "billTo" : {
      "address1" : "277 Park Avenue",
      "administrativeArea" : "NY",
      "buildingNumber" : "218",
      "country" : "US",
      "district" : "district",
      "locality" : "New York",
      "postalCode" : "10172",
      "email" : "example@email.com",
      "firstName" : "Joe",
      "lastName" : "Cool",
      "middleName" : "S",
      "nameSuffix" : "Jr",
      "title" : "Mr",
      "phoneNumber" : "1234567890",
      "phoneType" : "phoneType"
    },
    "shipTo" : {
      "address1" : "123 Cool St",
      "administrativeArea" : "CA",
      "buildingNumber" : "#12",
      "country" : "US",
      "district" : "string",
      "locality" : "Beverly Hills",
      "postalCode" : "90210",
      "firstName" : "Joe",
      "lastName" : "Cool"
    }
  }
}
```

9:41

FASHION

CONTACT DETAILS

Email

example@email.com

Mobile number

+44 7412 112233

Click to Pay didn't find linked cards for
example@email.com

9:41

FASHION

CONTACT DETAILS [Edit](#)

example@email.com
+44 7412 112233

PAYMENT DETAILS:

Card number

Expiry MM/YY

CVV

First name

Last name

Click to Pay didn't find linked cards for
example@email.com

[Switch ID](#)

Continue

9:41

FASHION

CONTACT DETAILS [Edit](#)

example@email.com
+44 7412 112233

PAYMENT DETAILS:

Card number

Expiry MM/YY

CVV

First name

Last name

☒ [Link this card to Click to Pay](#)

By clicking the button below, you agree to the [Terms](#) for Click to Pay and understand your data will be processed according to the [Privacy Notice](#).
We'll send you a confirmation email.

BILLING ADDRESS:

Address line 1

1 Sheldon Square

Address line 2

Paddington Central

Post code

City / Town

W2 6TT

London

Country

United Kingdom

☒ Shipping address is same as billing

Continue

Client-Side Set Up

This section contains the information you need to set up the client side. You use the Unified Checkout JavaScript library to add the payment interface to your e-commerce site. It has two primary components:

- The button widget, which lists the payment methods available to the customer.
- The payment acceptance page, which captures payment information from the cardholder. You can set up the payment acceptance page to be embedded with your webpage or added as a sidebar.

Follow these steps to set up the client:

1. Load the JavaScript library.
2. Initialize the accept object, the capture context JWT. For information JSON Web Tokens, see [JSON Web Tokens \(on page 136\)](#).
3. Initialize the unified payment object with optional parameters.
4. Show the button list or payment acceptance page or both.
5. Process the payment request using the instructions included within the capture mandate.

The response to these interactions is a transient token that you can use to retrieve the payment information captured by the UI.

Loading the JavaScript Library and Invoking the Accept Function

Use the client library asset path and client library integrity value that is returned by the capture context response to invoke Unified Checkout on your page.

You can retrieve these values from the **clientLibrary** and **clientLibraryIntegrity** fields that are returned in the JWT from <https://apitest.cybersource.com/up/v1/capture-contexts>. You can use these values to create your script tags.

You must perform this process for each transaction, as these values may be unique for each transaction. You must avoid hard-coding values for the **clientLibrary** and **clientLibraryIntegrity** fields to prevent client-side errors.

For example, a response from <https://apitest.cybersource.com/up/v1/capture-contexts> would include:

```
"data": {
  "clientLibrary": "[EXTRACT clientLibrary VALUE from here]",
  "clientLibraryIntegrity": "[EXTRACT clientLibraryIntegrity VALUE from here]"
}
```

Below is an example script tag:

```
<script src="[INSERT clientLibrary VALUE HERE]"
  integrity="[INSERT clientLibraryIntegrity VALUE HERE]"
  crossorigin="anonymous"></script>
```



Important: Use the **clientLibrary** and **clientLibraryIntegrity** parameter values in the capture context response to obtain the Unified Checkout JavaScript library URL and the integrity value. This ensures that you are always using the most up-to-date library and protects against fraud. Do not hard-code the Unified Checkout JavaScript library URL or integrity value.

When you load the library, the capture context from your initial server-side request is used to invoke the accept function.

JavaScript Example: Initializing the SDK

```
try {
  const accept = await Accept(captureContext);
  const up = await accept.unifiedPayments(sideBar);

} catch (error) {
  // merchant logic for handling issues
  console.error("something went wrong: " + error);
}
```

In this example, `captureContext` refers to the capture context JWT.

JavaScript Example: Displaying the Button List

After you initialize the Unified Checkout object, you can add the payment application and payment acceptance pages to your webpage. You can attach the embedded Unified Checkout tool and payment acceptance pages to any named element within your HTML. Typically, they are attached to explicit named components that are replaced with Unified Checkout's iframes.

```
try {
  const accept = await Accept(captureContext);
  const up = await accept.unifiedPayments(sidebar);
  const tt = await up.show(showArgs);
} catch (error) {
  // merchant logic for handling issues
  console.error("something went wrong: " + error);
}
```

To display the Unified Checkout Button List within your payment page, a call is made to the `unifiedPayments.Show()` function. This function accepts a JSON object that links the `<div>` tags within your payment page to place the Unified Checkout button list and optional embeddable payment page.

```
const showArgs = {
  containers: {
    paymentSelection: '#buttonPaymentListContainer',
    paymentScreen: '#embeddedPaymentContainer'
  }
};
```

The response to the `unifiedPayment.show()` method is a JWT data object referred to here as a transient token. The transient token contains all the payment information captured during the Unified Checkout payment journey.

JavaScript Example: Client-Defined Trigger for Click to Pay or PAN Entry

When you display `CLICKTOPAY` or `PANENTRY` as allowed payment types, you can load the UI without displaying the Unified Checkout checkout button. You can do this by creating a trigger that defines what event loads the UI.

You can create a trigger by calling the `createTrigger()` method on an existing unified payments object and pass in these two parameters:

- The payment method that the trigger is linked to. This is required.
- The container for the payment screen. It is required when you are in embedded mode.



Important: You can create a trigger only for [CLICKTOPAY](#) or [PANENTRY](#) payment methods.

```
// Example: Basic usage with full sidebar experience
// Create a trigger
const trigger = up.createTrigger("CLICKTOPAY");

// Show the trigger
// In this example, when a button in your UI is clicked
const myButton = document.getElementById("myButton");

myButton.addEventListener("click", async () => {
  const transientToken = await trigger.show();
  console.log(transientToken);
})
```

```
// Example: Payment screen in a container
// Define the container for the payment screen to be rendered in
var options = { containers: { paymentScreen: '#paymentScreenContainer' } };
// Create the trigger
const trigger = up.createTrigger("CLICKTOPAY", options);

// Show the trigger
// In this example, when a button in your UI is clicked
const myButton = document.getElementById("myButton");

myButton.addEventListener("click", async () => {
  const transientToken = await trigger.show();
  console.log(transientToken);
})
```



Important: When you use the [createTrigger\(\)](#) method for Click to Pay, you must create a custom UI. See [Click to Pay UI Guidelines \(on page 187\)](#).

JavaScript Example: Processing a Payment

Payment is initiated when Unified Checkout captures the customer's payment information by calling the [unifiedPayment.complete\(\)](#) function and passing the transient token.



Important: If you are updating an existing Unified Checkout configuration to use the complete mandate, you must update your JavaScript to include the **unifiedPayment.complete()** function.

```
try {
  const accept = await Accept(captureContext);
  const up = await accept.unifiedPayments(sideBar);
  const tt = await up.show(showArgs);
  const completeResponse = await up.complete(tt);

  console.log(completeResponse); // merchant logic for handling complete
  response
} catch (error) {
  // merchant logic for handling issues
  console.error("something went wrong: " + error);
}
}
```

When you include this in your capture context, Unified Checkout is leveraged to initiate payment and any follow-on services that you include in your capture context request. Alternatively, you can call the payment APIs directly. See [Authorizations with a Transient Token \(on page 60\)](#).

Complete Integration Examples

These examples show a full Unified Checkout integration and use the complete method for processing a payment.

JavaScript Example: Setting Up with Full Sidebar

```
<html>
<head>
  <script
    src="[INSERT clientLibrary VALUE HERE]"
    integrity="[INSERT clientLibraryIntegrity VALUE HERE]"
    crossorigin="anonymous"
  ></script>
</head>
<body>
  <h1>Unified Checkout Integration</h1>
  <input
    type="hidden"
    name="captureContext"
    value="[INSERT captureContext HERE]"
  />
  <script type="text/javascript">
    const sidebar = true;
    const captureContext = document.getElementById("captureContext").value;
    const showArgs = { containers: {
      paymentSelection: '#buttonPaymentListContainer',
    }
  };

    async function launchCheckout() {
      try {
        const accept = await Accept(captureContext);
        const up = await accept.unifiedPayments(sidebar);
        const tt = await up.show(showArgs);
        const completeResponse = await up.complete(tt);

        console.log(completeResponse); // merchant logic for handling complete
response
      } catch (error) {
        // merchant logic for handling issues
        console.error("something went wrong: " + error);
      }
    }

    // Call the function
```

```
    launchCheckout();  
</script>  
</body>
```

JavaScript Example: Setting Up with the Embedded Component

The main difference between using an embedded component and the sidebar is that the **accept.unifiedPayments** object is set to `false`, and the location of the payment screen is passed in the `containers` argument.



Important: If you do not specify a location for the payment acceptance page, it is placed in the side bar.

```
<html>  
<head>  
  <script  
    src="[INSERT clientLibrary VALUE HERE]"  
    integrity="[INSERT clientLibraryIntegrity VALUE HERE]"  
    crossorigin="anonymous"  
  ></script>  
</head>  
<body>  
  <h1>Unified Checkout Integration</h1>  
  <input  
    type="hidden"  
    name="captureContext"  
    value="[INSERT captureContext HERE]"  
  />  
  <script type="text/javascript">  
    const sidebar = false;  
    const captureContext = document.getElementById("captureContext").value;  
    const showArgs = {  
      containers: { paymentSelection: "#buttonPaymentListContainer",  
paymentScreen: '#embeddedPaymentContainer' }  
    };  
  
    async function launchCheckout() {  
      try {  
        const accept = await Accept(captureContext);  
        const up = await accept.unifiedPayments(sidebar);  
        const tt = await up.show(showArgs);  
        const completeResponse = await up.complete(tt);  

```

```
        console.log(completeResponse); // merchant logic for handling complete
response
    } catch (error) {
        // merchant logic for handling issues
        console.error("something went wrong: " + error);
    }
}

// Call the function
launchCheckout();
</script>
</body>
```

Capture Context API

The capture context request contains all of the merchant-specific parameters that tell the front-end JavaScript library what to do within your payment experience.

The capture context is a signed JSON Web Token (JWT) containing this information:

- Merchant-specific parameters that dictate the customer payment experience for the current payment transaction.
- A one-time public key that secures the information flow during the current payment transaction.

The capture context request includes these fields:

- **allowedCardNetworks**
- **allowedPaymentTypes**
- **clientVersion**
- **targetOrigins**

For information on JSON Web Tokens, see [JSON Web Tokens \(on page 136\)](#).

Allowed Card Networks

Use the **allowedCardNetworks** field to define the card types.

These card networks are available for card entry:

- American Express
- Cartes Bancaires
- Carnet
- China UnionPay
- Diners Club
- Discover
- EFTPOS
- ELO
- Jaywan

- JCB
- JCrew
- KCP
- mada
- Maestro
- Mastercard
- Meeza
- PayPak
- UATP
- Visa

To support dual-branded or co-badged cards, you must list your supported card type values for the **allowedCardNetworks** field based on your preference for processing card numbers. For example, if a card is dual-branded as Visa and Cartes Bancaires, and Cartes Bancaires is listed first, the card type is set to Cartes Bancaires after the card number is entered in your Unified Checkout card collection form. For information on dual-branded or co-badged cards, see [Dual-Branded Cards \(on page 59\)](#).



Important:

Some card types, such as KCP and UATP, do not have security codes (CVV or CVN). If you include only card types that do not have security codes in the **allowedCardNetworks** field, Unified Checkout does not display the security code field in the UI.

If you include card types that do not have security codes and cards types that do have security codes in the **allowedCardNetworks** field, Unified Checkout displays the security code field in the UI. The field is disabled in the UI when the cardholder enters a card number for a card type with no security code.

Target Origin

The [target origin](#) is defined by the scheme (protocol), hostname (domain), and port number (if used).

You must use the `https://` protocol. Sub domains must also be included in the target origin.

Any valid top-level domains, such as `.com`, `.co.uk`, and `.gov.br`, are supported. Wildcards are not supported.

For example, if you are launching Unified Checkout on example.com, the target origin could be any of the following:

- <https://example.com>
- <https://subdomain.example.com>
- <https://example.com:8080>

When you use Unified Checkout in an iframe, you must include the domain for the URL that loads the iframe and the iframe URL in the **targetOrigins** field.

Allowed Payment Types

You can specify the type of Unified Checkout digital payment methods that you want to accept in the capture context.

Use the **allowedPaymentTypes** field to define the payment type:

- [AFTERPAY](#)
- [APPLEPAY](#)
- [BANCONTACT](#)
- [CHECK](#)
- [CLICKTOPAY](#)
- [DRAGONPAY](#)
- [GOOGLEPAY](#)
- [IDEAL](#)
- [KONBINI](#)
- [MULTIBANCO](#)
- [MYBANK](#)
- [P24](#)
- [PANENTRY](#)
- [PAZE](#)
- [TINKPAYBYBANK](#)



Important: Click to Pay accepts American Express, Mastercard, and Visa for saved cards. Visa and Mastercard tokenize payment credentials using network tokenization for all Click to Pay requests. Click to Pay uses Click to Pay Token Requester IDs (TRIDs) rather than your existing TRIDs to generate network tokens.

For more information on enabling and managing these digital payment methods, see these topics:

- [Enrolling in Apple Pay \(on page 79\)](#)
- [Enabling Click to Pay \(on page 81\)](#)
- [Enrolling in Google Pay \(on page 87\)](#)
- [Enrolling in Alternative Payment Methods \(on page 89\)](#)

Capture Mandate

The capture mandate enables you to define which fields are captured within Unified Checkout. You must include the fields and set the values in the capture context based on the information that you want Unified Checkout to collect. This enables the cardholder to review and edit their details where the UI includes these fields. When the UI is used to capture cardholder information, all captured information is available within the payment details API response. When you want the cardholder to review existing address data, you can include the known customer data in the capture context and this information is pre-filled in the Unified Checkout UI. For information about the payment details API, see [Payment Details API \(on page 63\)](#).

Capture Mandate Field Values and Outcomes

Capture Mandate Field	Value	Outcome
billingType	FULL	These fields are shown in the UI to capture cardholder billing details. When you include the billing details in the capture context, these details are pre-filled in the Unified Checkout UI. All information that is collected from these fields are tokenized in the transient token and sent

Capture Mandate Field Values and Outcomes (continued)

Capture Mandate Field	Value	Outcome
		for payment processing where the Complete Mandate is used.
	NONE	No fields are shown in the UI to capture cardholder billing details. If you are using the Complete Mandate, you must provide billing details in the capture context. All information that is collected from these fields is tokenized in the transient token and sent for payment processing. For information about which fields are required for payment processing, see the Payments Developer Guide.
	PARTIAL	Only the billing postal code and billing country are collected in the UI. Set to this value when you use relaxed address verification services (AVS). This includes markets where postal code and billing country are enough for successful payment processing.
requestEmail	true	The email address is shown and captured in the UI. If you are using Click to Pay, this email address is used to find the cardholder's Click to Pay account.
	false	No email address is shown in the UI.

Capture Mandate Field Values and Outcomes (continued)

Capture Mandate Field	Value	Outcome
		If you are using Click to Pay, this email address is used to find the cardholder's Click to Pay account and it appears in the UI when requestEmail is set to <code>false</code> .
requestPhone	<code>true</code>	The phone number is shown and captured in the UI.
	<code>false</code>	No phone number is shown or captured in the UI.
requestShipping	<code>true</code>	Shipping fields are shown in the UI and are collected by Unified Checkout. When you include the shipping details in the capture context, the information appears prefilled in the UI.
	<code>false</code>	No shipping information is captured in the UI. When shipping details are required for payment processing and are used for follow on services such as Decision Manager, you can include these fields in the capture context. These details are tokenized and passed through.
shipToCountries	ISO country code	When the requestShipping field is set to <code>true</code> , only the countries that are included in this field can be selected by the cardholder for their shipping address.

Complete Mandate

The complete mandate feature provides service orchestration within Unified Checkout and simplifies your integration. Service orchestration enables Unified Checkout to orchestrate services on your behalf. The complete mandate feature provides instructions to the **unifiedPayment.complete()** method in the JavaScript. You must include both the **unifiedPayment.complete()** object in the Javascript and the **completeMandate** field object in your capture context to enable Unified Checkout to initiate services on your behalf from the browser.



Important: If you are updating an existing Unified Checkout configuration to use the complete mandate, you must update your JavaScript to include the **unifiedPayment.complete()** function.



Important: When the **billingType** field is set to **NONE** you must include the required fields within the capture context request to ensure that the required fields are included for payment processing. For information about the fields that are required for payment services, see the [Payments Developer Guide](#).

The complete mandate feature is defined by these fields:

- **completeMandate.type:** This field is required to run the complete mandate and is used to indicate how a payment should be processed.

Possible values:

- **AUTH:** Authorize the payment and capture the funds at a later date.
 - **CAPTURE:** Perform a sale. A sale is a combined authorization and capture in a single request.
 - **PREFER_AUTH:** Perform an authorization if possible. If a payment method requires the funds to be captured immediately, then Unified Checkout captures the payment.
- **completeMandate.decisionManager:** This field determines whether Decision Manager is run. Set this field to **true** and include **completeMandate.type** in your request to run Decision Manager and device fingerprinting services. When Decision Manager runs, it uses the associated Decision Manager configuration based on the merchant ID that is included in the request.

When this field is set to **false** or is not included in the request, Decision Manager and device fingerprinting services do not run.

- **completeMandate.consumerAuthentication:** This field determines whether Payer Authentication should be used. Set this field to `true` and include **completeMandate.type** in your request to run Payer Authentication. When this field set to `true`, Payer Authentication runs. When this field is set to `false` or is not included in the request, Payer Authentication does not run.

When you use Unified Checkout with Payer Authentication, device data is collected through Payer Authentication setup and Unified Checkout completes all calls that are associated with Payer Authentication.

Unified Checkout supports the pass-through fields that are required for Payer Authentication challenge codes. For information about the required fields, see [Unified Checkout Field Reference \(on page 168\)](#). For information about challenge codes, see **consumerAuthenticationInformation.challengeCode** in the [REST API Field Reference](#). Unified Checkout supports 3-D Secure data only when your payment processor supports it. For information see [Visa Data Only](#) in the *Payer Authentication Developer Guide*.

To test Payer Authentication, you must use Payer Authentication test cards. See [Test Cases for 3-D Secure 2.x](#) in the *Payer Authentication Developer Guide*.

Consumer authentication is available for these card types:

- American Express
- Cartes Bancaires
- China UnionPay
- Diners Club
- Discover
- EFTPOS
- ELO
- Jaywan
- JCB
- mada
- Maestro
- Mastercard
- Visa
- Jaywan

Consumer authentication runs for these payment methods when they are supported in your Unified Checkout configuration:

- `PANENTRY`
- `CLICKTOPAY` when the transaction is not authenticated with Click to Pay.
- `GOOGLEPAY` when the transaction is not authenticated with Google Pay.

Unified Checkout does not attempt to authenticate for Click to Pay and Google Pay if the transaction has already been authenticated when it is received by Unified Checkout. For information about testing authentication, see [Test Authentication \(on page 185\)](#).

- **`completeMandate.tms.tokenCreate`**: This field determines if a TMS token is created for the customer's selected payment method. When this field is set to `true`, a token is created. When this field is set to `false` or not included in the request, a token is not created.
- **`completeMandate.tms.tokenTypes`**: This is an optional field that you can use to indicate the token type for the token that is created. When this field is not included in the request, a token is created based on your TMS vault configuration. You can set this field to these values:

- `customer`
- `instrumentIdentifier`
- `paymentInstrument`
- `shippingAddress`

If you want Unified Checkout to capture the cardholder's consent to save the card before a request to create a token is completed, then you must set **`captureMandate.requestSaveCard`** to `true`. When this field is set to `true`, Unified Checkout presents a **Save card for future payments** checkbox within the UI and enables the cardholder to give consent. Do not include **`captureMandate.requestSaveCard`** in your request if you have already gained cardholder consent to create a TMS token or do not require consent.

This table indicates if a token is created given the requested payment method:

Payment Method	Capture Context	Result
PAN Entry and Click to Pay	<code>completeMandate.tms.tokenCreate</code> = <code>true</code>	TMS token is created at the token level(s) specified in the request or based on the default for the token vault.

Payment Method	Capture Context	Result
	completeMandate.tms.tokenCreate = <code>true</code> and captureMandate.requestSaveCard = <code>true</code>	Cardholder can check Save Payment Information in Unified Checkout. The request to create a token is made when the cardholder checks this field in the UI. When it is not checked, no token is created.
Apple Pay, Google Pay, and Paze	completeMandate.tms.tokenCreate = <code>true</code>	TMS token is created at the token level(s) specified in the request or based on the default for the token vault.
	completeMandate.tms.tokenCreate = <code>true</code> and captureMandate.requestSaveCard = <code>true</code>	Unified Checkout cannot obtain consent to create a token and no token is created when the customer completes the payment.
Echeck	completeMandate.tms.tokenCreate = <code>true</code>	TMS token is created at the token level(s) specified in the request or based on the default for the token vault.
	completeMandate.tms.tokenCreate = <code>true</code> and captureMandate.requestSaveCard = <code>true</code>	Unified Checkout cannot obtain consent to create a token and no token is created when the customer completes the payment.

Include Card Prefix

You can control the length of the card number prefix to be received in the response to the capture context `/sessions` request:

- Six digits
- Eight digits
- No prefix

To specify your preferred card number prefix length, include or exclude the **transientTokenResponseOptions.includeCardPrefix** field in the capture context `/sessions` request.

To receive a six-digit card number prefix in the response, follow this step:

Do not include the **transientTokenResponseOptions.includeCardPrefix** field in the capture context `/sessions` request.

This example shows how a six-digit card number prefix `411111` is returned in the transient token response:

```
"maskedValue" : "XXXXXXXXXXXX1111",  
  "bin" : "411111"
```

To receive an eight-digit card number prefix in the response, follow this step:

Include the **transientTokenResponseOptions.includeCardPrefix** field in the capture context request, and set the value to `true`.



Important:

This PCI DSS requirement applies only to card numbers longer than 15 digits and only for Discover, JCB, Mastercard, UnionPay, and Visa brands.

- If the card type entered is not part of these brands, a six-digit card number prefix is returned instead.
- If the card type entered is not part of these brands but is *co-branded* with these brands, an eight-digit card number prefix is returned.

This example shows how an eight-digit card prefix `41111102` is returned in the transient token response:

```
"maskedValue" : "XXXXXXXXXXXX1111",  
  "prefix" : "41111102"
```

To not receive a card number prefix in the response, follow this step:

Include the **transientTokenResponseOptions.includeCardPrefix** field in the capture context request, and set the value to `false`.

This example shows how a card number is returned without a card number prefix in the transient token response:

```
"maskedValue" : "XXXXXXXXXXXX1111"
```

Best practice: If your application does not require card number prefix information for routing or identification, Cybersource recommends that you include the **transientTokenResponseOptions.includeCardPrefix** field in the capture context request and set its value to `false`. Doing so limits the exposure of payment data to only what is necessary for your processing needs.

For more information about PCI DSS, see [Frequently Asked Questions](#) on the PCI Security Standards Council site.



Important:

Cybersource recommends that you dynamically parse the response for the fields that you are looking for when you integrate with Cybersource APIs. Cybersource may add additional fields in the future.

You must ensure that your integration can handle new fields that are returned in the response. Even though the underlying data structures do not change, you must also ensure that your integration can handle changes to the order in which the data is returned. Cybersource uses semantic versioning practices, which enables you to retain backwards compatibility as new fields are introduced in minor version updates.

Features

Unified Checkout comprises these features.

Save Card

This feature enables you to display a consent option in the Unified Checkout UI for the cardholder to save their payment details for future use. If you use the complete mandate to create a token, see [Capture Context API \(on page 30\)](#).

When you use this field without using the complete mandate, the transient token payload includes the **consumerPreference.saveCard** field with the value set to `true` when the cardholder has checked to save the payment information for future purchases:

```
"captureMandate" : {  
  "requestSaveCard": true  
}
```

Email Autolookup

When you include Click to Pay as an **allowedPaymentType**, an automatic email lookup occurs when an email address is included in the capture context request. If the user has a Click to Pay account but is not on a recognized device, a one-time password (OTP) screen appears and the user is prompted to enter their OTP. If the user does not have a Click to Pay account, the user must enter their card information manually. They will have the option to create a Click to Pay account.

To enable email autolookup, you must include `CLICKTOPAY` as a value in the **allowedPaymentTypes** field and include an email address in the capture context.

Removal of Confirm and Continue Screen

When **showConfirmationStep** is set to `false`, you can remove the final summary confirmation screens from the checkout experience. When the UI displays cardholder data, the cardholder can review and, if necessary, edit their payment details before checkout is complete.

```
{
  "captureMandate": {
    "showConfirmationStep": false
  }
}
```

Click to Pay Enrollment Pre-Check

You can have the Click to Pay box pre-checked when a user is manually entering their card details and Click to Pay is enabled. The customer can uncheck the box if necessary, which means the request is processed as a one-time manual PAN transaction. This is available when you set the **billingType** field to `PARTIAL` or `FULL` in the capture context. This ensures that the customer's billing country can be validated in the UI.

Click to Pay enrollment pre-check is available in these countries:

- Argentina
- Brazil
- Chile
- Colombia
- Kuwait
- Mexico
- Peru
- Qatar
- Saudi Arabia

- South Africa
- Ukraine
- United Arab Emirates

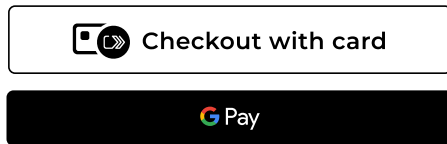
```
{
  "allowedPaymentTypes": [
    "PANENTRY",
    "GOOGLEPAY",
    {
      "type": "CLICKTOPAY",
      "options": {
        "autoCheckEnrollment": true
      }
    },
    "APPLEPAY",
    "PAZE"
  ]
}
```

Unified Checkout Checkout Button Name

When Unified Checkout loads, the payment buttons displayed are based on what you include in the **allowedPaymentTypes** object in the capture context. Unified Checkout enables you to customize the text on the payment buttons. You can do this by setting the **buttonType** field object in the capture context to one of these values:

- `ADD_CARD`
- `CARD_PAYMENT`
- `CHECKOUT_AND_CONTINUE`
- `DEBIT_CREDIT`
- `DONATE`
- `PAY`
- `PAY_WITH_CARD`
- `SUBSCRIBE_WITH_CARD`

If you do not include the **buttonType** field in your request, the payment button text defaults to **Checkout with card**. For example:



Mobile as Identity for Click to Pay

Click to Pay supports mobile numbers as way to identify a user. This enables cardholders to use their mobile number instead of their email address in certain markets for Visa and Mastercard transactions.

When the **requestEmail** field is set to `false` and the **requestPhone** field is set to `true`, the cardholder is identified using the provided mobile number. When the **requestEmail** field is set to `true` and the **requestPhone** field is set to `false`, the cardholder is identified using the provided email address. When the **requestEmail** field is set to `true` and the **requestPhone** field is also set to `true`, the cardholder is identified using the provided email address first and then the mobile number if there is no match.

Features Available in Brazil

These features are available only in Brazil:

Combo Cards

A combo card is a single card in Brazil that functions as both a debit and a credit card. Unified Checkout enables the cardholder to choose whether to pay for a transaction using a debit or credit card. The cardholder can choose the card that they want to use when they enter their card details or when they choose a stored Visa card from their Click to Pay wallet during checkout. While in the card details section of the payment form, the cardholder is prompted for a debit or credit card. Credit is the default option.

To enable combo cards during checkout, you must include the **comboCard** field in your capture context request and set the field value to `true`. When the **comboCard** field value is set to `true`, the option to use a debit or credit card appears for all Visa cards that are entered in Unified Checkout and for all cards that are already stored in Click to Pay. If you do not want to offer a combo card at checkout, do not include the **comboCard** field in your capture context request:

```
"captureMandate" : {  
  "comboCard": true  
}
```

Cadastro de Pessoas Físicas (CPF) – Brazilian Tax ID

The tax ID feature is for customers in Brazil and provides your customers with a way to include their Consumer National Identifier when it is requested at checkout. Include this field in the capture context to display this field within the flow for manual card entry and Click to Pay transactions:

```
"captureMandate" : {  
  "CPF": {  
    "required": true  
  }  
}
```

Client Version History

Below is a list of client versions and the features that are included in each version.



Important: Cybersource recommends that you use the most recent client version in your integration.

0.23

Accepts these card networks in the **allowedCardNetworks** field for manual card entry:

- Carnet
- Cartes Bancaires
- China UnionPay with card verification value (CVV)
- EFTPOS
- ELO
- JCrew PLCC
- mada
- Meeza

Ordering controls for the **allowedPaymentTypes** button.

De-coupling of PANENTRY from other payment types in the **allowedPaymentTypes** field.

0.24

Support for enabling combo cards in the capture context.

Support for eight-digit BINs.

Support for enabling card save in the capture context.

0.25

Addition of **Skip Verification next time** in the Click to Pay payment flow.

Support for CPF in the capture context.

0.26

Support for auto-lookup in Click to Pay when an email is included in the capture context.

Inclusion of the **cardDetails** field object in the transient token response.

Support for the **cardholderAuthenticationStatus** field object in the transient token response.

Support for the complete mandate.

0.28

Complete mandate enhancement to support Payer Authentication for manual card entry for Visa, Mastercard, American Express, Discover, JCB, Cartes Bancaires, China UnionPay, and ELO card brands.

Support for Afterpay as an **allowedPaymentType**.

Support for PayPak as an **allowedCardNetwork**.

Auto-enrollment for Click to Pay in supported markets.

Removal of the confirm or continue screen for specific use cases.

Static button for Click to Pay flows.

0.30

Support for iDeal, Multibanco, and Przelewy24|P24.

Complete mandate enhancement to support Payer Authentication for Google Pay and Click to Pay.

Support for Pakistan locales (en_PK and ur_PK).

New look and feel of Unified Checkout in line with EMVCO best practices.

0.31

Addition of the **data** object of the **orderInformation** field object and pass-through fields.

Support for **tokenCreate** in the complete mandate.

Support of pass-through fields, including challenge codes and data only, for Payer Authentication.

Support for Jaywan as an **allowedCardNetwork**.

Updated the payment details response to return detected card types. Multiple card types are shown when more than one card type is detected.

Support for Bancontact, Dragonpay, MyBank, and Tink Pay By Bank.

0.32

Support for KCP and UATP in the **allowedCardNetwork** field.

Support for Konbini as a post-pay reference payment method.

A radio button in the UI for Cartes Bancaires dual-branded cards.

0.33

Support for mobile as identity for Click to Pay accounts.

Japanese language translation updates.

UX captures billing and shipping information when they are not included in the capture context.

0.34

Iframes are used instead of pop-ups to reduce pop-up blocking and streamlining mobile deployment.

Additional BIN range for Jaywan card types.

Requesting the Capture Context

This section contains the information you need in order to request the capture context.

Endpoint

Production: POST <https://api.cybersource.com/up/v1/capture-contexts>

Test: POST <https://apitest.cybersource.com/up/v1/capture-contexts>

Required Fields for Requesting the Capture Context

Use these required fields to request the capture context:

allowedPaymentTypes

clientVersion

country

locale

data.orderInformation.amountDetails.currency

data.orderInformation.amountDetails.totalAmount

targetOrigins

The URL in this field value must contain [https](#).

type

Include this field when you use the complete mandate.

Related Information

- [API field reference guide for the REST API](#)

REST Example: Requesting the Capture Context

Request

```
{
  "targetOrigins" : [ "https://test.com" ],
  "clientVersion": "0.34",
  "buttonType": "CHECKOUT_AND_CONTINUE",
  "allowedCardNetworks": [
    "VISA",
    "MASTERCARD"
  ],
  "allowedPaymentTypes": [
    "CLICKTOPAY",
    "APPLEPAY",
    "GOOGLEPAY"
  ],
  "completeMandate": {
    "type": "CAPTURE",
```

```

    "decisionManager": true,
    "consumerAuthentication": true,
    "tms": {
      "tokenCreate": true,
      "tokenTypes": [
        "customer",
        "paymentInstrument",
        "instrumentIdentifier",
        "shippingAddress"
      ]
    }
  },
  "country": "US",
  "locale": "en_US",
  "captureMandate": {
    "billingType": "FULL",
    "requestEmail": true,
    "requestPhone": true,
    "requestShipping": true,
    "shipToCountries": [
      "US",
      "GB"
    ],
    "showAcceptedNetworkIcons": true
  },
  "data": {
    "orderInformation": {
      "billTo": {
        "country": "US",
        "firstName": "NEW",
        "lastName": "Test",
        "phoneNumber": "1234567890",
        "address2": "Desk M3-5573",
        "address1": "901 Metro Center Blvd",
        "buildingNumber": "150",
        "postalCode": "94404",
        "locality": "Foster City",
        "administrativeArea": "CA",
        "email": "test@example.com"
      },
      "shipTo": {
        "country": "US",
        "firstName": "NEW",
        "lastName": "Test",
        "address2": "Desk M3-5573",
        "address1": "901 Metro Center Blvd",
        "buildingNumber": "150",
        "postalCode": "94404",

```

```

    "locality": "Foster City",
    "administrativeArea": "CA"
  },
  "amountDetails": {
    "totalAmount": "13.00",
    "currency": "USD"
  }
},
"clientReferenceInformation": {
  "code": "TAGX001"
}
}
}

```

Encrypted JWT Response to a Successful Request

eyJrawQi0iJqNCIsImFsZyI6IjJTMjU2In0.eyJmbGgiOnsicGF0aCI6Ii9mbGV4L3YyL3Rva2VucyIsImNvbXBsZXRlUGF0aCI6Ii9mbGV4L3YyL2NvbXBsZXRlIiwizGF0YSI6ImY4TGhHRZFrMzdDMVl5UjM3YVh0QmhBQUVJWldjS1Yya29iUFA10VlubHVWUUhBZZldRwtiSlQ0bm1La2tWbEdCM3lhS1J4di83Uis3WwXvMLJsN2kvZXl4b1JHMnRxNkhyNDZFREh2a3FlcUgvOWQyS3R2bmZBK25QeFRFZlJRd2F4L2l4ZXNvK3ZkckM4MitITUvMvDM0QTRIwkJHbjBwODFlUowawVsc2RCdGpCOTJsnkZlL2cvRkR3S2ZueTY2aDZUbnJ0NTZ1YT hnb2p1YjUwRkVKaXpiMGi0S2NNVE4ydjVzWExXZlY5c21VXHUwMDNkICIsIm9yaWdpbiI6Imh0dHBz0i8v c3RhZ2VmbGV4LmN5YmVyc291cmNlLmNvbSIsImNvbXBsZXRlVXJsIjoiaHR0cHM6Ly9zdGFnZWZsZXguY3 liZXJzb3VyY2UuY29tL2ZsZXgvdjIvY29tcGxldGUiLCJhdXRoZW50aWNoZGlub1NldHVwUGF0aCI6Ii9m bGV4L3YyL2V4dGVybmfSLXNlc3Npb25zL3NjYSIsImp3ayI6eyJrdHkiOiJSU0EiLCJlIjoiQVFBQiIsIn VzZSI6ImVuYyIsIm4iOiJ1cU5KwRfZlNxr0o3UW8wTWxMU2lqZDJGaDdoaLdaRDk5N0ZnaHZEbEtfZVdt OG5TS1BZSHJxX0ctN3VmeVJ0QmM0aw8xb09NU3VqUE1VSwhsMWNzSXJ4bFJ4RDdDcC15Rn1TSncwRHROm0 1KNGpwZ1RQTXFLZkwyYi00UmQ1UXl0TFZlZWUZVT21Takl3STJLQk1ERnpGRk5WLRkcjlmS2ZnNXAzVTBz dmVZYXdtTl1DcmQzWXPxAG1xaU9TRFZrTlhIaGQ30WVBVndpdVJFWlN2T2NVZDR6dXd4RkFvX0N0MVV6aX FuLU9xdkw3UmZLLU9QaDg5N3FfWFFTUfNHYlUwZWltd24yZmpOVkYydnVoZXFDdmFDMmtIS2p0bFcZMVMt SHE0MEJUanNsbmUyMudCV3U0Ywc40HN5LTaxZnR5NzBjVnBXQnFpTVNfY3BY0Vh6VLEiLCJrawQi0iIwMD dQS1Z0VkJ5IVEhZnLJvazNFRUM3YktiNUNKbzJwUSJ9fSwiY3R4IjpbeyJkYXRhIjpw7ImFsbG93ZWRQYXlt ZW50VHlwZXMiOlslUEFORU5UULkiLCJDTelDS1RPUeFZiIiwizGF0YSI6ImY4TGhHRZFrMzdDMVl5UjM3YVh0QmhBQUVJWldjS1Yya29iUFA10VlubHVWUUhBZZldRwtiSlQ0bm1La2tWbEdCM3lhS1J4di83Uis3WwXvMLJsN2kvZXl4b1JHMnRxNkhyNDZFREh2a3FlcUgvOWQyS3R2bmZBK25QeFRFZlJRd2F4L2l4ZXNvK3ZkckM4MitITUvMvDM0QTRIwkJHbjBwODFlUowawVsc2RCdGpCOTJsnkZlL2cvRkR3S2ZueTY2aDZUbnJ0NTZ1YT hnb2p1YjUwRkVKaXpiMGi0S2NNVE4ydjVzWExXZlY5c21VXHUwMDNkICIsIm9yaWdpbiI6Imh0dHBz0i8v c3RhZ2VmbGV4LmN5YmVyc291cmNlLmNvbSIsImNvbXBsZXRlVXJsIjoiaHR0cHM6Ly9zdGFnZWZsZXguY3 liZXJzb3VyY2UuY29tL2ZsZXgvdjIvY29tcGxldGUiLCJhdXRoZW50aWNoZGlub1NldHVwUGF0aCI6Ii9m bGV4L3YyL2V4dGVybmfSLXNlc3Npb25zL3NjYSIsImp3ayI6eyJrdHkiOiJSU0EiLCJlIjoiQVFBQiIsIn VzZSI6ImVuYyIsIm4iOiJ1cU5KwRfZlNxr0o3UW8wTWxMU2lqZDJGaDdoaLdaRDk5N0ZnaHZEbEtfZVdt OG5TS1BZSHJxX0ctN3VmeVJ0QmM0aw8xb09NU3VqUE1VSwhsMWNzSXJ4bFJ4RDdDcC15Rn1TSncwRHROm0 1KNGpwZ1RQTXFLZkwyYi00UmQ1UXl0TFZlZWUZVT21Takl3STJLQk1ERnpGRk5WLRkcjlmS2ZnNXAzVTBz dmVZYXdtTl1DcmQzWXPxAG1xaU9TRFZrTlhIaGQ30WVBVndpdVJFWlN2T2NVZDR6dXd4RkFvX0N0MVV6aX FuLU9xdkw3UmZLLU9QaDg5N3FfWFFTUfNHYlUwZWltd24yZmpOVkYydnVoZXFDdmFDMmtIS2p0bFcZMVMt SHE0MEJUanNsbmUyMudCV3U0Ywc40HN5LTaxZnR5NzBjVnBXQnFpTVNfY3BY0Vh6VLEiLCJrawQi0iIwMD dQS1Z0VkJ5IVEhZnLJvazNFRUM3YktiNUNKbzJwUSJ9fSwiY3R4IjpbeyJkYXRhIjpw7ImFsbG93ZWRQYXlt ZW50VHlwZXMiOlslUEFORU5UULkiLCJDTelDS1RPUeFZiIiwizGF0YSI6ImY4TGhHRZFrMzdDMVl5UjM3YVh0QmhBQUVJWldjS1Yya29iUFA10VlubHVWUUhBZZldRwtiSlQ0bm1La2tWbEdCM3lhS1J4di83Uis3WwXvMLJsN2kvZXl4b1JHMnRxNkhyNDZFREh2a3FlcUgvOWQyS3R2bmZBK25QeFRFZlJRd2F4L2l4ZXNvK3ZkckM4MitITUvMvDM0QTRIwkJHbjBwODFlUowawVsc2RCdGpCOTJsnkZlL2cvRkR3S2ZueTY2aDZUbnJ0NTZ1YT hnb2p1YjUwRkVKaXpiMGi0S2NNVE4ydjVzWExXZlY5c21VXHUwMDNkICIsIm9yaWdpbiI6Imh0dHBz0i8v c3RhZ2VmbGV4LmN5YmVyc291cmNlLmNvbSIsImNvbXBsZXRlVXJsIjoiaHR0cHM6Ly9zdGFnZWZsZXguY3 liZXJzb3VyY2UuY29tL2ZsZXgvdjIvY29tcGxldGUiLCJhdXRoZW50aWNoZGlub1NldHVwUGF0aCI6Ii9m bGV4L3YyL2V4dGVybmfSLXNlc3Npb25zL3NjYSIsImp3ayI6eyJrdHkiOiJSU0EiLCJlIjoiQVFBQiIsIn VzZSI6ImVuYyIsIm4iOiJ1cU5KwRfZlNxr0o3UW8wTWxMU2lqZDJGaDdoaLdaRDk5N0ZnaHZEbEtfZVdt OG5TS1BZSHJxX0ctN3VmeVJ0QmM0aw8xb09NU3VqUE1VSwhsMWNzSXJ4bFJ4RDdDcC15Rn1TSncwRHROm0 1KNGpwZ1RQTXFLZkwyYi00UmQ1UXl0TFZlZWUZVT21Takl3STJLQk1ERnpGRk5WLRkcjlmS2ZnNXAzVTBz dmVZYXdtTl1DcmQzWXPxAG1xaU9TRFZrTlhIaGQ30WVBVndpdVJFWlN2T2NVZDR6dXd4RkFvX0N0MVV6aX FuLU9xdkw3UmZLLU9QaDg5N3FfWFFTUfNHYlUwZWltd24yZmpOVkYydnVoZXFDdmFDMmtIS2p0bFcZMVMt SHE0MEJUanNsbmUyMudCV3U0Ywc40HN5LTaxZnR5NzBjVnBXQnFpTVNfY3BY0Vh6VLEiLCJrawQi0iIwMD dQS1Z0VkJ5IVEhZnLJvazNFRUM3YktiNUNKbzJwUSJ9fSwiY3R4IjpbeyJkYXRhIjpw7ImFsbG93ZWRQYXlt ZW50VHlwZXMiOlslUEFORU5UULkiLCJDTelDS1RPUeFZiIiwizGF0YSI6ImY4TGhHRZFrMzdDMVl5UjM3YVh0QmhBQUVJWldjS1Yya29iUFA10VlubHVWUUhBZZldRwtiSlQ0bm1La2tWbEdCM3lhS1J4di83Uis3WwXvMLJsN2kvZXl4b1JHMnRxNkhyNDZFREh2a3FlcUgvOWQyS3R2bmZBK25QeFRFZlJRd2F4L2l4ZXNvK3ZkckM4MitITUvMvDM0QTRIwkJHbjBwODFlUowawVsc2RCdGpCOTJsnkZlL2cvRkR3S2ZueTY2aDZUbnJ0NTZ1YT hnb2p1YjUwRkVKaXpiMGi0S2NNVE4ydjVzWExXZlY5c21VXHUwMDNkICIsIm9yaWdpbiI6Imh0dHBz0i8v c3RhZ2VmbGV4LmN5YmVyc291cmNlLmNvbSIsImNvbXBsZXRlVXJsIjoiaHR0cHM6Ly9zdGFnZWZsZXguY3 liZXJzb3VyY2UuY29tL2ZsZXgvdjIvY29tcGxldGUiLCJhdXRoZW50aWNoZGlub1NldHVwUGF0aCI6Ii9m bGV4L3YyL2V4dGVybmfSLXNlc3Npb25zL3NjYSIsImp3ayI6eyJrdHkiOiJSU0EiLCJlIjoiQVFBQiIsIn VzZSI6ImVuYyIsIm4iOiJ1cU5KwRfZlNxr0o3UW8wTWxMU2lqZDJGaDdoaLdaRDk5N0ZnaHZEbEtfZVdt OG5TS1BZSHJxX0ctN3VmeVJ0QmM0aw8xb09NU3VqUE1VSwhsMWNzSXJ4bFJ4RDdDcC15Rn1TSncwRHROm0 1KNGpwZ1RQTXFLZkwyYi00UmQ1UXl0TFZlZWUZVT21Takl3STJLQk1ERnpGRk5WLRkcjlmS2ZnNXAzVTBz dmVZYXdtTl1DcmQzWXPxAG1xaU9TRFZrTlhIaGQ30WVBVndpdVJFWlN2T2NVZDR6dXd4RkFvX0N0MVV6aX FuLU9xdkw3UmZLLU9QaDg5N3FfWFFTUfNHYlUwZWltd24yZmpOVkYydnVoZXFDdmFDMmtIS2p0bFcZMVMt SHE0MEJUanNsbmUyMudCV3U0Ywc40HN5LTaxZnR5NzBjVnBXQnFpTVNfY3BY0Vh6VLEiLCJrawQi0iIwMD dQS1Z0VkJ5IVEhZnLJvazNFRUM3YktiNUNKbzJwUSJ9fSwiY3R4IjpbeyJkYXRhIjpw7ImFsbG93ZWRQYXlt ZW50VHlwZXMiOlslUEFORU5UULkiLCJDTelDS1RPUeFZiIiwizGF0YSI6ImY4TGhHRZFrMzdDMVl5UjM3YVh0QmhBQUVJWldjS1Yya29iUFA10VlubHVWUUhBZZldRwtiSlQ0bm1La2tWbEdCM3lhS1J4di83Uis3WwXvMLJsN2kvZXl4b1JHMnRxNkhyNDZFREh2a3FlcUgvOWQyS3R2bmZBK25QeFRFZlJRd2F4L2l4ZXNvK3ZkckM4MitITUvMvDM0QTRIwkJHbjBwODFlUowawVsc2RCdGpCOTJsnkZlL2cvRkR3S2ZueTY2aDZUbnJ0NTZ1YT hnb2p1YjUwRkVKaXpiMGi0S2NNVE4ydjVzWExXZlY5c21VXHUwMDNkICIsIm9yaWdpbiI6Imh0dHBz0i8v c3RhZ2VmbGV4LmN5YmVyc291cmNlLmNvbSIsImNvbXBsZXRlVXJsIjoiaHR0cHM6Ly9zdG

[illegible]


```
ctZXZlbnRzIiwiYXNzZXRzUGF0aCI6Ii91Yy92MS9hc3NldHMvYVgydWx4bjJqd01uNGhqQnhPaEthV2hf
aVJNV19jRjAzeE1TDlYYVU5aDUwXzR0bEFvdEp4bU9RYzlaalhweUg5OUFYU3g0eXhhc24xbU9xdHlFZD
BMSzFScjFENTBFdzdBZFJ4VjZCUkNwb0FKcXJUEtaYVM2NVk2VndWTEVaTVRPdm95NEcxQlpTTlEiLCJj
bGllbnRMAWJyYXJ5SW50ZWdyaXR5Ijoic2hhMjU2LWRYQ3BRRULdD1hnt09QR013NE4xSjcyd0hlZ2l4Z2
RZL1Y2azR2TG9nenNcdTAwM2QifSwidHlwZSI6ImdkYS0wLjEwLjAifV0sImIzcyI6IkZsZXggQVBjIiwi
ZXhwIjozNzYyODk0MzcxcjYXQiOjE3NjI4OTM0NzEsImp0aSI6IldkWWR4aDBLVUNtblhDeEIifQ.cWI
EtuT_CD4iiqMgFcTLCyV3zbyVxeCfLfj2E0Fh3J8sTE4tZDa0Bt9IWG-x3LWZ8pQb0g6JEZ-4FRoLpstU9
BWJfiPVZQURdXWLR8pnMLK38AUJI_WP85NYEeSsuaAZkh9elfr47agIB2dx1E3kwjk2IEkJt32WCvsctOr
ImNSibpkcA3LgfOTjGHVmj7NqVsbVyxEi3Zg_0D2SACNpKHj0z76yL6Y5JeMe807-43vHithNn_skoVbaR
zz3He17Ej497N8-g6EUBSmq8sQAQaI0QbBMCm0EXTH-aI0q2m193XoV2SR8KcN20g4e90NmspHtEXypaCI
qRU8ka2lT4A
```

Decrypted Capture Context Header

```
{
  "kid": "j4",
  "alg": "RS256"
}
```

Decrypted Capture Context Body with Selected Fields

```
{
  "flx" : {
    // filled with token metadata
  },
  "ctx" : [ {
    // filled with data related to your capture context request parameters
    "data" : {
      "clientLibrary" :
        "https://apitest.cybersource.com/up/v1/assets/0.34.0/SecureAcceptance.js"
    },
    "type" : "gda-0.9.0"
  } ],
  "iss" : "Flex API",
  "exp" : "1762894371",
  "iat" : "1762893471",
  "jti" : "WdYdxh0KUCmnXCxB"
}
```

Validating the Capture Context

The capture context that you generate is a JSON Web Token (JWT) data object. The JWT is digitally signed using a public key and confirms the validity of the JWT and that it comes from Cybersource. When you do not have a key in the JWT header, Cybersource recommends that you follow cryptography best practices and validate the capture context signature.

To validate a JWT, you must obtain its public key. This public RSA key is in JSON Web Key (JWK) format. The public key is associated with the capture context on the Cybersource domain.

To get the public key of a capture context from the header of the capture context itself, you must retrieve the key ID associated with the public key and then pass the key ID to the [/flex/v2/public-keys](#) endpoint:

1. From the header of the capture context, get the key ID (**kid**):

```
{
  "kid": "3g",
  "alg": "RS256"
}
```

2. Send a GET request to the [/flex/v2/public-keys](#) endpoint and include the key ID. For example:

- **Test:** GET <https://apitest.cybersource.com/flex/v2/public-keys/{3g}>
- **Production:** GET <https://api.cybersource.com/flex/v2/public-keys/{3g}>

Depending on the cryptographic method you use to validate the public key, you might need to convert the key to privacy-enhanced mail (PEM) format.

3. The resource returns the public key:

```
eyJmbHgiOnsicGF0aCI6Ii9mbGV4L3YyL3Rva2VucyIsImRhGEi0iI2bUFLNTNPNVpGTUk5Y3RobWZmd2doQUFFRGNqNU5QYzcxelErbm8reDN6WStLOTVWQ2c5bThmQWs4czlTRXBtT21zMmVhbEx5NkhHZ29oQ0JEWjVlN3ZUSGQ5YTR5a2tNRDLNVHhqK3ZowXVDUMRDadhVY1dwVUNZWlZnbTE1UXVFMKeiLCJvcmlnaW4iOiJodHRwcZovL3Rlc3RmbGV4LmN5YmVyc291cmNlLmNvbSIsImp3ayI6eyJrdHkiOiJSU0EiLCJlIjoibWYtMC4xMS4wIn1dLCJpc3MiOiJGbgV4IEFQSSIsImV4cCI6MTYxNjc3OTAMSwiaWF0IjoxNjE2Nzc4MTkxLCJqdGkiOiJ6SG1tZ25uaTVON3ptdGY0In0.GvBzyw6JKl3b2Pz
```



```
tHb9rZXawx2T817nYqu6goxpe4PsjqBY1qeTo19R-CP_DkJXov9hdJZgdlzlNmRY6yoiziSZnGJdp
nZ-pCqILC06qrpJVEDob30_efR9L03Gz7F5JLL0iTXSj6nVwC5mRlcP032ytPDEx5TMI9Y0hmBadJ
YnhEMwQnn_paMm3wLh2v6rfTkaBqd8n6rPvCNrWM0woMdoTeFyku-
```

Use this public RSA key to validate the capture context.

4. Parse the JWT capture context to get the **kid** from its header:

```
{
  "kid": "3g",
  "alg": "RS256"
}
```

5. Send a GET request to retrieve the public key from `/flex/v2/public-keys/3g`:

```
{
  "kty": "RSA",
  "use": "enc",
  "kid": "3g",
  "n": "ir7Nl1Bj8G9rxr3co5v_JLkP3o9UxXZRX1LIZFZeckguEf7Gdt5kGFFfTsymKBesm3Pe
8o1hwfkq7KmJZEZSuDbiJSZvFBZycK2pEeBjycahw9CqOweM7aKG2F_bhvwHrY4YdKsp
_cSJe_ZMXFUqYmjk7D0p7clX6CmR1QgMl41Ajb7NHI23u0WL7PyfJQwP1X8HdunE6ZwK
DNcavqx0W5VuW6nfsGvtygKQxjeHrI-gpyMXF0e_PeVpUIG0KVjmb5-em_Vd2SbyPNme
nADGJGCmECYMg5L5hEvnTuyAybwgVwuM9amyfFqIbRcrAIzclT4jQBeZFwkzZfQF7MgA6QQ",
  "e": "AQAB"
}
```

Transient Tokens

The response to a successful customer interaction with Unified Checkout is a transient token. This is returned in the response from the **unifiedPayment.show()** function. The transient token is a reference to the payment data collected on your behalf. Transient tokens allow secure card payments to occur without risk of exposure to sensitive payment information. The transient token is a short-term token that expires after 15 minutes. This reduces your PCI burden/responsibility and ensures that sensitive information is not exposed to your back-end systems.

Transient tokens can be included requests sent to the Payment Details API for the customer payment data that is collected.

Transient Token Format

The transient token is issued as a JSON Web Token (JWT) ([RFC 7519](#)). For information on JSON Web Tokens, see [JSON Web Tokens \(on page 136\)](#).

The payload portion of the token is a Base64URL-encoded JSON string and contains various claims. For more information, see [JSON Web Tokens \(on page 136\)](#).

Example: Transient Token Format

Transient Token Payload

```
{
  "metadata" : {
    "sequenceNumber" : "1",
    "cardholderAuthenticationStatus" : false,
    "paymentType" : "PANENTRY"
  },
  "iss" : "Flex/00",
  "exp" : 1762870464,
  "type" : "gda-0.10.0",
  "iat" : 1762869564,
  "jti" : "1D4Q8FJSSZ9ASKQ9ZCJ7E13IF0IT00H2GGHY6TRZ3028TUQ1BN8H691344C098CA",
  "content" : {
    "deviceInformation" : {
      "fingerprintSessionId" : { }
    },
    "orderInformation" : {
      "billTo" : {
```

```

    "country" : { },
    "lastName" : { },
    "firstName" : { },
    "phoneNumber" : { },
    "address1" : { },
    "postalCode" : { },
    "locality" : { },
    "buildingNumber" : { },
    "company" : {
      "name" : { }
    },
    "administrativeArea" : { },
    "email" : { }
  },
  "amountDetails" : {
    "totalAmount" : { },
    "currency" : { }
  },
  "shipTo" : {
    "firstName" : { },
    "lastName" : { },
    "country" : { },
    "address1" : { },
    "postalCode" : { },
    "locality" : { },
    "buildingNumber" : { },
    "administrativeArea" : { }
  }
},
"paymentInformation" : {
  "card" : {
    "expirationYear" : {
      "value" : "2027"
    },
    "number" : {
      "maskedValue" : "XXXXXXXXXXXX1111",
      "bin" : "411111"
    },
    "securityCode" : { },
    "expirationMonth" : {
      "value" : "03"
    },
    "typeSelectionIndicator" : {
      "value" : "1"
    },
    "type" : {
      "value" : "001"
    }
  }
}

```

```
}  
}  
}
```



Important: The empty field values in the transient token indicate which fields were captured by the application without exposing you to personally identifiable information directly.

PAN BIN in `metadata` Object

The `cardDetails` object, including the PAN BIN, is included in the transient token `metadata` when a Click to Pay network token is used as a payment method. This allows you to display information about the card on invoices and see the BIN details that are linked to the underlying card.

```
"metadata": {  
  "cardDetails": {  
    "suffix": "9876",  
    "prefix": "123456",  
    "expirationMonth": "MM",  
    "expirationYear": "YYYY"  
  }  
}
```

Authentication Status in `metadata` Object

The `cardholderAuthenticationStatus` object is included in the `metadata` and enables you to determine if the payload is fully authenticated. When `cardholderAuthenticationStatus` is set to `true`, the payload is fully authenticated. When `cardholderAuthenticationStatus` is set to `false`, the transaction is not authenticated.

If you are using Unified Checkout with `unifiedPayment.complete()` and `consumerAuthentication` is set to `true` in the complete mandate request, then Payer Authentication is called automatically if it is available for the selected payment method and card network. If you use a transient token to request follow-on services directly, the value of this field indicates if the transaction has been authenticated.

```
"metadata": {  
  "cardholderAuthenticationStatus": "true"  
}
```

Token Verification

When you receive the transient token, you should cryptographically verify its integrity using the public key embedded within the capture context. Doing so verifies that Cybersource issued the token and that the data has not been tampered with in transit. Verifying the transient token JWT involves verifying the signature and various claims within the token. Programming languages each have their own specific libraries to assist. For an example in Java, see: [Java Example in Github](#).

Dual-Branded Cards

Unified Checkout accepts dual-branded cards. To use this feature, you must include the card networks that have overlapping BIN ranges in the capture context request. For example:

```
"allowedCardNetworks": ["VISA", "MASTERCARD", "AMEX", "CARTESBANCAIRES"]
```

When a card number within an overlapping BIN range is entered, the network that is listed first in the value array for the **allowedCardNetworks** field is used. Based on the previous example, if the card number 403550XXXXXXXXXX is entered, the payment network for payment processing is Visa.

During the transaction, the card type is populated with the first network in the list, and the **detectedCardTypes** field returned in the transient token includes all of the detected card types in the transient token.

The **detectedCardTypes** field is returned in the transient token response only when more than one card type is detected.

If you include Cartes Bancaires as a supported dual-branded card type, Unified Checkout displays a radio button with Visa and Mastercard options at checkout. This enables the customer to select which payment scheme they want to use to process the payment. The radio button defaults to the card type that you specify in the capture context request, but the payment is processed using the option selected by the customer during checkout.

Authorizations with a Transient Token

This section provides the information required in order to perform a successful authorization with a Unified Checkout transient token. You can use this method to construct more complex payment scenarios that are not supported by the `unifiedPayment.complete()` payment method.



Important: When you process payments through Unified Checkout using `unifiedPayments.complete()`, Unified Checkout invokes service orchestration directly. When you send an authorization request using a transient token, you must request the follow-on services that you want to use. For information about the required fields for the payment services that you request, see the [Payments Developer Guide](#).

The transient token is a short-term token that expires after 15 minutes. Doing so eliminates the need to send sensitive payment data along with the request. For more information on transient tokens, see [Transient Tokens \(on page 56\)](#).

To send the transient token with a request, use the `tokenInformation.transientTokenJwt` field.

This example shows a transient token in the context of an authorization request:

```
"tokenInformation": {
  "transientTokenJwt":
    "eyJraWQiOiIwOG4zUnVsRTJGQXJDRktycVRkZFkwGZSWFhMNXFOhNSIsImFsZyI6IlJTMjU2In0.eyJp
    c3MiOiJGbgV4LzA3IiwiaXhwIjoxNTk3MDg0ODk3LCJ0eXBlijoiz2RhLTAuMS4xIiwiaWF0IjoxNTk3MD
    gzOTk3LCJqdGkiOiIxQzI2VlpSkvJUUIPTzVIMDUwNEtINDdJMEFNmklaRkM0M1Y1TDU0MUhCTE45Q09JM
    0w3NUYzMtK0RTE5NkExIn0.SNm1VZaZr3DkTqUg9CdV0F5arRe-uQU9oUWPKfWIpbIzIPZutRokv5DSDcM
    7asZIKNJyNIBx5DLsl_yQPrKgzhwQxZ8qbhto7cu3t-v8DHG2y0951pLPQVQnj7x-vEDcXkLUL1F8sqY23
    R5HW-xSDAQ3AFLawCckn7Q2eudRGeuMhLWH742Gflf9Hz3KyKnmeNKA3o9yW2na16nmeVZaYgqbUSPVITd
    l5cMA0o9lEob8E30QH0HHdmIsu5uMA4x7DeBjftKD1rQxFP3JBNVcv30AIMLkNcw0pHbtHDVzKBWxUVxvn
    m3zFEdiBuSAco2uWhC9zFqHrrp64ZvzxZqoGA"
}
```

To retrieve non-sensitive data from a Unified Checkout transient token, use the `payment-details` endpoint. This data includes cardholder name and billing and shipping details. For more information, see [Payment Details API \(on page 63\)](#).



Important: Fields supplied directly in an API request supersede those that are also present in the transient token. For example, in the request below, the total amount might have been overridden because of a tax calculation.

Endpoint

Production: POST <https://api.cybersource.com/pts/v2/payments>

Test: POST <https://apitest.cybersource.com/pts/v2/payments>

Required Field for an Authorization with a Transient Token

tokenInformation.transientTokenJwt

REST Example: Requesting an Authorization with a Transient Token

```
{
  "clientReferenceInformation": {
    "code": "TC50171_3"
  },
  "processingInformation": {
    "commerceIndicator": "internet"
  },
  "tokenInformation": {
    "transientTokenJwt":
    "eyJraWQiOiIwOG4zUnVsRTJGQXJDRktycVRkZFlkWGZSWFhMNXFoNSIsImFs
    ZyI6IlJTMjU2In0.eyJpc3MiOiJGbGV4LzA3IiwiaXhwIjoxNTk3MDg0ODk3LCJ0eXBli
    joiZ2RhLTAuMS4xIiwiaWF0IjoxNTk3MDgzOTk3LCJqdGkiOiIxQzI2VlpsSRkVJU
    U1PTzVIMDUwNEtINDdJMEFNMklaRkM0M1Y1TDU0MUhCTE45Q09JM0w3NUYzMTk0
    RTE5NkExIn0.SNm1VZaZr3DkTqUg9CdV0F5arRe-uQU9oUWPKfWIpbIzIPZutR
    okv5DSDcM7asZIKNJyNIBx5DLsL_yQPrKgzhwQxZ8qbhto7cu3t-v8DHG2y0951p
    lPQVQnj7x-vEDcXkLUL1F8sqY23R5HW-xSDAQ3AFLawCckn7Q2eudRGeuMhLWH742G
    flf9Hz3KyKnmeNKA3o9yW2na16nmeVZaYGqbUSPVITdl5cMA0o9lEob8E30QH0
    HHdmIsu5uMA4x7DeBjfTKD1rQxFP3JBNVcv30AIMLkNcw0pHbtHDVzKBWxUVxv
    nm3zFEdiBuSAco2uWhC9zFqHrrp64ZvzxZqoGA"
  },
  "orderInformation": {
    "amountDetails": {
      "totalAmount": "21.00",
      "currency": "USD"
    },
    "billTo": {
      "firstName": "John",
```

```
    "lastName": "Doe",  
    "address1": "1Market St",  
    "address2": "Address 2",  
    "locality": "san francisco",  
    "administrativeArea": "CA",  
    "postalCode": "94105",  
    "country": "US",  
    "email": "test@cybs.com",  
    "phoneNumber": "4158880000"  
  }  
}  
}
```


Payment Details API

This section contains the information you need to retrieve the non-sensitive data associated with a Unified Checkout transient token and the payment details API. This API can be used to retrieve personally identifiable information, such as the cardholder name and billing and shipping details, without retrieving payment credentials, which helps ease the PCI compliance burden.

There are two methods of authentication, and they are described in the *Getting Started with REST Developer Guide*:

- [Set Up a JSON Web Token Message](#)
- [Set Up HTTP Signature Message](#)



Important:

Cybersource recommends that you dynamically parse the response for the fields that you are looking for when you integrate with Cybersource APIs. Cybersource may add additional fields in the future.

You must ensure that your integration can handle new fields that are returned in the response. Even though the underlying data structures do not change, you must also ensure that your integration can handle changes to the order in which the data is returned. Cybersource uses semantic versioning practices, which enables you to retain backwards compatibility as new fields are introduced in minor version updates.

Endpoint

Production: `GET https://api.cybersource.com/up/v1/payment-details/{id}`

Test: `GET https://apitest.cybersource.com/up/v1/payment-details/{id}`

The `{id}` is the full JWT received from Unified Checkout as the result of capturing payment information. The transient token is a JWT object that you retrieved as part of a successful capture of payment information from a cardholder.

REST Example: Retrieving Transient Token Payment Details

Request

```
GET https://apitest.cybersource.com/up/v1/payment-details/{id}
```

Response to Successful Request

```
{
  "paymentInformation": {
    "card": {
      "expirationYear": "2026",
      "number": "XXXXXXXXXXXX1111",
      "expirationMonth": "05",
      "type": "001"
    }
  },
  "orderInformation": {
    "amountDetails": {
      "totalAmount": "21.00",
      "currency": "USD"
    },
    "billTo": {
      "lastName": "Lee",
      "country": "US",
      "firstName": "Tanya",
      "email": "tanyalee@example.com"
    },
    "shipTo": {
      "locality": "Small Town",
      "country": "US",
      "administrativeArea": "CA",
      "address1": "123 Main Street",
      "postalCode": "98765"
    }
  }
}
```

JavaScript API Reference

This reference provides details about the JavaScript API for creating the Unified Checkout payment form.

Class: Accept

Accept

Returns

Type: Promise.<Accept>

Example

Basic Setup

```
<script src="[INSERT clientLibrary VALUE HERE]" integrity="[INSERT  
  clientLibraryIntegrity VALUE HERE]" crossorigin="anonymous"></script>  
//Note: Script location and integrity value should be sourced from the capture  
  context response clientLibrary and clientLibraryIntegrity values.  
<script> Accept('header.payload.signature').then(function(accept) {  
  // use accept object  
});  
</script>
```

Methods

`dispose()` → {void}

Dispose of this Accept instance.

Returns

Type: void

`unifiedPayments(sidebar)` → {Promise.<UnifiedPayments>}

Create a Unified Payments integration.

Parameters

Name	Type	Attributes	Description
sidebar	Boolean	<optional>	Set the option to <code>false</code> to enable embedded functionality of Unified Checkout. This will configure Unified Checkout to place the Payment Entry form inline. If this value is not set, the default is <code>true</code> and Unified Checkout will open the Payment Entry form in the sidebar configuration.

Throws: `AcceptError`

Returns:

Type: `Promise.<UnifiedPayments>`

Examples

Minimal Setup - sidebar

```
const captureContext = document.getElementById('captureContext').value;

Accept(captureContext)
  .then(accept => accept.unifiedPayments())
```

Embedded Payment Entry

```
const captureContext = document.getElementById('captureContext').value;

Accept(captureContext)
  .then(accept => accept.unifiedPayments(false))
```

Error Handling

```
const captureContext = document.getElementById('captureContext').value;

Accept(captureContext)
  .then(accept => accept.unifiedPayments())
  .then(up => up.show(showArgs))
  .then(tt => {
    document.getElementById('transientToken').value = tt;
    document.getElementById("authForm").submit();
  })
  .catch(error => {
    console.error(error);
  })
```

```
document.getElementById('logo').text = `Checkout error:
${JSON.stringify(error)}. Try again.`;
});
```

Class: AcceptError

AcceptError

This class defines how errors are returned by the Unified Checkout JavaScript.

Members

(static, readonly) Reason Codes - Accept object creation

Possible errors that can occur during the creation of an Accept object.

Properties:

Name	Type	Description
CAPTURE_CONTEXT_INVALID	string	Occurs when you pass an invalid JWT.
CAPTURE_CONTEXT_EXPIRED	string	Occurs when the JWT you pass has expired.
SDK_XHR_ERROR	string	Occurs when a network error is encountered while attempting to load the SDK.

(static, readonly) Reason Codes - Show Errors

Possible errors that can occur during the rendering of payment selection list.

Properties:

Name	Type	Description
CHECKOUT_ERROR	string	Occurs when checkout failed to load.
CLICK_TO_PAY_SDK_LOAD_ERROR	string	Occurs when the Click to Pay SDK fails to load.
ENCRYPT_CARD_FOR_SRC_ENROLMENT_ERROR	string	Occurs when the card encryption for SRC enrollment fails to load.
LAUNCH_SRC_CHECKOUT_ERROR	string	Occurs when the SRC checkout fails to load.

Properties: (continued)

Name	Type	Description
SHOW_LOAD_CONTAINER_SELECTOR	string	Occurs when a DOM element cannot be located using the supplied CSS Selector string.
SHOW_LOAD_ERROR	string	Occurs when there is an issue loading the payment iframe.
SHOW_LOAD_INVALID_CONTAINER	string	Occurs when an invalid container parameter is supplied.
SHOW_LOAD_SIDEBAR_OPTIONS	string	Occurs when an invalid container parameter is supplied when sidebar is selected.
SHOW_PAYMENT_TIMEOUT	string	Occurs when an error is encountered during the handling of a payment option.
SHOW_PAYMENT_UNAVAILABLE	string	Occurs when no payment types can be presented to the customer.
SHOW_TOKEN_TIMEOUT	string	Occurs when the createToken call is unable to proceed.
SHOW_TOKEN_XHR_ERROR	string	Occurs when a network error is encountered while attempting to create a token.
UNIFIED_PAYMENTS_ALREADY_SHOWN	string	Occurs when you attempt to show a Unified Payments instance multiple times.
UNKNOWN_ERROR	string	Occurs when an unknown error has occurred.

(static, readonly) Reason Codes - Unified Payments Errors

Possible errors that can occur during the creation of a Unified Payments object.

Properties:

Name	Type	Description
CREATE_TOKEN_TIMEOUT	string	Occurs when the createToken call times out.
CREATE_TOKEN_XHR_ERROR	string	Occurs when a network error is encountered while attempting to create a token.

Properties: (continued)

Name	Type	Description
UNIFIED_PAYMENTS_PAYMENT_PARAMETERS	string	Occurs when no valid payment parameters exist while initializing button.
UNIFIED_PAYMENTS_VALIDATION_PARAMS	string	Occurs when there's an issue with the parameters supplied to UnifiedPayments constructor.

`(nullable) correlationId :string`

The **correlationId** of any underlying API call that resulted in this error.

Type: string

`(nullable) details :array`

Additional error-specific information.

Type: array

`(nullable) informationLink :string`

A URL link to online documentation for this error.

Type: string

`message :string`

A human-readable explanation of the error that has occurred.

Type: string

`reason :string`

A reason corresponding to the specific error that has occurred.

Type: string

`(static, readonly) Reason Codes - Complete API Response Errors`

Possible errors that can occur when calling the complete API.

Properties:

Name	Type	Description
COMPLETE_AUTHENTICATION_CANCELED	string	Occurs when the user cancels the authentication process during Cardinal step-up.

Properties: (continued)

Name	Type	Description
COMPLETE_AUTHENTICATION_FAILED	string	Occurs when the complete authentication process fails during Cardinal step-up.
COMPLETE_ERROR	string	Occurs when an error occurs while attempting to process a payment using the complete API.
COMPLETE_IN_PROGRESS	string	Occurs when complete has already been invoked but has not yet finished processing.
COMPLETE_NOT_ALLOWED	string	Occurs if complete is not allowed for this transaction.
COMPLETE_TRANSACTION_CANCELLED	string	Occurs when consumer cancelled the transaction.
COMPLETE_TRANSACTION_FAILED	string	Occurs when consumer transaction fails.
COMPLETE_VALIDATION_ERROR	string	Occurs when there is a validation issue relating to the parameters you have supplied in your complete call.

Class: UnifiedPayments

UnifiedPayments

An instance of this class is returned upon the creation of a Unified Payments integration using **accept.unifiedPayments()**. Using this object you can add the payment options list to your checkout.

Methods

`hide() → {Promise}`

Hide button list.

Returns: Type Promise

Example

Basic Usage


```
up.hide()
  .then(() => console.log('Hidden'))
  .catch(err => console.error(err));
```

`show(optionsopt) → {Promise.<UnifiedPayments~TransientToken>}`

Show button list.

Parameters

Name	Type	Attributes	Description
options	object	<optional>	
containers	object	<optional>	CSS selectors to locate containers in which to place various UI elements. If not specified, these will operate in a sidebar.
payment Selection	string	<optional>	For showing payment buttons.
payment Screen	string	<optional>	For the main payment flows.

Returns: Type Promise

Examples

Basic Usage With Full Sidebar Experience

```
const showArgs = {
  containers: {
    paymentSelection: '#buttonPaymentListContainer'
  }
};

up.show(showArgs).then(transientToken => console.log(transientToken));
```

All Screens Embedded in Containers

```
const showArgs = {
  containers: {
    paymentSelection: '#buttonPaymentListContainer',
    paymentScreen: '#embeddedPaymentContainer'
  }
};
```

```
up.show(showArgs).then(transientToken => console.log(transientToken));
```

Type Definitions

TransientToken

The response to a successful customer interaction with Unified Checkout is a transient token. The transient token is a reference to the payment data collected on your behalf. Tokens allow secure card payments to occur without risk of exposure to sensitive payment information. The transient token is a short-term token that lasts 15 minutes. This reduces your PCI burden and responsibility and ensures that sensitive information is not exposed to your backend systems.

It is in a JSON Web Token format. The payload of the transient token may contain useful metadata in relation to the stored sensitive info. However, all of this info is safe to use and store on your systems.

The transient token can be used to complete a payment or other services, after which the transient data will be evicted from the token store.

Type: string

Examples

How to Split the Transient Token

```
const transientToken = 'hhhhhhhhh.pppppppppp.ssssssssss';

const segments = transientToken.split('.');
const urlBase64Decode = (s) => atob(s.replace(/_/g, '/').replace(/-/g, '+'));

const header = JSON.parse(urlBase64Decode(segments[0]));
const payload = JSON.parse(urlBase64Decode(segments[1]));
const signature = segments[2];
```

Decoded Body

```
{
  "iss" : "Flex/00",
  "exp" : 1706910242,
  "type" : "gda-0.9.0",
  "iat" : 1706909347,
  "jti" : "1D1I202CSTMW3UIX0KEQFI40QX1L7CMSKDE3LJ8B5DVZ6WBJGKLQ65BD6222D426",
  "content" : {
    "orderInformation" : {
      "billTo" : {
```

```

        // Empty fields present within this node indicate which fields were
        captured by
        // the application without exposing you to personally identifiable
        information
        // directly.
    },
    "amountDetails" : {
        // Empty fields present within this node indicate which fields were
        captured by
        // the application without exposing you to personally identifiable
        information
        // directly.
    },
    "shipTo" : {
        // Empty fields present within this node indicate which fields were
        captured by
        // the application without exposing you to personally identifiable
        information
        // directly.
    }
},
"paymentInformation" : {
    "card" : {
        "expirationYear" : {
            "value" : "2028"
        },
        "number" : {
            "maskedValue" : "XXXXXXXXXXXX1111",
            "bin" : "411111"
        },
        "securityCode" : { },
        "expirationMonth" : {
            "value" : "06"
        },
        "type" : {
            "value" : "001"
        }
    }
}
}
}
}

```

Unified Checkout Configuration

This section contains information necessary to configure Unified Checkout in the Business Center:

- [Enable Digital Payments \(on page 79\)](#)
- [Manage Permissions \(on page 90\)](#)

Webhooks Support

Unified Checkout supports webhooks. You can use webhooks to obtain the complete response from the **completeMandate** call. To receive a webhook notification, you must first subscribe to the webhook. For information on setting up a webhook, see the [How to Set Up Webhook Subscriptions](#) section of the *Webhooks Developer Guide*.



Important: Webhook payloads are encrypted. In order to receive a Unified Checkout webhook notification, you must enable message-level encryption (MLE). For information about enabling MLE, see [Enable Message-Level Encryption](#) in the *Getting Started with REST Developer Guide*.

Unified Checkout Webhook Events

Product ID	Event Types	Description
<code>unifiedCheckout</code>	<code>uc.orders.transactionresults</code>	Full payload response from the payment service call made by Unified Checkout

Example: Webhooks Request for Unified Checkout Events

```
{
  "organizationId": "ps_hpa",
  "webhookId": "2d55e648-d96c-d727-e063-3cb8d30a938e",
  "productId": "unifiedCheckout",
  "requestType": "NEW",
  "payload": {
    "details": {
      "_links": {
        "authReversal": {
          "method": "POST",
          "href": "/pts/v2/payments/7435188899356405003091/reversals"
        },
        "self": {
          "method": "GET",
          "href": "/pts/v2/payments/7435188899356405003091"
        },
        "capture": {
          "method": "POST",
          "href": "/pts/v2/payments/7435188899356405003091/captures"
        }
      }
    },
    "clientReferenceInformation": {
      "code": "1743518889914",
      "transactionId": "123456789ABCDEFGHIJKLMNOPQRST1"
    }
  },
}
```

```

    "consumerAuthenticationInformation": {
      "token":
        "Axj/7wSTkw/5TqVCHfNTABeg1auGjlgybQ3FmNXmxaSiXV63TywFRLq9bp5ekDpxBPjhk23GVpz0tI7g
        Tk5MP+U6lQh3zUwA0TVW"
    },
    "id": "7435188899356405003091",
    "orderInformation": {
      "amountDetails": {
        "authorizedAmount": "100.00",
        "currency": "USD"
      }
    },
    "paymentAccountInformation": {
      "card": {
        "type": "001"
      }
    },
    "paymentInformation": {
      "tokenizedCard": {
        "type": "001"
      },
      "scheme": "VISA DEBIT",
      "bin": "411111",
      "accountType": "Visa Classic",
      "issuer": "CONOTOXIA SP. Z 0.0",
      "card": {
        "type": "001"
      },
      "binCountry": "PL"
    },
    "pointOfSaleInformation": {
      "terminalId": "092940"
    },
    "processorInformation": {
      "merchantNumber": "000000012345678",
      "approvalCode": "888888",
      "networkTransactionId": "123456789619999",
      "transactionId": "123456789619999",
      "responseCode": "100",
      "avs": {
        "code": "X",
        "codeRaw": "I1"
      }
    },
    "reconciliationId": "55849026C8YFWMER",
    "riskInformation": {
      "localTime": "7:48:10",
      "score": {
        "result": "0",

```

```

        "factorCodes": [
            "B",
            "Q",
            "Y"
        ],
        "modelUsed": "default"
    },
    "infoCodes": {
        "address": [
            "INTL-SA",
            "MM-A",
            "MM-BIN",
            "MM-C",
            "MM-CO",
            "MM-ST",
            "MM-Z",
            "UNV-ADDR"
        ],
        "phone": [
            "RISK-PH",
            "UNV-OC"
        ],
        "globalVelocity": [
            "VELV-SA"
        ],
        "suspicious": [
            "INTL-BIN",
            "RISK-SD"
        ],
        "identityChange": [
            "MORPH-B",
            "MORPH-P",
            "MORPH-S"
        ]
    },
    "profile": {
        "earlyDecision": "ACCEPT",
        "name": "Rob Demo",
        "selectorRule": "Rob"
    },
    "casePriority": "1"
},
"status": "AUTHORIZED",
"submitTimeUtc": "2025-04-01T14:48:11Z"
},
"id": "7435188899356405003091",
"message": "Request processed successfully.",
"outcome": "AUTHORIZED",
"status": "AUTHORIZED"

```

```
},  
  "retryNumber": 0,  
  "eventType": "uc.orders.transactionresults",  
  "eventDate": "2025-03-27T08:44:55",  
  "transactionTraceId": "62dbbee0-c722-4ee0-b60c-1c2a62aff90e"  
}
```


Enable Digital Payments

To enable digital payments on Unified Checkout, you must first register for each digital payment method that you would like enabled on your page. This enablement process sends the appropriate information to the digital payment systems and registers your page with each system.

This section contains information about enabling and managing digital payments on Unified Checkout in the Business Center:

- Apple Pay
 - Enrolling in Apple Pay
 - Preparing a device for testing Apple Pay on Unified Checkout
- Click to Pay
 - Enabling Click to Pay
- Google Pay
 - Enrolling in Google Pay
 - Managing Google Pay authentication types

Enrolling in Apple Pay

Apple Pay is a digital payment service that enables users to make secure and convenient transactions using their Apple devices. Users can add their credit or debit cards to the Wallet app and use them to pay online or in apps in a safe and convenient consumer experience.

To enable Apple Pay you must first host a public certificate on your web page and then pass your merchant name and domain name to Apple. Apple crawls out to your web page to validate the presence of this certificate to ensure the web pages are properly vetted and registered with Apple.

Follow these steps to validate your domain and enroll in Apple Pay:

1. Navigate to **Payment Configuration > Unified Checkout**.
2. In the Apple Pay section, click **Set Up**.
3. Follow the link to download the certificate.
4. Upload the *apple-developer-merchantid-domain-association* certificate file to your web server at:

[/.well-known/apple-developer-merchantid-domain-association](#)

You must verify that the file is accessible through HTTPS. You can validate this by visiting <https://<your-domain>/.well-known/apple-developer-merchantid-domain-association>.

5. Click **Verify Domain**.

6. Enter the domain name where you are hosting Apple Pay. This must be the same domain to which you uploaded the public certificate.
Your domain is now verified for Apple Pay.



Important: In order to run an end-to-end test of the Apple Pay service on Unified Checkout, you must perform additional setup steps. See [Preparing a Device for Testing Apple Pay on Unified Checkout \(on page 80\)](#).

Preparing a Device for Testing Apple Pay on Unified Checkout

To run an end-to-end test of the Apple Pay service on Unified Checkout, you must prepare an Apple test device by loading Apple Pay test cards onto the device.

Follow these steps to prepare your Apple test device for end-to-end testing:

1. Make sure your Apple Developer account is configured for Apple Pay.
2. Register your Apple Pay test device with Apple.
3. Load Apple Pay test cards onto your Apple test device.
The Apple Developer center provides the instructions in the [Sandbox Testing](#) page for Apple Pay:
 - a. Follow the steps described in *Create a Sandbox Tester Account*.
 - b. Follow the steps described in *Adding a Test Card Number*.

Enabling Click to Pay

Click to Pay is a digital payment solution that allows customers to pay with their preferred card network and issuer without entering their card details on every website. Customers can use Visa, Mastercard, and American Express cards to streamline their purchase experience. Click to Pay provides a fast, secure, and consistent checkout experience across devices and browsers.

Follow these steps to enable in Click to Pay on Unified Checkout:

1. Navigate to **Payment Configuration > Unified Checkout**.
2. In the Click to Pay section, click **Set Up**.
3. Enter your business name and website URL.
4. Click **Submit**.
5. Contact your implementation contact or technical account manager to request that you be enabled for tokenization within Click to Pay. Your implementation contact or technical account manager will confirm that you were configured successfully and that you can now accept digital payments with Click to Pay.



Important: Click to Pay uses network tokenization for transactions. These network tokens are stored in the vault of the token requestor ID (TRID) for the card scheme.

Set Up Customer Authentication for Visa Click to Pay

Follow these steps to use the Business Center to enable customer authentication through Click to Pay. Authentication methods differ in each region and are dependent on the issuer, the cardholder device, and the Click to Pay configuration. These authentication methods are available:

- 3-D Secure
- FIDO
- Card verification value (CVV)
- One-time password (OTP)



Important: After you complete these steps, Visa determines which authentication method to use. When Visa determines that they will authenticate, they authenticate each Click to Pay transaction through the appropriate method. This may be a frictionless authentication or the customer may need to provide more information when required by the issuer. This is available only through Visa.



Important: Visa Click to Pay customer authentication is not the same as Payer Authentication using the complete mandate. See [Test Authentication \(on page 185\)](#).

1. Log in to the Business Center:

Test URL: <https://businesscentertest.cybersource.com/ebc2>

Production URL: <https://businesscenter.cybersource.com>

If you are unable to access this page, contact your sales representative.

2. In the Business Center, go to the left navigation panel and choose **Payment Configuration > Unified Checkout**.

You must have Click to Pay enabled as a digital payment method in order to use this method of authentication. Click **Manage** to view the digital payment methods that you have enabled.

Payment Configuration

Unified Checkout

Digital Payment Solutions

ENABLED

You have set up and enabled one or more digital payment solutions. You can manage your payment solution enablement here. Set up additional services or modify existing payment solutions configuration. These payment solutions are available through the integrated Unified Checkout product. Easily enable digital payment options for acceptance within your webpage.



Manage

If Click to Pay is not enabled, click **On** next to Click to Pay.

Digital Payment Solutions

Available Solutions

Enable Digital Payment options for your Unified Checkout Integration. Digital payments offer a friction free way for your consumers to pay.

Select an Option

Off

On ✓

Click to Pay

Enable Click to Pay for your Unified Checkout Integration. Click to Pay provides a fast, simple, digital checkout experience that reduces consumer friction, which can help to lower cart abandonment and increase conversions.



- Click **Set up** under Value Added Solutions. The Value Added Solutions page appears.

Value Added Solutions

Available Solutions

Enable additional value-added services within the Unified Checkout product. These services enhance transactions and provide extra capabilities to the payment acceptance process.



3DS

NOT SET UP

Enable 3DS and biometric authentication within your Click to Pay experience.

Set up

- Click **Set up** to set up 3-D Secure. The 3DS page appears.
- Enter the required information in the Merchant Details section. You must enter the information that is provided to you by your acquirer or processor.

3DS

Merchant Details

Fields marked with a * are required.

Acquirer Merchant ID as assigned to you by your acquiring entity.

Acquirer Merchant ID *

Merchant Name as registered by your acquirer.

Merchant Name *

Acquirer Bank Identification Number(BIN) *

Note: Enablement of this process may result in additional costs associated with invoking 3DS.

Save **Cancel**

This completes the authentication setup for the entered acquirer merchant ID and BIN. If you do not know what these values are, you must contact your acquirer. Completing this information enables Cybersource to send Visa the information that is required for authentication.



Important: Charges for 3-D Secure may apply. You must speak with your acquirer for more information about the charges associated with 3-D Secure.

Click to Pay Customer Authentication

When you enable customer authentication through Click to Pay, you give Cybersource permission to send Visa the required authentication information for each transaction. When the customer completes a transaction using a Visa card that is already stored in Click to Pay, authentication is managed with Click to Pay. When the customer checks out using manual card entry and does not save their card to Click to Pay, the transaction is not processed through Click to Pay and you must complete authentication based on your existing authentication method.

Click to Pay authentication is only available for Visa branded cards that are tokenized with Click to Pay. If Click to Pay does not authenticate the transaction, but you are using the complete mandate with the **consumerAuthentication** field set to `true`, authentication is attempted as part of this request. When you do not use the complete mandate, you must check the result of the **cardholderAuthenticationStatus** field in the transient token and request Payer Authentication directly when it is required.



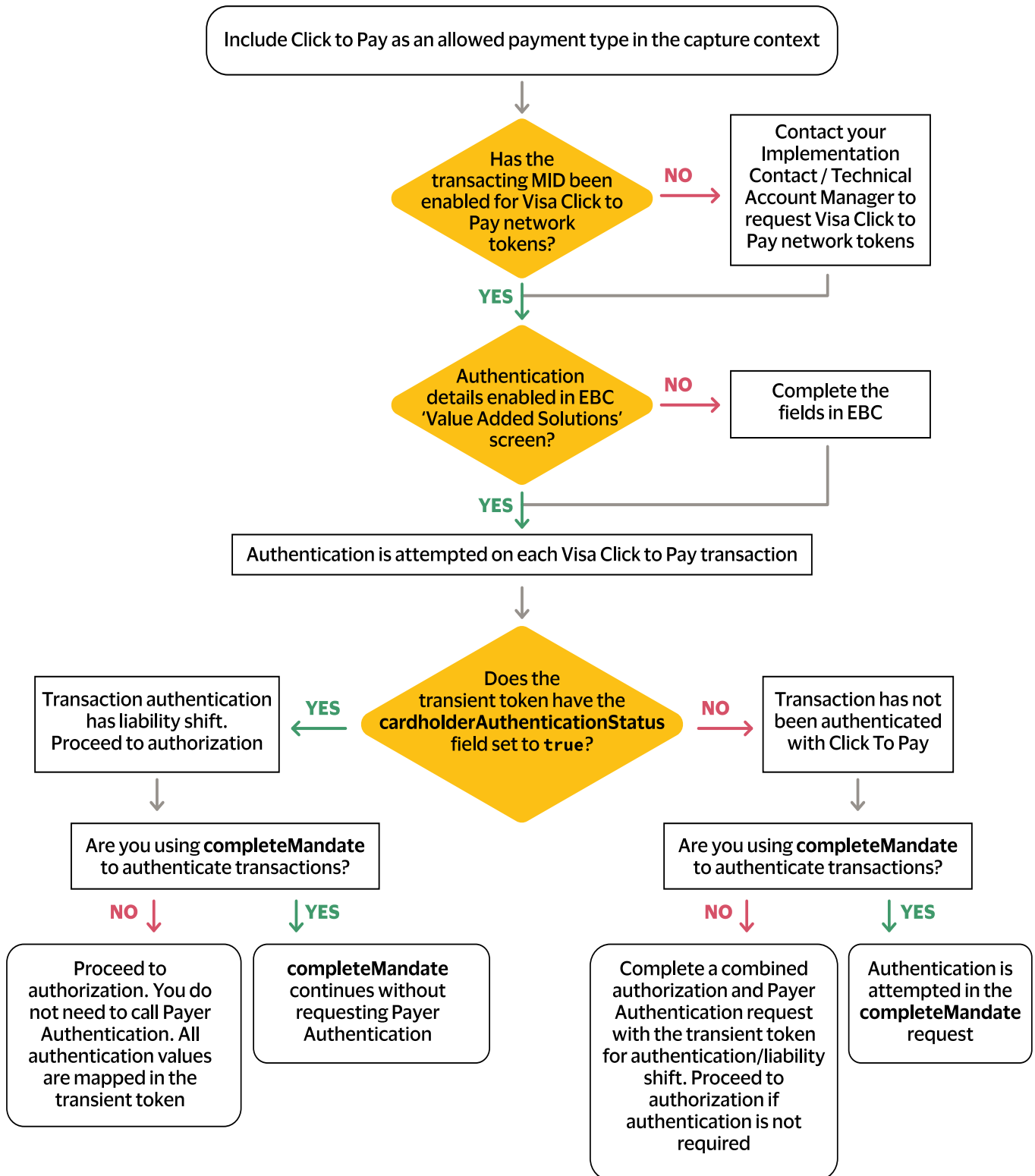
Important: American Express and Mastercard card brands cannot be authenticated through Click to Pay customer authentication.

Visa Prerequisites

Before you can begin customer authentication using Click to Pay, you must meet these requirements:

- The **allowPaymentTypes** field must include `CLICKTOPAY` in the capture context. For more information, see [Capture Context \(on page 17\)](#).
- The transacting merchant ID that sends the transaction requests must be configured for tokenization with Click to Pay. You must contact your implementation consultant or technical account manager to configure tokenization with Click to Pay.
- Set up Visa customer authentication in the Business Center. For information, see [Set Up Customer Authentication for Visa Click to Pay \(on page 81\)](#).

Authentication Flow



Enrolling in Google Pay

Google Pay is a digital payment product offered by Google through Chrome browsers and Android devices.

Follow these steps to enroll in Google Pay on Unified Checkout:

1. Navigate to **Payment Configuration > Unified Checkout**.
2. In the Google Pay section, click **Set Up**.
3. Enter your business name.
4. Click **Submit**.

You can now accept digital payments with Google Pay.



Important: When you enable Google Pay on Unified Checkout, you can specify an optional parameter that defines the types of credentials that Google Pay sends you. See [Managing Google Pay Authentication Types \(on page 87\)](#).

Managing Google Pay Authentication Types

Additional controls are available for Google Pay on Unified Checkout. When you enable Google Pay on Unified Checkout, you can specify optional parameters that define the types of card authentication you receive from Google Pay.

To manage the types of credentials that Google Pay sends, use this expanded payment type object within the **allowedPaymentTypes** section of the sessions request:

```
{
  "type": "GOOGLEPAY",
  "options": { "allowedAuthMethods": "<authentication type>"
}
}
```

The expanded payment type object has these parameters:

- **type:** Defines the type of payment option.
- **options:** Contains specific payment types parameters.

For Google Pay, use the new data element **allowedAuthMethods** within the **options** section of the payment types object to specify the authentication type you will receive from Google Pay. Possible values:

- **PAN_ONLY**: Google returns primary account number (PAN) values
- **CRYPTOGRAM_3DS**: Google returns fully authenticated network token values.

By default, Google sends both authentication types.

When the complete mandate is used and Google Pay does not authenticate the transaction, then Unified Checkout completes the authentication request as part of the complete mandate.

REST Example: Specify Only PAN Authentication Accepted from Google

This sessions request example specifies that Google Pay is to send only PAN values.

```
"allowedPaymentTypes": [
  "PANENTRY" {
    "type": "GOOGLEPAY",
    "options": {
      "allowedAuthMethods": "PAN_ONLY"
    }
  },
  "CLICKTOPAY",
  "PAZE"
]
```

REST Example: Simple Google Pay Request

This sessions request example specifies that Google Pay can send all authentication types.

```
"allowedPaymentTypes": [
  "PANENTRY",
  "GOOGLEPAY",
  "CLICKTOPAY",
  "PAZE",
  "CHECK"
]
```

Enrolling in Echeck/ACH Services

Unified Checkout can accept bank account payments using the eCheck product. To accept eCheck payments through Unified Checkout, you must have the eCheck processing service enabled. To request access to eCheck processing and enable eCheck, you must submit an application in the Business Center. Once your application is approved, you can accept eCheck payments.

For step-by-step instructions on enrolling and enabling eCheck, see the “Getting Started with the eCheck Service” section of the [eCheck Merchant Guide](#). If eCheck is not listed in the Available Products section in the Business Center, you must contact your portfolio owner to enable your account to apply for eCheck.



Important: If you have a business account or a financial relationship with Bank of America, Wells Fargo, or Chase, and you would like them to process your transactions, you must contact our Sales or Support team for more information on our ACH product.

Enrolling in Alternative Payment Methods

Before you can enroll in alternative payment methods on Unified Checkout, you must first be enabled for the alternative payment platform. Contact your portfolio administrator for more information.

Manage Permissions

Portfolio administrators can set permissions for new or existing Business Center user roles for Unified Checkout. Administrators retain full read and write permissions. They enable you to regulate access to specific pages and specify who can access, view, or amend digital products within Unified Checkout.

Portfolio administrators must apply the appropriate user role permission for any existing or newly created Business Center user roles for Unified Checkout. For information on managing permissions as a portfolio administrator, see [Managing Permissions as a Portfolio Administrator \(on page 92\)](#).

If you are a transacting merchant, you might find that your permissions are restricted. If your permissions are restricted, a message appears indicating that you do not have access, or buttons might appear gray. To make changes to your digital products within Unified Checkout that have restricted permissions, contact your portfolio administrator's customer support representative. For more information, see [Managing Permissions as a Direct Merchant \(on page 91\)](#).

Managing Permissions as a Direct Merchant

Follow these steps to configure and manage user permissions in the Business Center for Unified Checkout as a direct merchant:

1. On the left navigation panel, navigate to **Account Management**.
2. Click **Roles** to display a list of your user roles.
3. Click the pencil icon next to the user role that you want to update.
4. Click **Payment Configuration Permission**.
5. Select the relevant permission for the specific user role you are editing. You can select from these Unified Checkout permissions:
 - Unified Checkout View
 - Unified Checkout Manage



Important:

If you are a transacting merchant without view permissions, Unified Checkout will still appear on the navigation bar, however, a *no access* message appears when you access Unified Checkout.

If you are a transacting merchant with view permissions but not management permissions, you can access the Unified Checkout screens and view the different payment methods configurations, however, you cannot edit or enroll new products.

Managing Permissions as a Portfolio Administrator

Follow these steps to configure and manage user permissions in the Business Center for Unified Checkout as a portfolio administrator:

1. On the left navigation panel, navigate to **Account Management**.
2. Click **Roles** to see a list of your user roles.
3. Click the pencil icon next to the user role that you want to update.
4. Click **Payment Configuration Permission**.
5. Select the relevant permission for the specific user role you are editing. You can choose from these Unified Checkout permissions:
 - Unified Checkout View
 - Unified Checkout Manage
 - Unified Checkout Portfolio View (available for portfolio users only)
 - Unified Checkout Portfolio Manage (available for portfolio users only)



Important:

If all permissions are left unselected, the user has restricted permission. A *no access* message appears when the user tries to access the Unified Checkout digital product enablement pages. The user is advised to contact a customer representative.

If a portfolio user has view permissions and does not have a management role, they can access the Unified Checkout pages, but they cannot modify toggles for different digital payments.

Test Your Unified Checkout Configuration

This section contains information about testing your Unified Checkout configuration.

Test Payment Details

Use these test card numbers to test your Unified Checkout configuration.

Combine the BIN with the card number when sending to Unified Checkout.

Unified Checkout Test Cards

To test Payer Authentication, you must use Payer Authentication test cards. See [Test Cases for 3-D Secure 2.x](#) in the *Payer Authentication Developer Guide*.

Test Card Numbers

Card Brand	BIN	Card Number	Expiration Date	CVV
Visa	411111	1111111111	12/2025	123
Mastercard	555555	5555554444	02/2026	265
American Express	378282	246310005	03/2026	7890
Cartes Bancaires	436000	0001000005	04/2040	123
Carnet	506221	0000000009	04/2024	123
China UnionPay	627988	6248094966	04/2040	123
Diners Club	305693	09025904	04/2040	123
Discover	644564	4564456445	04/2040	123
JCB	353011	13333 0000	04/2040	123
Jaywan	679009	0000002009	04/2040	123
Paypak	220543	0000003002	04/2040	123
Maestro	675964	9826438453	04/2040	123
mada	446404	0000000007	04/2040	123
ELO	451416	0000000003	04/2040	123
JCrew	515997	1500000005	04/2040	123
EFTPOS	401795	000000000009	04/2040	123
Meeza	507808	3000000002	04/2040	123

Visa Click to Pay Test Cards

These Visa test cards can be added to your Click to Pay wallet. Replace the X in the card number with 0.

You can manage your Visa Click to Pay test cards and account here:

- **Production:** <https://src.visa.com/login>
- **Test:** <https://sandbox.src.visa.com/login>

To manage Visa test cards for customer authentication, contact your implementation consultant or technical account manager.



Important: These test cards are not valid for testing in production. To test in production, you must leverage production credentials.

Visa Test Card Numbers

Card Number	Expiration Date	CVV
46229431231X2700	12/2026	938
46229431231X2718	12/2026	605
46229431231X2726	12/2026	579
46229431231X2734	12/2026	141
46229431231X2742	12/2026	279
46229431231X2759	12/2026	669
439584XXX449X11X	12/2025	509
439584XXX282X11X	12/2025	693

Use these test cards to test when you use a Click to Pay card with authentication performed outside Click to Pay and the **consumerAuthentication** field is set to [true](#) in the capture context.

Test Card Numbers for Authentication Outside Click to Pay Flow

Card Brand	Card Number	Expiration Date	CVV
Visa	46229431231X2X56	12/2026	432
	46229431231X232X	12/2026	581
Mastercard	512X35X1XXX64578	Any future date	Any
	512X35X1XXX64552	Any future date	Any

Use these cards when authentication is performed by Click to Pay within the Click to Pay flow.

Click to Pay Test Card Numbers for Authentication in Click to Pay Flow

Card Brand	Card Number	Expiration Date	CVV
Visa	43958XXX0449X11X	12/2025	509
	439584XXX282X11X	12/2025	693
	439584XX91X1XX11	12/2025	676
	439584XX9119XX11	12/2025	789

For information about testing authentication, see [Test Authentication \(on page 185\)](#). For information about enabling Visa customer authentication, see [Set Up Customer Authentication for Visa Click to Pay \(on page 81\)](#).

Mastercard Click to Pay Test Cards

Mastercard test cards can be added to your Click to Pay wallet. You must retrieve Mastercard test cards from their Click to Pay test page: https://developer.mastercard.com/mastercard-checkout-solutions/documentation/testing/test_cases/click_to_pay_case/#test-cards

Mastercard has different test cards for retrieving tokenized and non-tokenized data. Cybersource recommends that you use these test cards as follows:

- Test cards to retrieve PAN data: Use these cards when the customer is completing checkout as a one-time guest and does not have a Click to Pay account or want to create one.
- Test cards to retrieve token data: Use these cards for tokenized Click to Pay transactions.

You can manage your Mastercard Click to Pay test cards and account here:

- **Live:** <https://src.mastercard.com/profile/enroll>
- **SBX:** <https://sandbox.src.mastercard.com/profile/enroll>

To manage Mastercard test cards for customer authentication, contact your implementation consultant or technical account manager.

Echeck Test Values

These eCheck test values can be used to process a test eCheck transactions:

- **Routing number:** Set to 071923284
- **Account number:** Set to any supported value. For example, 1234567890.

Payment Methods

This section describes the payment methods you can use in your Unified Checkout integration. After you successfully integrate one payment method, you can add another from the same category with minimal adjustments to your existing configuration.

These payment methods are available:

- [Cards \(on page 99\)](#)
- [Online Bank Transfers \(on page 103\)](#)
- [Buy Now, Pay Later \(on page 115\)](#)
- [Post-Pay Reference Payments \(on page 119\)](#)

Cards

Unified Checkout accepts multiple card types including global networks such as Visa, Mastercard, and American Express. Unified Checkout also accepts local schemes such as Cartes Bancaires in France, EFTPOS in Australia, and PayPak in Pakistan.

Card Support

Support for card brands varies based on the payment method for these services:

- Payments
- Decision Manager
- Payer Authentication

This table shows which card types are accepted for each payment method and which region:

Card Brand by Region and Payment Method

Region	Card Brand	Manual Card Entry	Apple Pay	Click to Pay	Google Pay	Paze
Asia Pacific	China UnionPay	✓	✗	✗	✗	✗
Asia Pacific	EFTPOS	✓	✗	✗	✗	✗
Asia Pacific	JCB	✓	✗	✗	✗	✗
CEMEA	mada	✓	✗	✗	✗	✗
CEMEA	Meeza	✓	✗	✗	✗	✗
CEMEA	Jaywan	✓	✗	✗	✗	✗
CEMEA	PayPak	✓	✗	✗	✗	✗
Europe	Cartes Bancaires	✓	✗	✗	✗	✗
Global	American Express	✓	✓	✓	✓	✗
Global	Diners Club	✓	✗	✗	✗	✗
Global	Mastercard	✓	✓	✓	✓	✓
Global	Visa	✓	✓	✓	✓	✓

Card Brand by Region and Payment Method (continued)

Region	Card Brand	Manual Card Entry	Apple Pay	Click to Pay	Google Pay	Paze
Global and Europe	Maestro	✓	✗	✗	✗	✗
Latin America	Carnet	✓	✗	✗	✗	✗
Latin America	ELO	✓	✗	✗	✗	✗
US and Canada	Discover	✓	✗	✗	✗	✗
US and Canada	JCrew	✓	✗	✗	✗	✗

Card Brand by Region and Payment Method

Region	Card Brand	Manual Card Entry	Apple Pay	Click to Pay	Google Pay	Paze
Asia Pacific	China UnionPay	✓	✗	✗	✗	✗
Asia Pacific	EFTPOS	✓	✗	✗	✗	✗
Asia Pacific	JCB	✓	✗	✗	✗	✗
CEMEA	mada	✓	✗	✗	✗	✗
CEMEA	Meeza	✓	✗	✗	✗	✗
CEMEA	Jaywan	✓	✗	✗	✗	✗
CEMEA	PayPak	✓	✗	✗	✗	✗
Europe	Cartes Bancaires	✓	✗	✗	✗	✗
Global	American Express	✓	✓	✓	✓	✗
Global	Diners Club	✓	✗	✗	✗	✗
Global	Mastercard	✓	✓	✓	✓	✓
Global	Visa	✓	✓	✓	✓	✓

Card Brand by Region and Payment Method (continued)

Region	Card Brand	Manual Card Entry	Apple Pay	Click to Pay	Google Pay	Paze
Global and Europe	Maestro	✓	✗	✗	✗	✗
Latin America	Carnet	✓	✗	✗	✗	✗
Latin America	ELO	✓	✗	✗	✗	✗
US and Canada	Discover	✓	✗	✗	✗	✗
US and Canada	JCrew	✓	✗	✗	✗	✗

This table shows which card types are supported for each complete mandate feature by region.

Card Support for the Complete Mandate

Region	Card Brand	Authorization	Payer Authentication	Decision Manager	Token Create by Token Management Service
Asia Pacific	China UnionPay	✓	✓	✓	✓
Asia Pacific	EFTPOS	✓	✗	✓	✓
Asia Pacific	JCB	✓	✓	✓	✗
CEMEA	mada	✓	✗	✓	✓
CEMEA	Meeza	✓	✗	✓	✗
CEMEA	Jaywan	✓	✗	✓	✗
CEMEA	PayPak	✓	✗	✓	✗
Europe	Cartes Bancaires	✓	✓	✓	✓
Global	American Express	✓	✓	✓	✓
Global	Diners Club	✓	✗	✓	✓

Card Support for the Complete Mandate (continued)

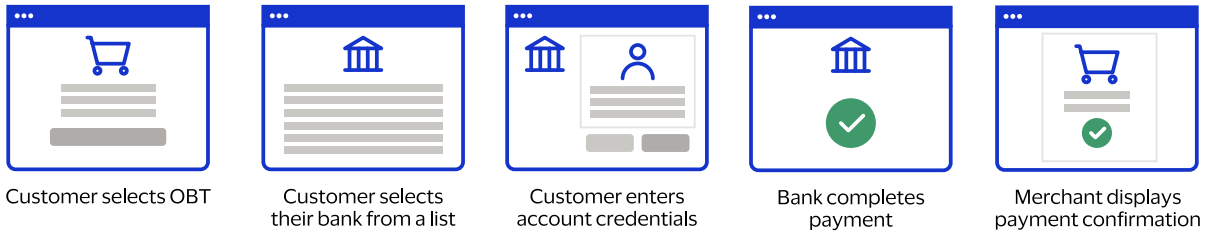
Region	Card Brand	Authorization	Payer Authentication	Decision Manager	Token Create by Token Management Service
Global	Mastercard	✓	✓	✓	✓
Global	Visa	✓	✓	✓	✓
Global and Europe	Maestro	✓	✓	✓	✓
Latin America	Carnet	✓	✗	✓	✓
Latin America	ELO	✓	✓	✓	✓
US and Canada	Discover	✓	✓	✓	✓
US and Canada	JCrew	✓	✗	✓	✓

Online Bank Transfers

Online bank transfers enable customers to complete their purchase by securely logging into their online banking environment. This method is secure, trusted, and widely used in many European countries.

This is how online bank transfers work:

Online Bank Transfers



1. The customer chooses online bank transfer as their payment method during checkout.
2. The customer chooses their bank from the list of available banks and is redirected to their bank's website or application where they are prompted to enter their account credentials.
3. The customer confirms their payment and completes the authorization process.
4. The customer is notified that the payment is complete.
5. The customer returns to your website for payment confirmation.

Unified Checkout supports these online bank transfer payment methods:

- Bancontact
- DragonPay
- iDeal
- Multibanco
- MyBank
- Przelewy24|P24
- Tink Pay By Bank

Online Bank Transfer Payment Methods Enabled by PPRO

Payment Method	Method	Capture Context allowed Payment Types	Capture Context completeMandate.type	Separate Capture?	Payment Confirmation	Customer Country (Country Code)	Customer Currency
iDEAL	PPRO	IDEAL	CAPTURE or PREFER_AUTH	No	Immediate	Netherlands (NL)	EUR
Multibanco	PPRO	MULTIBANCO	CAPTURE or PREFER_AUTH	No	Immediate	Portugal (PT)	EUR
Przelewy 24 P24	PPRO	P24	CAPTURE or PREFER_AUTH	No	Immediate	Poland (PL)	PLN
Bancontact	PPRO	BANCONTACT	CAPTURE or PREFER_AUTH	No	Immediate	Belgium (BE)	EUR
MyBank	PPRO	MYBANK	CAPTURE or PREFER_AUTH	No	Immediate	Italy (IT)	EUR
						Belgium (BE)	
						Portugal (PT)	
						Spain (ES)	
Dragon Pay	PPRO	DRAGONPAY	CAPTURE or PREFER_AUTH	No	Immediate	Philippines (PH)	PHP

Online Bank Transfer Payment Methods Not Enabled by PPRO

Payment Method	Capture Context allowed Payment Types	Capture Context completeMandate.type	Separate Capture?	Payment Confirmation	Customer Country (Country Code)	Customer Currency
Tink Pay by Bank	TINKPAYBYBANK	CAPTURE or PREFER_AUTH	No	Immediate	United Kingdom (GB)	GBP

Bancontact

Bancontact enables customers to make secure online and in-store purchases directly from their bank accounts. Bancontact is a leading payment method in Belgium.

When the total amount of the order is outside the range of accepted transaction amounts, the Bancontact payment button is not displayed in Unified Checkout. These are the accepted transaction amounts:

- **Minimum transaction amount:** EUR 0.01
- **Maximum transaction amount:** Not applicable

Opt in to Bancontact on Unified Checkout

Follow these steps to opt in to Bancontact on Unified Checkout:

1. Add Bancontact to your integration by adding `BANCONTACT` to the **allowedPaymentTypes** field object within the capture context request:
2. Set the **completeMandate.type** field value to `CAPTURE` or `PREFER_AUTH`.

You can capture the funds immediately if you include the **completeMandate.type** field in the capture context request and set the value to `CAPTURE`.

If you support more than one payment type and must perform an authorization where funds are collected at a later time, set the **completeMandate.type** field to `PREFER_AUTH`. The funds will be captured immediately for the online bank transfer.

3. Include these required fields for online bank transfers in the capture context request:
 - **orderInformation.billTo.country**
 - **orderInformation.billTo.firstName**
 - **orderInformation.billTo.lastName**
4. Include this optional field for online bank transfers in the capture context request:
 - **orderInformation.billTo.address1**
 - **orderInformation.billTo.email**

DragonPay

DragonPay provides Filipino customers and businesses with a secure payment channel that does not require customers to be banked or have a credit card. Customers can make purchases online and pay by bank transfer.

When the total amount of the order is outside the range of accepted transaction amounts, the DragonPay payment button is not displayed in Unified Checkout. These are the accepted transaction amounts:

- **Minimum transaction amount:** PHP 50.01
- **Maximum transaction amount:** Not applicable

Opt in to DragonPay on Unified Checkout

Follow these steps to opt in to Multibanco on Unified Checkout:

1. Add DragonPay to your integration by adding `DRAGONPAY` to the **allowedPaymentTypes** field object within the capture context request:
2. Set the **completeMandate.type** field value to `CAPTURE` or `PREFER_AUTH`.

You can capture the funds immediately if you include the **completeMandate.type** field in the capture context request and set the value to `CAPTURE`.

If you accept more than one payment type and must perform an authorization where funds are collected at a later time, set the **completeMandate.type** field to `PREFER_AUTH`. The funds are captured immediately for the online bank transfer.

3. Include these required fields for online bank transfers in the capture context request:
 - **orderInformation.billTo.country**
 - **orderInformation.billTo.firstName**
 - **orderInformation.billTo.lastName**
4. Include this optional field for online bank transfers in the capture context request:
 - **orderInformation.billTo.address1**
 - **orderInformation.billTo.email**

iDeal

iDEAL enables customers to pay online through their mobile banking app or online bank account and provides you with a payment guarantee. iDEAL supports these banks:

- ABN AMRO
- ASN Bank
- bunq
- ING
- Knab
- Rabobank
- RegioBank
- Revolut
- SNS
- Svenska Handelsbanken
- Triodos Bank
- Van Lanschot

When the total amount of the order is outside the range of accepted transaction amounts, the iDeal payment button is not displayed in Unified Checkout. These are the accepted transaction amounts:

- **Minimum transaction amount:** EUR 0.01
- **Maximum transaction amount:** Subject to transaction approval from the customer's account.

Opt in to iDeal on Unified Checkout

Follow these steps to opt in to iDeal on Unified Checkout:

1. Add iDeal to your integration by adding `IDEAL` to the **allowedPaymentTypes** field object within the capture context request:
2. Set the **completeMandate.type** field value to `CAPTURE` or `PREFER_AUTH`.

You can capture the funds immediately if you include the **completeMandate.type** field in the capture context request and set the value to `CAPTURE`.

If you accept more than one payment type and must perform an authorization where funds are collected at a later time, set the **completeMandate.type** field to [PREFER_AUTH](#). The funds are captured immediately for the online bank transfer.

3. Include these required fields for online bank transfers in the capture context request:

- **orderInformation.billTo.country**
- **orderInformation.billTo.firstName**
- **orderInformation.billTo.lastName**

4. Include this optional field for online bank transfers in the capture context request:

- **orderInformation.billTo.address1**
- **orderInformation.billTo.email**

MyBank

MyBank enables customers to pay for their online purchases in an easy and safe way using real-time bank transfers. MyBank customers complete payments by selecting their bank and logging in with their online banking credentials.

When the total amount of the order is outside the range of accepted transaction amounts, the MyBank payment button is not displayed in Unified Checkout. These are the accepted transaction amounts:

- **Minimum transaction amount:** EUR 0.01
- **Maximum transaction amount:** EUR 999,999,999.99

Opt in to MyBank on Unified Checkout

Follow these steps to opt in to MyBank on Unified Checkout:

1. Add MyBank to your integration by adding [MYBANK](#) to the **allowedPaymentTypes** field object within the capture context request:
2. Set the **completeMandate.type** field value to [CAPTURE](#) or [PREFER_AUTH](#).

You can capture the funds immediately if you include the **completeMandate.type** field in the capture context request and set the value to [CAPTURE](#).

If you accept more than one payment type and must perform an authorization where funds are collected at a later time, set the **completeMandate.type** field to `PREFER_AUTH`. The funds are captured immediately for the online bank transfer.

3. Include these required fields for online bank transfers in the capture context request:

- **orderInformation.billTo.country**
- **orderInformation.billTo.firstName**
- **orderInformation.billTo.lastName**

4. Include this optional field for online bank transfers in the capture context request:

- **orderInformation.billTo.address1**
- **orderInformation.billTo.email**

Multibanco

Multibanco enables customers to pay for a range of goods and services by bank transfer. These services include e-commerce, licenses, and taxes post-purchase. Multibanco is supported by all banks in Portugal.

When the total amount of the order is outside the range of accepted transaction amounts, the Multibanco payment button is not displayed in Unified Checkout. These are the accepted transaction amounts:

- **Minimum transaction amount:** No minimum
- **Maximum transaction amount:** EUR 99,999

Opt in to Multibanco on Unified Checkout

Follow these steps to opt in to Multibanco on Unified Checkout:

1. Add Multibanco to your integration by adding `MULTIBANCO` to the **allowedPaymentTypes** field object within the capture context request:
2. Set the **completeMandate.type** field value to `CAPTURE` or `PREFER_AUTH`.

You can capture the funds immediately if you include the **completeMandate.type** field in the capture context request and set the value to `CAPTURE`.

If you accept more than one payment type and must perform an authorization where funds are collected at a later time, set the **completeMandate.type** field to [PREFER_AUTH](#). The funds are captured immediately for the online bank transfer.

3. Include these required fields for online bank transfers in the capture context request:

- **orderInformation.billTo.country**
- **orderInformation.billTo.firstName**
- **orderInformation.billTo.lastName**

4. Include this optional field for online bank transfers in the capture context request:

- **orderInformation.billTo.address1**
- **orderInformation.billTo.email**

Przelewy24|P24

Przelewy24, or P24, is a Poland-based real-time online bank transfer payment method. P24 is one of the most popular payment methods in Poland covering all major consumer banks.

When the total amount of the order is outside the range of accepted transaction amounts, the P24 payment button is not displayed in Unified Checkout. These are the accepted transaction amounts:

- **Minimum transaction amount:** PLN 0.01, EUR 0.01
- **Maximum transaction amount:** PLN 55,000.00, EUR 12,500.00

Opt in to Przelewy24|P24 on Unified Checkout

Follow these steps to opt in to Przelewy24|P24 on Unified Checkout:

1. Add Przelewy24|P24 to your integration by adding [P24](#) to the **allowedPaymentTypes** field object within the capture context request:
2. Set the **completeMandate.type** field value to [CAPTURE](#) or [PREFER_AUTH](#).

You can capture the funds immediately if you include the **completeMandate.type** field in the capture context request and set the value to [CAPTURE](#).

If you support more than one payment type and must perform an authorization where funds are collected at a later time, set the **completeMandate.type** field to [PREFER_AUTH](#). The funds will be captured immediately for the online bank transfer.

3. Include these required fields for online bank transfers in the capture context request:

- **orderInformation.billTo.country**
- **orderInformation.billTo.email**
- **orderInformation.billTo.firstName**
- **orderInformation.billTo.lastName**

4. Include this optional field for online bank transfers in the capture context request:

- **orderInformation.billTo.address1**

Tink Pay By Bank

Tink is an alternative payment method that uses the *pay by bank* payment method. Tink Pay By Bank enables customers to make payments directly from their bank account to the seller's account and bypasses traditional payment methods such as credit cards.

When the total amount of the order is outside the range of accepted transaction amounts, Tink Pay By Bank is not displayed in Unified Checkout. These are the accepted transaction amounts:

- **Minimum transaction amount:** Not applicable
- **Maximum transaction amount:** GBP 8,500

Opt in to Tink Pay By Bank on Unified Checkout

Follow these steps to opt in to Tink Pay By Bank on Unified Checkout:

1. Add Tink Pay by Bank to your integration by adding **TINKPAYBYBANK** to the **allowedPaymentTypes** field object within the capture context request:
2. Set the **completeMandate.type** field value to **CAPTURE** or **PREFER_AUTH**.

You can capture the funds immediately if you include the **completeMandate.type** field in the capture context request and set the value to **CAPTURE**.

If you accept more than one payment type and must perform an authorization where funds are collected at a later time, set the **completeMandate.type** field to **PREFER_AUTH**. The funds are captured immediately for the online bank transfer.

3. Include these required fields for online bank transfers in the capture context request:

- **orderInformation.billTo.country**

4. Include this optional field for online bank transfers in the capture context request:

- **orderInformation.billTo.firstName**
- **orderInformation.billTo.lastName**
- **orderInformation.shipTo.address1**
- **orderInformation.shipTo.address2**
- **orderInformation.shipTo.country**
- **orderInformation.shipTo.district**
- **orderInformation.shipTo.firstName**
- **orderInformation.shipTo.lastName**
- **orderInformation.shipTo.locality**
- **orderInformation.shipTo.postalCode**

Verify Status for Online Bank Transfers (PPRO Enabled)

When the status of your payment request is [PENDING](#), you can verify the status by using the URL method and the payload that is included in the **transactionStatus.url** field in the webhook response:

```
{
  "payload": {
    "transactionResult": {
      "id": "7557753337236357904806",
      "rootId": "7557753337236357904806",
      "reconciliationId": "XFZ40EJPGL5K",
      "submitTimeUTC": "2025-08-21T11:22:13Z",
      "merchantId": "uc_apm_tester004"
    },
    "transactionStatus": {
      "url":
        "https://apitest.cybersource.com/tss/v2/transactions/7557753337236357904806",
      "method": "GET"
    }
  }
}
```

For more information, see [Webhooks Support \(on page 75\)](#).

Verify Status for Online Bank Transfers (Tink Pay By Bank)

When the status of your payment request is [PENDING](#), you can verify the status by using the URL method and the payload that is included in the **transactionStatus.url** field in the webhook response:

```
"payload": {
  "transactionResult": {
    "submitTimeUtc": "2025-07-22T08:16:24Z",
    "reconciliationId": "KPUJHD4X2G31",
    "processorInformation": {
      "responseCode": "00004"
    },
  },
  "id": "7531173918516064204807",
  "message": "Request was processed successfully.",
  "status": "SETTLED"
},
"transactionStatus": {
  "url":
    "https://apitest.cybersource.com/pts/v2/refresh-payment-status/7531173918516064204807",
  "method": "POST",
  "payload": {
    "clientReferenceInformation": {
      "applicationName": "unifiedCheckout"
    },
    "processingInformation": "processingInformation",
    "paymentInformation": {
      "paymentType": {
        "method": {
          "name": "tinkPayByBank"
        },
        "name": "bankTransfer"
      }
    }
  }
}
```

You can also send a request to this endpoint to verify the status:

Production: `POST https://api.cybersource.com/pts/v2/refresh-payment-status/{id}`

Test: `POST https://apitest.cybersource.com/pts/v2/refresh-payment-status/{id}`

The `{id}` is the ID that is returned in the webhook response. For more information, see [Webhooks Support \(on page 75\)](#).

Handle Responses

When you process a payment using **unifiedPayment.complete()** in Unified Checkout, you must handle both successful responses and various errors. After the payment is complete, the **completeResponse** object contains information about the transaction outcome.

When a payment is processed successfully, you must parse the response to confirm the payment status, update their order records, and trigger any post-payment workflows. Post-payment workflows include sending confirmation emails or updating inventory. See [JavaScript Example: Processing a Payment \(on page 25\)](#).

Your error handling should account for specific cases such as `COMPLETE_TRANSACTION_CANCELED` and `COMPLETE_TRANSACTION_FAILED`. `COMPLETE_TRANSACTION_CANCELED` occurs when the user cancels the transaction and `COMPLETE_TRANSACTION_FAILED` indicates that the consumer's transaction failed.

For PPRO-enabled online bank transfers, only cancellation errors are returned, and Tink Pay By Bank returns failure and cancellation errors. For information about possible errors that can occur when calling the complete API see [Class: AcceptError \(on page 67\)](#).

Buy Now, Pay Later

Buy Now, Pay Later payment methods enable customers to purchase goods or services immediately and pay in installments over time. With Buy Now, Pay Later, you are paid immediately and in full, while your customers pay nothing or only a portion of the total at the time of purchase. The remaining balance is typically spread over equal, often interest-free, payments.

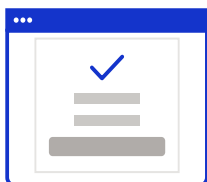
Buy Now, Pay Later is increasingly popular for both online and in-store purchases.

This is how Buy Now, Pay Later works:

Buy Now, Pay Later



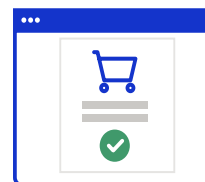
Customer selects BNPL



Customer chooses how much to pay



Unpaid amount is paid in installments



You receive full payment

1. The customer chooses their Buy Now, Pay Later payment method during checkout.
2. The customer chooses how much they want to pay, such as nothing, installments, or the total amount.
3. The unpaid amount is divided into equal installments that are paid over a fixed amount of time.
4. You receive the full payment after the customer completes checkout, and the Buy Now, Pay Later provider collects the installment payments from your customer.

Unified Checkout supports the Afterpay/Clearpay Buy Now, Pay Later payment method.

Buy Now, Pay Later Payment Method Support

Payment Method	Capture Context allowedPaymentTypes	Capture Context completeMandate.type	Separate Capture?	Payment Confirmation	Customer ISO Country Code	Customer ISO Currency Code
Afterpay	AFTERPAY	CAPTURE	No	Immediate	CA	CAD
		AUTH or PREFER_AUTH	Yes	Delayed	CA	CAD
		CAPTURE	No	Immediate	AU	AUD
		AUTH or PREFER_AUTH	Yes	Delayed	AU	USD
		CAPTURE	No	Immediate	NZ	NZD

Buy Now, Pay Later Payment Method Support (continued)

Payment Method	Capture Context allowedPaymentTypes	Capture Context completeMandate.type	Separate Capture?	Payment Confirmation	Customer ISO Country Code	Customer ISO Currency Code
		AUTH or PREFER_AUTH	Yes	Delayed	NZ	NZD
Cash App Afterpay		CAPTURE	No	Immediate	US	USD
		AUTH or PREFER_AUTH	Yes	Delayed	US	USD
Clearpay		CAPTURE	No	Immediate	GB	GBP
		AUTH or PREFER_AUTH	Yes	Delayed	GB	GBP

For information on ISO country codes, see [ISO Standard Country Codes](#).

For information on ISO currency codes, see [ISO Standard Currency Codes](#).

Afterpay

Afterpay is a Buy Now, Pay Later service that allows customers to purchase items immediately and pay for them in four interest-free installments over a period of 6 weeks. Afterpay is also known as Clearpay in the UK, and Cash App Afterpay in the US. For more information, see the [Afterpay and Clearpay Developer Guide](#).

When the total amount of the order is outside the range of accepted transaction amounts, the Afterpay/Clearpay payment button is not displayed in Unified Checkout. These are the accepted transaction amounts:

- **Minimum transaction amount:** 1 (CAD, AUD, NZD, USD, and GBP)
- **Maximum transaction amount:** Not applicable

Opt in to Afterpay on Unified Checkout

Follow these steps to opt in to the Afterpay/Clearpay payment method in Unified Checkout:

1. Add Afterpay to your integration by adding `AFTERPAY` to the **allowedPaymentTypes** field within the capture context request. The default field value is `AFTERPAY` even if you want to support Cash App Afterpay in the US or Clear Pay in the UK.
2. Set the **completeMandate.type** field value to `AUTH`, `CAPTURE` or `PREFER_AUTH`. You can perform a sale and capture the funds immediately if you include the **completeMandate.type** field in the capture context request and set the value to `CAPTURE`.

You can capture the funds later if you include the **completeMandate.type** field in the capture context request and set the value to `AUTH`. When you capture the funds later, you must perform a capture using the payments API. See [Captures \(on page 118\)](#).

If you accept more than one payment type and must perform an authorization where funds are collected at a later time, set the **completeMandate.type** field to `PREFER_AUTH`. You must perform a capture using the payments API when an authorization is performed. A capture is performed automatically if an authorization is not allowed by the payment type.

3. Include these required fields in the capture context request:

- **orderInformation.billTo.email**
- **orderInformation.billTo.firstName**
- **orderInformation.billTo.lastName**
- **orderInformation.billTo.address1**
- **orderInformation.billTo.locality**
- **orderInformation.billTo.postalCode**
- **orderInformation.billTo.administrativeArea**
- **orderInformation.billTo.country**

4. Include these optional fields in the capture context request:



Important: These fields are required when the **requestShipping** field is set to `true`.

- **orderInformation.shipTo.firstName**
- **orderInformation.shipTo.lastName**
- **orderInformation.shipTo.address1**
- **orderInformation.shipTo.locality**

- **orderInformation.shipTo.postalCode**
- **orderInformation.shipTo.administrativeArea**
- **orderInformation.shipTo.country**

Handle Responses

When you process a payment using **unifiedPayment.complete()** in Unified Checkout, you must handle both successful responses and various errors. After the payment is complete, the **completeResponse** object contains information about the transaction outcome.

When a payment is processed successfully, you must parse the response to confirm the payment status, update their order records, and trigger any post-payment workflows. Post-payment workflows include sending confirmation emails or updating inventory. See [JavaScript Example: Processing a Payment \(on page 25\)](#).

Your error handling should account for specific cases such as `COMPLETE_TRANSACTION_CANCELED` and `COMPLETE_TRANSACTION_FAILED`. `COMPLETE_TRANSACTION_CANCELED` occurs when the user cancels the transaction and `COMPLETE_TRANSACTION_FAILED` indicates that the consumer's transaction failed.

Captures

When you set the **completeMandate.type** field value to `AUTH` or `PREFER_AUTH`, you must send a request to capture an authorized payment. Full and partial captures are supported.

Endpoint

Production: `POST https://api.cybersource.com/pts/v2/payments/{id}/captures`

Test: `POST https://apitest.cybersource.com/pts/v2/payments/{id}/captures`

The `{id}` is the transaction ID returned in the authorization response.

Example: Authorization Response from Unified Checkout

```
{
  "details": {
    "clientReferenceInformation": {
      "code": "1753351101383"
    },
    "orderInformation": {
```



```

    "amountDetails": {
      "currency": "USD",
      "totalAmount": "21.00"
    },
    "processorInformation": {
      "approvalCode": "AUTH456789",
      "responseCode": "00003",
      "responseDetails": "00003",
      "transactionId": "2016011808153910011808153AUTH"
    },
    "reconciliationId": "04RYADD29YR0",
    "submitTimeUtc": "2025-07-24T09:58:21Z"
  },
  "id": "7533511014286971803092",
  "message": "Request processed successfully.",
  "outcome": "AUTHORIZED",
  "status": "AUTHORIZED"
}

```

Response Status

Cybersource responds to your capture request with one of these statuses:

- **FAILED**: The capture request failed.
- **PENDING**: The capture request is accepted but not captured. Send a request to the check status service to retrieve status updates.
- **SETTLED**: The capture request is settled for the amount requested.

Post-Pay Reference Payments

Post-pay reference payments provide an alternative way for your customers to complete online purchases using cash. During checkout, customers select a voucher payment option and receive a unique code or payment slip. To finalize their purchase, they visit a designated offline location, such as a convenience store or payment kiosk, and pay the required amount in person. Once the payment is made, the merchant receives a notification that they can process the order.

Post-pay reference payments are widely used in countries where cash transactions are common or where many customers don't have access to credit or debit cards.

This is how post-pay reference payments work:



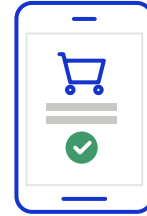
Customer selects
voucher-based
payment method



Customer receives
voucher



Customer presents
voucher at store



Customer notified
that the payment is
complete

1. The customer selects a convenience store/voucher payment method during checkout.
2. The customer receives their payment code/voucher or QR code.
3. The customer presents the payment code/voucher or QR in-store to complete the payment.
4. The customer receives a notification that the payment is complete.

Unified Checkout supports the Konbini voucher-based payment option:

Post-Pay Reference Payment Support

Payment Method	Capture Context allowedPaymentTypes	Capture Context completeMandate.type	Separate Capture?	Payment Confirmation	Customer Country	Customer Currency
Konbini	KONBINI	CAPTURE or PREFER_AUTH	No	Immediate	Japan (JP)	JPY

Konbini

Konbini is used to make cash payments in Japan. Konbini payments enable your customers to pay for bills and online purchases at convenience stores in Japan. To complete a transaction, your customers receive payment codes for specific convenience stores and a confirmation number. Customers must then bring the information to a convenience store to make a cash payment. Your customers can pay at these convenience stores in Japan:

- 7-Eleven
- Family Mart
- Lawson
- Ministop
- Seicomart

You receive the payment confirmation immediately and the funds are available after 4 business days.

When the total amount of the order is outside the range of accepted transaction amounts, the Konbini payment button is not displayed in Unified Checkout. These are the accepted transaction amounts:

- **Minimum transaction amount:** JPY 1
- **Maximum transaction amount:** Not applicable

Opt in to Konbini on Unified Checkout

Follow these steps to opt in to the Konbini payment method in Unified Checkout:

1. Add Konbini to your integration by adding `KONBINI` to the **allowedPaymentTypes** field within the capture context request.
2. Set the **completeMandate.type** field value to `AUTH`, `CAPTURE` or `PREFER_AUTH`. You can perform a sale and capture the funds immediately if you include the **completeMandate.type** field in the capture context request and set the value to `CAPTURE`.

If you accept more than one payment type and must perform an authorization where funds are collected at a later time, set the **completeMandate.type** field to `PREFER_AUTH`. You must perform a capture using the payments API when an authorization is performed. A capture is performed automatically if an authorization is not allowed by the payment type.

3. Include these required fields in the capture context request:

- **orderInformation.billTo.country**
- **orderInformation.billTo.firstName**
- **orderInformation.billTo.lastName**
- **orderInformation.billTo.phoneNumber**

4. Include these optional fields in the capture context request:



Important: These fields are required when the **requestShipping** field is set to `true`.

- **orderInformation.billTo.address1**
- **orderInformation.billTo.email**

Verify Status for Post-Pay Reference (PPRO Enabled)

When the status of your payment request is `PENDING`, you can verify the status by sending a GET request to the URL that is included in the **transactionStatus.url** field in the response:

```
"payload": {
  "transactionResult": {
    "id": "7557753337236357904806",
    "rootId": "7557753337236357904806",
    "reconciliationId": "XFZ40EJPL5K",
    "submitTimeUTC": "2025-08-21T11:22:13Z",
    "merchantId": "uc_apm_tester004"
  },
  "transactionStatus": {
    "url":
    "https://apitest.cybersource.com/tss/v2/transactions/7557753337236357904806",
    "method": "GET"
  }
}
```

You can also send a request to this endpoint to verify the status:

Production: `POST https://api.cybersource.com/pts/v2/refresh-payment-status/{id}`

Test: `POST https://apitest.cybersource.com/pts/v2/refresh-payment-status/{id}`

The `{id}` is ID that is returned in the webhook response. For more information, see [Webhooks Support \(on page 75\)](#).

Handle Responses

When you process a payment using **unifiedPayment.complete()** in Unified Checkout, you must handle both successful responses and various errors. After the payment is complete, the **completeResponse** object contains information about the transaction outcome.

When a payment is processed successfully, you must parse the response to confirm the payment status, update their order records, and trigger any post-payment workflows. Post-payment workflows include sending confirmation emails or updating inventory. See [JavaScript Example: Processing a Payment \(on page 25\)](#).

Your error handling should account for specific cases such as `COMPLETE_TRANSACTION_CANCELED` which is returned when the user cancels the transaction.

For PPRO-enabled online bank transfers, only cancellation errors are returned. For information about possible errors that can occur when calling the complete API see [Class: AcceptError \(on page 67\)](#).

Unified Checkout Appendix

Unified Checkout UI

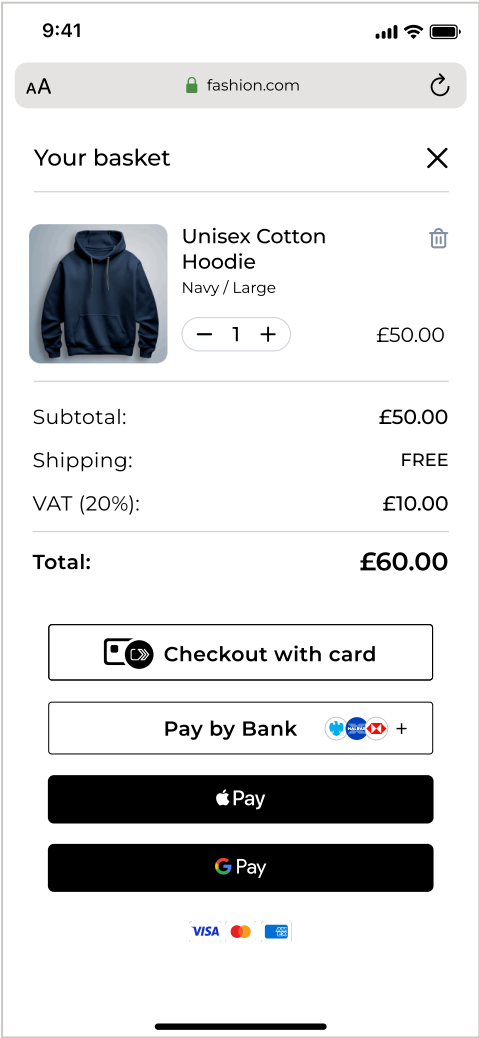
Completing a payment with Unified Checkout requires the customer to navigate through a sequence of interfaces. This section includes examples of the interfaces your customers can expect when completing a payment with these payment methods on Unified Checkout:

- Apple Pay
- Click to Pay
- Google Pay
- Manual payment entry
- Payment with a bank account
- Paze

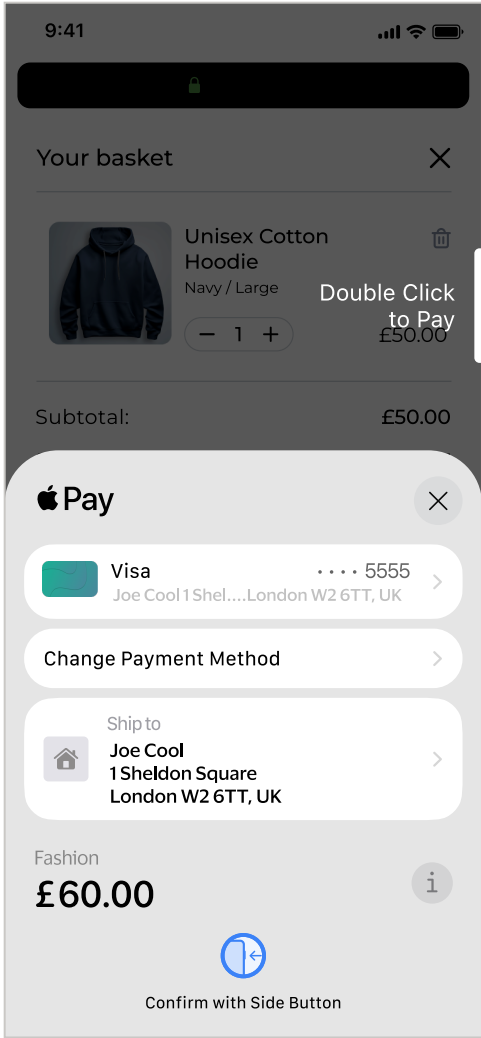
Apple Pay UI

These screen captures show the sequence of events your customer can expect when completing a payment with Apple Pay.

Apple Pay UI



Apple Pay in the payment method list



Apple Pay checkout

Click to Pay UI

These screen captures show the sequence of events your customer can expect when completing a payment with Click to Pay.

Click to Pay UI

9:41

FASHION

CONTACT DETAILS

Email

example@email.com

Mobile number

+44 7412 112233

Looking for your linked cards...

9:41

FASHION

CONTACT DETAILS

Email

example@email.com

Mobile number

+44 7412 112233

Click to Pay didn't find linked cards for example@email.com

9:41

FASHION

CONTACT DETAILS

example@email.com

+44 7412 112233

PAYMENT DETAILS:

Card number

Expiry MM/YY

CVV

First name

Last name

Click to Pay didn't find linked cards for example@email.com

Switch ID

Continue

9:41

FASHION

CONTACT DETAILS

example@email.com

+44 7412 112233

PAYMENT DETAILS:

Card number

Expiry MM/YY

CVV

First name

Last name

☒

Link this card to Click to Pay

By clicking the button below, you agree to the Terms for Click to Pay and understand your data will be processed according to the Privacy Notice.

We'll send you a confirmation email.

BILLING ADDRESS:

Address line 1

1 Sheldon Square

Address line 2

Paddington Central

Post code

City / Town

W2 6TT

London

Country

United Kingdom

☐ Shipping address is same as billing

Continue

9:41

FASHION

CONTACT DETAILS

example@email.com

+44 7412 112233

SHIPPING ADDRESS

First name

Last name

Betty

Miller

Address line 1

30 Manor Road

Address line 2

Post code

City / Town

N94 2NZ

London

Country

United Kingdom

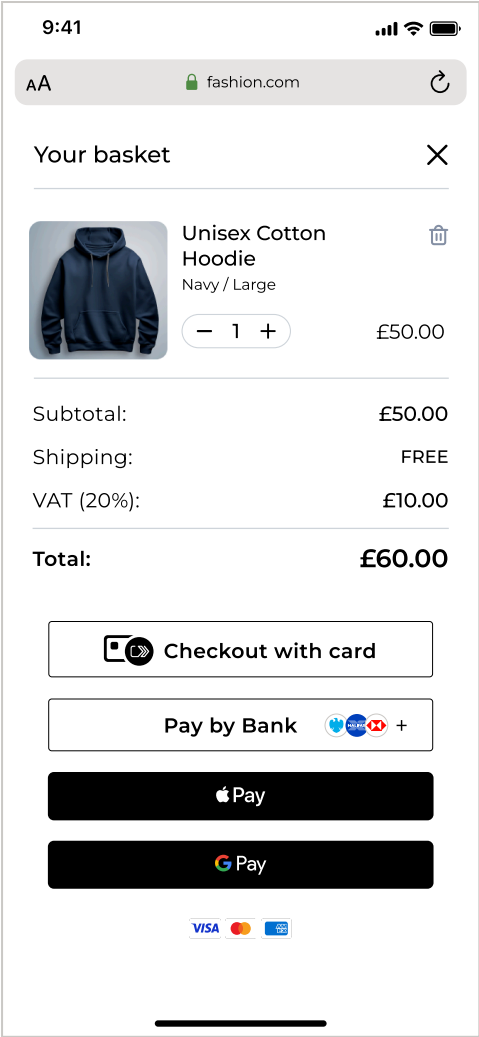
Pay

Unified Checkout | Unified Checkout Appendix | 127

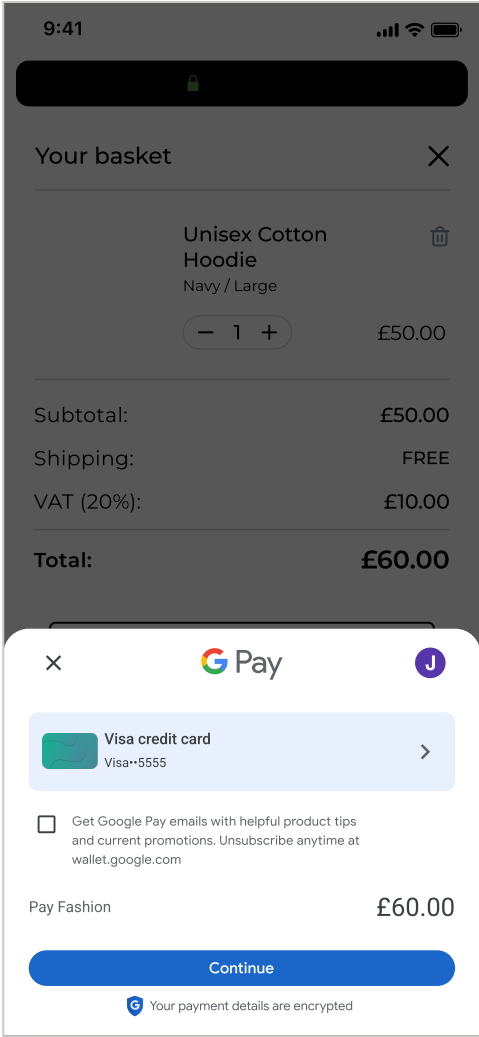
Google Pay UI

These screen captures show the sequence of events your customer can expect when completing a payment with Google Pay.

Google Pay UI



Google Pay in the payment method list



Google Pay checkout

Manual Payment Entry UI

These screen captures show the sequence of events your customer can expect when completing a payment by manually entering payment, shipping, and contact information.

Manual Entry Payment Details

USD

FREE SHIPPING & RETURNS NEED HELP? SIGN IN

FASHION

NEW CLOTHING SHOES WATCHES WALLETS LUGGAGE

🔍 ❤️ 🛒

Checkout

CONTACT DETAILS

example@email.com

+44 7412 112233

Edit

PAYMENT DETAILS:

Card number

Expiry MM/YY

CVV

First name

Last name

Joe

Cool

☒ Link this card to [Click to Pay](#)

By clicking the button below, you agree to the [Terms](#) for Click to Pay and understand your data will be processed according to the [Privacy Notice](#).

We'll send you a confirmation email.

Country code phone number

+44

7412 112233

BILLING ADDRESS:

Address line 1

Address line 2

Post code

City / Town

Country

United Kingdom

☒ Shipping address is same as billing

🔒 Pay

Order summary

Edit

Unisex Cotton Hoodie
Black / Large
Qnt. 1\$ 55.00

Classic Baseball Cap
Grey / Medium
Qnt. 1\$ 25.00

Subtotal:\$ 80.00

Shipping:FREE

Tax:\$ 16.00

Total:\$ 96.00

Unified Checkout | Unified Checkout Appendix | 129

Manual Entry Contact Details

USD 

FREE SHIPPING & RETURNS NEED HELP? SIGN IN

FASHION

NEW CLOTHING SHOES WATCHES WALLETS LUGGAGE

Checkout

CONTACT DETAILS

Email

Mobile number

 Looking for your linked cards...

Order summary

[EDIT](#)

	Unisex Cotton Hoodie Black / Large Qnt. 1	\$ 55.00
	Classic Baseball Cap Grey / Medium Qnt. 1	\$ 25.00
Subtotal:		\$ 80.00
Shipping:		FREE
Tax:		\$ 16.00
Total:		\$ 96.00

Manual Entry Review and Confirm

USD 

FREE SHIPPING & RETURNS NEED HELP? SIGN IN

FASHION

NEW CLOTHING SHOES WATCHES WALLETS LUGGAGE

Checkout

CONTACT DETAILS [Edit](#)

example@email.com
+44 7412 112233

PAYMENT DETAILS: [Edit](#)

 VISA 5555

SHIP TO: [Edit](#)

Joe Cool
1 Sheldon Square, Paddington
Central London W2 6TT, UK

 Pay

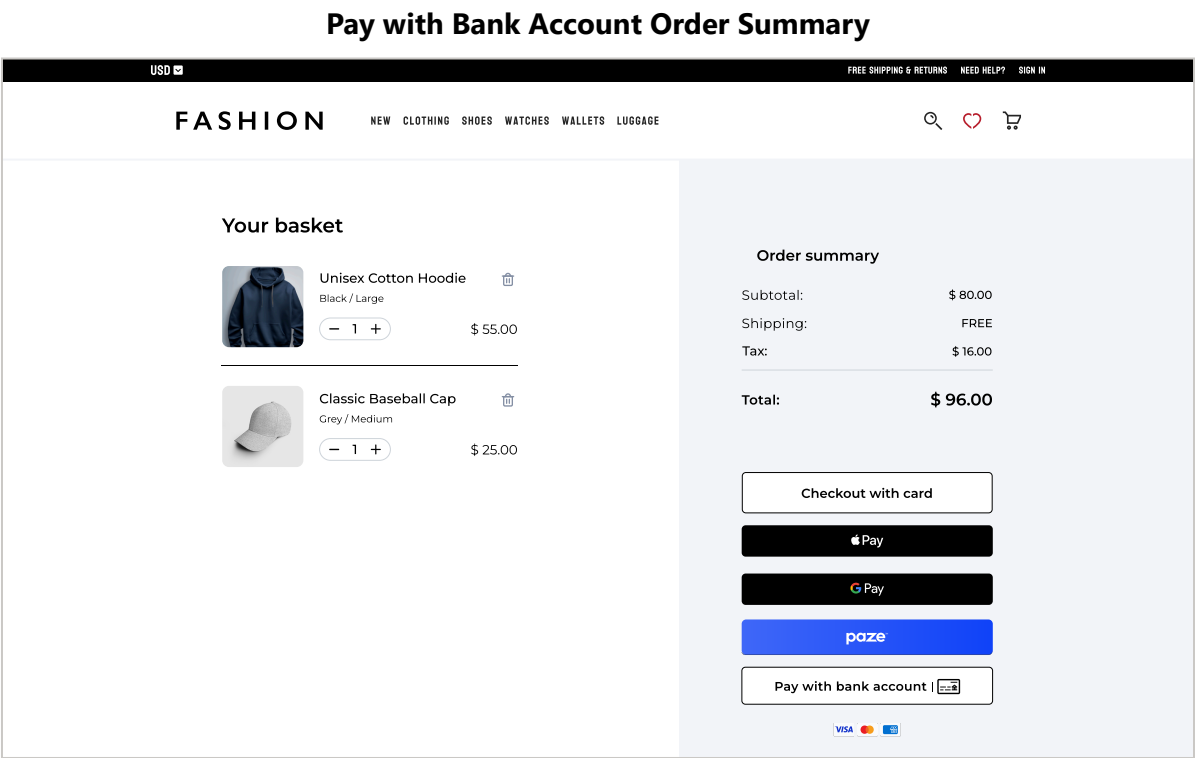
Order summary

[EDIT](#)

	Unisex Cotton Hoodie Black / Large Qnt. 1	\$ 55.00
	Classic Baseball Cap Grey / Medium Qnt. 1	\$ 25.00
Subtotal:		\$ 80.00
Shipping:		FREE
Tax:		\$ 16.00
Total:		\$ 96.00

Pay with Bank Account UI

These screen captures show the sequence of events your customer can expect when completing a payment with a bank account.



Pay with Bank Account Checkout

USD

FREE SHIPPING & RETURNS

NEED HELP?

SIGN IN

FASHION

NEW CLOTHING SHOES WATCHES WALLETS LUGGAGE

Checkout

CONTACT DETAILS

example@email.com

+44 7412 112233

PAYMENT DETAILS:

Account type

Routing number

Account number

Confirm account number

☐ Set as default

BILLING ADDRESS:

Address line 1

1 Sheldon Square

Address line 2

Paddington Central

Post code

City / Town

W2 6TT

London

Country

United Kingdom

Pay

Order summary

[EDIT](#)

Unisex Cotton Hoodie
Black / Large
Qnt. 1 \$ 55.00

Classic Baseball Cap
Grey / Medium
Qnt. 1 \$ 25.00

Subtotal:

\$ 80.00

Shipping:

FREE

Tax:

\$ 16.00

Total:

\$ 96.00

Pay with Bank Account Review and Confirm

USD 

FREE SHIPPING & RETURNS NEED HELP? SIGN IN

FASHION

NEW CLOTHING SHOES WATCHES WALLET LUGGAGE

Checkout

CONTACT DETAILS

example@email.com
+44 7412 112233

PAYMENT DETAILS:

 ****9012

[Change payment](#)
[Add new account](#)

SHIP TO:

Joe Cool
1 Sheldon Square, Paddington
Central London W2 6TT, UK

[Edit](#)

 Pay

Order summary [EDIT](#)

 **Unisex Cotton Hoodie**
Black / Large
Qnt. 1 \$ 55.00

 **Classic Baseball Cap**
Grey / Medium
Qnt. 1 \$ 25.00

Subtotal: \$ 80.00

Shipping: FREE

Tax: \$ 16.00

Total: \$ 96.00

Unified Checkout | Unified Checkout Appendix | 133

Paze UI

These screen captures show the sequence of events your customer can expect when completing a payment with Paze.

Paze UI

Merchant Logo

BryansBikes
450 Bellevue Square, Bellevue, WA 98004
425-454-8096

Enter your email
Get offers delivered directly to your inbox...

[Continue](#)

Collecting an email for Paze lookup

Merchant Logo

BryansBikes
450 Bellevue Square, Bellevue, WA 98004
425-454-8096

A1 Helmet w/MIPS Classic
This lightweight, fully encapsulated helmet maximum coverage and dimension to keep you safe and protected in all riding conditions.

Price: \$50.47 x Enter Qty.: 2 Max 10 \$100.95

Total \$100.95

paze

Checkout with card

Paze in the payment method list

paze

Check out at BryansBikes with Paze, a new digital wallet offered by your bank or credit union.

Get started with a one-time SMS code to access your eligible cards from a single wallet. No entering full card numbers. No new username or password. Always up-to-date payment.

[Continue with Paze](#)

[Exit Paze and return to BryansBikes](#)

[Privacy Notice](#) | [Opt out](#)

Paze explainer

paze

BryansBikes
Subtotal **\$100.95**

Confirm it's you.
Enter the security code sent to the number ending in **6749** to confirm it's you.

Didn't receive a code? [Send a new code.](#)

[Exit Paze and return to BryansBikes](#)

[Opt out](#)

Paze one-time password

paze

BryansBikes
Subtotal **\$100.95**

Payment

Your Bank
**** 6568

Shipping

Kalpesh Vaidya
901 Metro Center Blvd
Foster City, CA 94404, United States

Contact

kalpesh@paze.com

[Confirm details](#)

[Exit Paze and return to BryansBikes](#)

Paze payment method selection

PaybyLink.com

Success! thank you for your payment.

Merchant Logo

BryansBikes
450 Bellevue Square, Bellevue, WA 98004
425-454-8096

A1 Helmet w/MIPS Classic
This lightweight, fully encapsulated helmet maximum coverage and dimension to keep you safe and protected in all riding conditions.
\$50.00 x 2 \$100.95

Total \$100.95

Transaction date 02/02/2023 at 11:20 AM PST
Transaction ID A68890-20
Auth code A12234

Payment method VISA **** 6568

Billing address
901 Metro Center Blvd
Foster City, CA 94404

Shipping address
Kalpesh Vaidya
901 Metro Center Blvd
Foster City, CA 94404

Paze payment confirmation

JSON Web Tokens

JSON Web Tokens (JWTs) are digitally signed JSON objects based on the open standard [RFC 7519](#). These tokens provide a compact, self-contained method for securely transmitting information between parties. These tokens are signed with an RSA-encoded public/private key pair. The signature is calculated using the header and body, which enables the receiver to validate that the content has not been tampered with.

A JWT takes the form of a string, and consists of three parts separated by dots:

```
<Header>.<Payload>.<Signature>
```

The header and payload is **Base64-encoded JSON** and contains these claims:

- **Header:** The algorithm and token type. For example:

```
{
  "kid": "zu",
  "alg": "RS256"
}
```

- **Payload:** The claims of what the token represents. For example:

```
{
  "sub": "1234567890",
  "name": "John Doe",
  "iat": 1516239022
}
```

- **Signature:** The signature is computed from the header and payload using a secret or private key.



Important: When working with JWTs, Cybersource recommends that you use a well-maintained JWT library to ensure proper decoding and parsing of the JWT.



Important: When parsing the JWT's JSON payload, you must ensure that you implement a robust solution for transversing JSON. Additional elements can be added to the JSON in future releases. Follow JSON parsing best practices to ensure that you can handle the addition of new data elements in the future.

Unified Checkout Integration Examples

This section contains the information you need in order to integrate Unified Checkout within your e-commerce experience and use the `unifiedPayment.complete()` method.



Important: If you want to use the complete mandate, you must ensure that your JavaScript and capture contexts are compatible with the complete mandate. See [Capture Context API \(on page 30\)](#).

Unified Checkout with Complete Mandate

This section contains the capture context and JavaScript examples to integrate Unified Checkout into your e-commerce page. You can use these examples to collect your customer's payment information and process an authorization. You must initiate a capture request to move funds and complete the transaction.

For information on the capture context, see [Capture Context API \(on page 30\)](#).

Example: Unified Checkout with Prefer Authorization

Capture Context Request

```
{
  "targetOrigins" : [ "https://the-up-demo.appspot.com" ],
  "clientVersion": "0.34",
  "buttonType": "CHECKOUT_AND_CONTINUE",
  "allowedCardNetworks": [
    "VISA",
    "MASTERCARD"
  ],
  "allowedPaymentTypes": [
    "PANENTRY",
    "CLICKTOPAY",
    "APPLEPAY",
    "GOOGLEPAY"
  ],
  "completeMandate": {
    "type": "PREFER_AUTH"
  },
  "country": "US",
  "locale": "en_US",
```

```

"captureMandate": {
  "billingType": "FULL",
  "requestEmail": true,
  "requestPhone": true,
  "requestShipping": true,
  "shipToCountries": [
    "US",
    "GB"
  ],
  "showAcceptedNetworkIcons": true
},
"data": {
  "orderInformation": {
    "billTo": {
      "country": "US",
      "firstName": "NEW",
      "lastName": "Test",
      "phoneNumber": "1234567890",
      "address2": "Desk M3-5573",
      "address1": "901 Metro Center Blvd",
      "buildingNumber": "150",
      "postalCode": "94404",
      "locality": "Foster City",
      "administrativeArea": "CA",
      "email": "test@example.com"
    },
    "shipTo": {
      "country": "US",
      "firstName": "NEW",
      "lastName": "Test",
      "address2": "Desk M3-5573",
      "address1": "901 Metro Center Blvd",
      "buildingNumber": "150",
      "postalCode": "94404",
      "locality": "Foster City",
      "administrativeArea": "CA"
    },
    "amountDetails": {
      "totalAmount": "13.00",
      "currency": "USD"
    }
  },
  "clientReferenceInformation": {
    "code": "TAGX001"
  }
}
}

```

JavaScript

```
<script type="text/javascript">
  const sidebar = true;
  const captureContext = document.getElementById("captureContext").value;
  const showArgs = {
    containers: { paymentSelection: "#buttonPaymentListContainer" }
  };

  async function launchCheckout() {
    try {
      const accept = await Accept(captureContext);
      const up = await accept.unifiedPayments(sidebar);
      const tt = await up.show(showArgs);
      const completeResponse = await up.complete(tt);

      console.log(completeResponse); // merchant logic for handling complete
response
    } catch (error) {
      // merchant logic for handling issues
      console.error("something went wrong: " + error);
    }
  }

  // Call the function
  launchCheckout();
</script>
```

Unified Checkout with Sale

This section contains the capture context and JavaScript examples to integrate Unified Checkout into your e-commerce page. You can use these examples to collect your customer's payment information and process a sale.

For information on the capture context, see [Capture Context API \(on page 30\)](#).

Example: Unified Checkout with Sale

Capture Context Request

```
{
  "targetOrigins" : [ "https://the-up-demo.appspot.com" ],
  "clientVersion": "0.34",
```

```

"buttonType": "CHECKOUT_AND_CONTINUE",
"allowedCardNetworks": [
  "VISA",
  "MASTERCARD"
],
"allowedPaymentTypes": [
  "PANENTRY",
  "CLICKTOPAY",
  "APPLEPAY",
  "GOOGLEPAY"
],
"completeMandate": {
  "type": "CAPTURE"
},
"country": "US",
"locale": "en_US",
"captureMandate": {
  "billingType": "FULL",
  "requestEmail": true,
  "requestPhone": true,
  "requestShipping": true,
  "shipToCountries": [
    "US",
    "GB"
  ],
  "showAcceptedNetworkIcons": true
},
"data": {
  "orderInformation": {
    "billTo": {
      "country": "US",
      "firstName": "NEW",
      "lastName": "Test",
      "phoneNumber": "1234567890",
      "address2": "Desk M3-5573",
      "address1": "901 Metro Center Blvd",
      "buildingNumber": "150",
      "postalCode": "94404",
      "locality": "Foster City",
      "administrativeArea": "CA",
      "email": "test@example.com"
    },
    "shipTo": {
      "country": "US",
      "firstName": "NEW",
      "lastName": "Test",
      "address2": "Desk M3-5573",

```

```

        "address1": "901 Metro Center Blvd",
        "buildingNumber": "150",
        "postalCode": "94404",
        "locality": "Foster City",
        "administrativeArea": "CA"
    },
    "amountDetails": {
        "totalAmount": "13.00",
        "currency": "USD"
    }
},
"clientReferenceInformation": {
    "code": "TAGX001"
}
}
}

```

JavaScript

```

<script type="text/javascript">
    const sidebar = true;
    const captureContext = document.getElementById("captureContext").value;
    const showArgs = {
        containers: { paymentSelection: "#buttonPaymentListContainer" }
    };

    async function launchCheckout() {
        try {
            const accept = await Accept(captureContext);
            const up = await accept.unifiedPayments(sidebar);
            const tt = await up.show(showArgs);
            const completeResponse = await up.complete(tt);

            console.log(completeResponse); // merchant logic for handling complete
response
        } catch (error) {
            // merchant logic for handling issues
            console.error("something went wrong: " + error);
        }
    }

    // Call the function
    launchCheckout();
</script>

```

Unified Checkout with Sale and Decision Manager

This section contains the capture context and JavaScript examples to integrate Unified Checkout into your e-commerce page. You can use these examples to collect your customer's payment information and process a sale while also invoking the Decision Manager fraud solution. Before the sale is initiated, Decision Manager is invoked for fraud screening.

For information on the capture context, see [Capture Context API \(on page 30\)](#).

Example: Unified Checkout with Sale and Decision Manager

Capture Context Request

```
{
  "targetOrigins" : [ "https://test.com" ],
  "clientVersion": "0.34",
  "buttonType": "CHECKOUT_AND_CONTINUE",
  "allowedCardNetworks": [
    "VISA",
    "MASTERCARD"
  ],
  "allowedPaymentTypes": [
    "PANENTRY",
    "CLICKTOPAY",
    "APPLEPAY",
    "GOOGLEPAY"
  ],
  "completeMandate": {
    "type": "CAPTURE",
    "decisionManager": true
  },
  "country": "US",
  "locale": "en_US",
  "captureMandate": {
    "billingType": "FULL",
    "requestEmail": true,
    "requestPhone": true,
    "requestShipping": true,
    "shipToCountries": [
      "US",
      "GB"
    ],
  },
  "showAcceptedNetworkIcons": true
},
"data":
{
```



```

"orderInformation": {
  "billTo": {
    "country": "US",
    "firstName": "NEW",
    "lastName": "Test",
    "phoneNumber": "1234567890",
    "address2": "Desk M3-5573",
    "address1": "901 Metro Center Blvd",
    "buildingNumber": "150",
    "postalCode": "94404",
    "locality": "Foster City",
    "administrativeArea": "CA",
    "email": "test@example.com"
  },
  "shipTo": {
    "country": "US",
    "firstName": "NEW",
    "lastName": "Test",
    "address2": "Desk M3-5573",
    "address1": "901 Metro Center Blvd",
    "buildingNumber": "150",
    "postalCode": "94404",
    "locality": "Foster City",
    "administrativeArea": "CA"
  },
  "amountDetails": {
    "totalAmount": "13.00",
    "currency": "USD"
  }
},
"clientReferenceInformation": {
  "code": "TAGX001"
}
}

```

JavaScript

```

<script type="text/javascript">
  const sidebar = true;
  const captureContext = document.getElementById("captureContext").value;
  const showArgs = {
    containers: { paymentSelection: "#buttonPaymentListContainer" }
  };

  async function launchCheckout() {
    try {

```

```

const accept = await Accept(captureContext);
const up = await accept.unifiedPayments(sideBar);
const tt = await up.show(showArgs);
const completeResponse = await up.complete(tt);

console.log(completeResponse); // merchant logic for handling complete
response
} catch (error) {
  // merchant logic for handling issues
  console.error("something went wrong: " + error);
}
}

// Call the function
launchCheckout();
</script>

```

Unified Checkout without Service Orchestration

This section contains the capture context and JavaScript examples to integrate Unified Checkout into your e-commerce page. You can use these examples to collect your customer's payment information. Payment processing and service orchestration are completed through the back end of the integrator.

For information on the capture context, see [Capture Context API \(on page 30\)](#).

Example: Unified Checkout without Service Orchestration

Capture Context Request

```

{
  "targetOrigins" : [ "https://test.com" ],
  "clientVersion": "0.34",
  "buttonType": "CHECKOUT_AND_CONTINUE",
  "allowedCardNetworks": [
    "VISA",
    "MASTERCARD"
  ],
  "allowedPaymentTypes": [
    "PANENTRY",
    "CLICKTOPAY",
    "APPLEPAY",
    "GOOGLEPAY"
  ],

```

```

"country": "US",
"locale": "en_US",
"captureMandate": {
  "billingType": "FULL",
  "requestEmail": true,
  "requestPhone": true,
  "requestShipping": true,
  "shipToCountries": [
    "US",
    "GB"
  ],
  "showAcceptedNetworkIcons": true
},
"data": {
  "orderInformation": {
    "billTo": {
      "country": "US",
      "firstName": "NEW",
      "lastName": "Test",
      "phoneNumber": "1234567890",
      "address2": "Desk M3-5573",
      "address1": "901 Metro Center Blvd",
      "buildingNumber": "150",
      "postalCode": "94404",
      "locality": "Foster City",
      "administrativeArea": "CA",
      "email": "test@example.com"
    },
    "shipTo": {
      "country": "US",
      "firstName": "NEW",
      "lastName": "Test",
      "address2": "Desk M3-5573",
      "address1": "901 Metro Center Blvd",
      "buildingNumber": "150",
      "postalCode": "94404",
      "locality": "Foster City",
      "administrativeArea": "CA"
    },
    "amountDetails": {
      "totalAmount": "13.00",
      "currency": "USD"
    }
  },
  "clientReferenceInformation": {
    "code": "TAGX001"
  }
}

```

```
}
```

JavaScript

```
<script type="text/javascript">
  const sidebar = true;
  const captureContext = document.getElementById("captureContext").value;
  const showArgs = {
    containers: { paymentSelection: "#buttonPaymentListContainer" }
  };

  async function launchCheckout() {
    try {
      const accept = await Accept(captureContext);
      const up = await accept.unifiedPayments(sidebar);
      const tt = await up.show(showArgs);
      console.log(tt); // merchant logic for passing the Transient token to
their backend for service orchestration
    } catch (error) {
      // merchant logic for handling issues
      console.error("something went wrong: " + error);
    }
  }
  // Call the function
  launchCheckout();
</script>
```

Echeck Payment with a Transient Token

This section contains a request example for making a payment with an eCheck using a transient token. You must meet these requirements to make an eCheck payment with a transient token:

- The **paymentType.name.value** field value must be set to **CHECK** in the transient token.
- You must include the **paymentInformation.paymentType.name** field in your request and the value must be set to **CHECK** in your request.

Echeck Payment Request with a Transient Token

Unified Checkout | Unified Checkout Appendix | 147

Example: Unified Checkout Complete Capture Context

Capture Context Request

```
{
  "country": "US",
  "locale": "en_GB",
  "targetOrigins": [
    "https://merchant.com",
    "https://reseller.com:8443"
  ],
  "clientVersion": "0.34",
  "allowedCardNetworks": [
    "VISA",
    "MASTERCARD",
    "AMEX"
  ],
  "captureMandate": {
    "billingType": "FULL",
    "requestEmail": true,
    "requestPhone": true,
    "requestShipping": true,
    "shipToCountries": [
      "US",
      "GB"
    ],
    "showAcceptedNetworkIcons": true
  },
  "completeMandate": {
    "type": "PREFER_AUTH",
    "decisionManager": true,
    "consumerAuthentication": true,
    "tms": {
      "tokenCreate": true,
      "tokenTypes": [
        "customer",
        "paymentInstrument",
        "instrumentIdentifier",
        "shippingAddress"
      ]
    }
  },
  "allowedPaymentTypes": [
    "PANENTRY",
    "CLICKTOPAY",
    "APPLEPAY",
    "GOOGLEPAY"
  ]
}
```

```

],
"data": {
  "orderInformation": {
    "amountDetails": {
      "totalAmount": "102.21",
      "currency": "USD",
      "surcharge": {
        "amount": "2.50"
      },
      "discountAmount": "2.00",
      "serviceFeeAmount": "5.00",
      "taxAmount": "10.00"
    },
    "billTo": {
      "address1": "string",
      "address2": "string",
      "address3": "string",
      "address4": "string",
      "administrativeArea": "st",
      "buildingNumber": "string",
      "country": "US",
      "district": "string",
      "locality": "string",
      "postalCode": "string",
      "company": {
        "name": "Visa Inc",
        "address1": "900 Metro Center Blvd",
        "address2": "address2",
        "address3": "address3",
        "address4": "address4",
        "administrativeArea": "CA",
        "buildingNumber": "1",
        "country": "US",
        "district": "district",
        "locality": "Foster City",
        "postalCode": "94404"
      },
      "email": "test@test.com",
      "firstName": "string",
      "lastName": "string",
      "middleName": "string",
      "nameSuffix": "string",
      "title": "string",
      "phoneNumber": "string",
      "phoneType": "string"
    },
    "shipTo": {
      "address1": "string",
      "address2": "string",

```

```

    "address3": "string",
    "address4": "string",
    "administrativeArea": "st",
    "buildingNumber": "string",
    "country": "US",
    "district": "string",
    "locality": "string",
    "postalCode": "string",
    "firstName": "string",
    "lastName": "string"
  },
  "lineItems": [
    {
      "productCode": "string",
      "productName": "string",
      "productSku": "string",
      "quantity": 2,
      "unitPrice": "string",
      "unitOfMeasure": "string",
      "totalAmount": "string",
      "taxAmount": "string",
      "taxRate": "string",
      "taxAppliedAfterDiscount": "y",
      "taxStatusIndicator": "N",
      "taxTypeCode": "1234",
      "amountIncludesTax": true,
      "typeOfSupply": "12",
      "commodityCode": "string",
      "discountAmount": "string",
      "discountApplied": true,
      "discountRate": "string",
      "invoiceNumber": "string",
      "taxDetails": [
        {
          "type": "string",
          "amount": "string",
          "rate": "string",
          "code": "1234",
          "taxId": "string",
          "applied": true,
          "exemptionCode": "1"
        }
      ]
    },
    {
      "fulfillmentType": "string",
      "weight": "string",
      "weightIdentifier": "N",
      "weightUnit": "mg",
      "referenceDataCode": "string",
      "referenceDataNumber": "string",

```



```

        "unitTaxAmount": "string",
        "productDescription": "string",
        "giftCardCurrency": "USD",
        "shippingDestinationTypes": "",
        "gift": false,
        "passenger": {
            "type": "string",
            "status": "string",
            "phone": "string",
            "firstName": "string",
            "lastName": "string",
            "id": "string",
            "email": "string",
            "nationality": "US"
        }
    }
}

},
"buyerInformation": {
    "personalIdentification": {
        "cpf": "12345678900"
    },
    "merchantCustomerId": "string",
    "companyTaxId": "string"
},
"clientReferenceInformation": {
    "code": "TAGX001",
    "partner": {
        "developerId": "1234",
        "solutionId": "4567"
    }
},
"consumerAuthenticationInformation": {
    "challengeCode": "01",
    "messageCategory": "01"
},
"merchantInformation": {
    "merchantDescriptor": {
        "name": "Jane Sales"
    }
},
"processingInformation": {
    "reconciliationId": "01234567",
    "authorizationOptions": {
        "aftIndicator": true,
        "initiator": {
            "credentialStoredOnFile": true,
            "merchantInitiatedTransaction": {
                "reason": "1"
            }
        }
    }
}
}

```

```

        }
    },
    "businessApplicationId": "AA"
},
"recipientInformation": {
    "firstName": "John",
    "middleName": "A",
    "lastName": "Buyer",
    "country": "GB",
    "accountId": "acc0123567",
    "administrativeArea": "GB",
    "accountType": "01"
},
"merchantDefinedInformation": [
    {
        "key": "1",
        "value": "12345"
    },
    {
        "key": "2",
        "value": "67890"
    },
    {
        "key": "3",
        "value": "DISCOUNT20"
    }
]
}
}

```

Supported Countries for Digital Payments

Apple Pay, Click to Pay, eCheck, Google Pay, and Paze are supported in different countries.

See these topics for the lists of the countries that support digital payments:

- [Supported Countries for Digital Payments A-D \(on page 153\)](#)
- [Supported Countries for Digital Payments E-K \(on page 155\)](#)
- [Supported Countries for Digital Payments L-R \(on page 158\)](#)
- [Supported Countries for Digital Payments S-Z \(on page 161\)](#)

Supported Countries for Digital Payments A-D

Supported Countries (A through D)

Country	Apple Pay	Click to Pay	eCheck	Google Pay
Afghanistan	✗	✗	✗	✓
Albania	✗	✗	✗	✓
Algeria	✗	✗	✗	✓
Andorra	✗	✗	✗	✓
Angola	✗	✗	✗	✓
Antigua and Barbuda	✗	✗	✗	✓
Argentina	✓	✓	✗	✓
Armenia	✓	✗	✗	✓
Australia	✓	✓	✗	✓
Austria	✓	✓	✗	✓
Azerbaijan	✓	✗	✗	✓
Bahamas	✗	✗	✗	✓
Bahrain	✓	✗	✗	✓

Supported Countries (A through D) (continued)

Country	Apple Pay	Click to Pay	eCheck	Google Pay
Bangladesh	✗	✗	✗	✓
Barbados	✗	✗	✗	✓
Belarus	✓	✗	✗	✓
Belgium	✓	✗	✗	✓
Brazil	✓	✓	✗	✓
Belize	✗	✗	✗	✓
Benin	✗	✗	✗	✓
Bhutan	✗	✗	✗	✗
Bolivia	✗	✗	✗	✓
Bosnia and Herzegovina	✗	✗	✗	✓
Botswana	✗	✗	✗	✓
Brunei Darussalam	✗	✗	✗	✓
Bulgaria	✓	✓	✗	✗
Burkina Faso	✗	✗	✗	✓
Burundi	✗	✗	✗	✓
Cambodia	✗	✗	✗	✓
Cameroon	✗	✗	✗	✓
Canada	✓	✓	✓	✓
Cape Verde	✗	✗	✗	✓
Central African Republic	✗	✗	✗	✓
Chad	✗	✗	✗	✓
Chile	✓	✓	✗	✓

Supported Countries (A through D) (continued)

Country	Apple Pay	Click to Pay	eCheck	Google Pay
China	✓	✓	✗	✗
Colombia	✓	✓	✗	✓
Comoros	✗	✗	✗	✓
Costa Rica	✓	✓	✗	✓
Côte d'Ivoire	✗	✗	✗	✓
Croatia	✓	✗	✗	✓
Cyprus	✓	✗	✗	✓
Czech Republic	✓	✓	✗	✓
Democratic Republic of the Congo	✗	✗	✗	✓
Denmark	✓	✓	✗	✓
Djibouti	✗	✗	✗	✓
Dominica	✗	✗	✗	✓
Dominican Republic	✗	✓	✗	✓

Supported Countries for Digital Payments E-K

Supported Countries (E through K)

Country	Apple Pay	Click to Pay	Google Pay
Ecuador	✗	✓	✓
Egypt	✗	✗	✓
El Salvador	✓	✓	✓
Equatorial Guinea	✗	✗	✓
Eritrea	✗	✗	✓

Supported Countries (E through K) (continued)

Country	Apple Pay	Click to Pay	Google Pay
Estonia	✓	✗	✓
Eswatini	✗	✗	✓
Ethiopia	✗	✗	✓
Faroe Islands	✓	✗	✗
Fiji	✗	✗	✓
Finland	✓	✓	✓
France	✓	✓	✓
Gabon	✗	✗	✓
Gambia	✗	✗	✓
Georgia	✓	✗	✓
Germany	✓	✓	✓
Ghana	✗	✗	✓
Gibraltar	✗	✗	✗
Greece	✓	✓	✓
Greenland	✓	✗	✗
Guernsey	✓	✗	✗
Grenada	✗	✗	✓
Guatemala	✓	✓	✓
Guinea	✗	✗	✓
Guinea-Bissau	✗	✗	✓
Guyana	✗	✗	✓
Haiti	✗	✗	✓
Honduras	✓	✓	✓

Supported Countries (E through K) (continued)

Country	Apple Pay	Click to Pay	Google Pay
Hong Kong	✓	✓	✓
Hungary	✓	✓	✓
Iceland	✓	✗	✓
Indonesia	✗	✓	✗
Iraq	✗	✗	✓
Ireland	✓	✓	✓
Isle of Man	✓	✗	✗
Israel	✓	✗	✓
Italy	✓	✓	✓
Jamaica	✗	✗	✓
Japan	✓	✓	✓
Jersey	✓	✗	✗
Jordan	✓	✓	✓
Kazakhstan	✓	✗	✓
Kenya	✗	✗	✓
Kiribati	✗	✗	✓
Kuwait	✓	✓	✓
Kyrgyzstan	✗	✗	✓

Supported Countries for Digital Payments L-R

Supported Countries (L through R)

Country	Apple Pay	Click to Pay	Google Pay
Laos	✗	✗	✓
Latvia	✓	✗	✓
Lebanon	✗	✗	✓
Lesotho	✗	✗	✓
Liberia	✗	✗	✓
Libya	✗	✗	✓
Liechtenstein	✓	✗	✓
Lithuania	✓	✗	✓
Luxembourg	✓	✗	✓
Macau	✓	✗	✓
Madagascar	✗	✗	✓
Malawi	✗	✗	✓
Malaysia	✓	✓	✓
Maldives	✗	✗	✓
Mali	✗	✗	✓
Malta	✓	✗	✓
Marshall Islands	✗	✗	✓
Mauritania	✗	✗	✓
Mauritius	✗	✗	✓
Mexico	✓	✓	✓
Micronesia, Federated States of	✗	✗	✓
Moldova	✓	✗	✓

Supported Countries (L through R) (continued)

Country	Apple Pay	Click to Pay	Google Pay
Monaco	✓	✗	✓
Mongolia	✗	✗	✓
Montenegro	✓	✗	✓
Morocco	✗	✗	✓
Mozambique	✗	✗	✓
Myanmar	✗	✗	✓
Namibia	✗	✗	✓
Nauru	✗	✗	✓
Nepal	✗	✗	✓
Netherlands	✓	✓	✓
New Zealand	✓	✓	✓
Nicaragua	✗	✓	✓
Niger	✗	✗	✓
Nigeria	✗	✗	✓
North Macedonia	✗	✗	✓
Norway	✓	✓	✓
Oman	✗	✗	✓
Pakistan	✗	✗	✓
Palau	✗	✗	✓
Palestinian Territories	✓	✗	✓
Panama	✓	✓	✓
Papua New Guinea	✗	✗	✓
Paraguay	✗	✓	✓

Supported Countries (L through R) (continued)

Country	Apple Pay	Click to Pay	Google Pay
Peru	✓	✓	✓
Philippines	✗	✗	✗
Poland	✓	✓	✓
Portugal	✓	✗	✓
Qatar	✓	✓	✓
Republic of the Congo	✗	✗	✓
Romania	✓	✓	✓
Rwanda	✗	✗	✓

Supported Countries for Digital Payments S-Z

Supported Countries (S through Z)

Country	Apple Pay	Click to Pay	eCheck	Google Pay	Paze
Saint Kitts and Nevis	✗	✗	✗	✓	✗
Saint Lucia	✗	✗	✗	✓	✗
Saint Vincent and the Grenadines	✗	✗	✗	✓	✗
Samoa	✗	✗	✗	✓	✗
San Marino	✓	✗	✗	✓	✗
Sao Tome and Principe	✗	✗	✗	✓	✗
Saudi Arabia	✓	✓	✗	✓	✗
Senegal	✗	✗	✗	✓	✗
Serbia	✓	✗	✗	✓	✗
Seychelles	✗	✗	✗	✓	✗
Sierra Leone	✗	✗	✗	✓	✗
Singapore	✓	✓	✗	✓	✗
Slovakia	✓	✓	✗	✓	✗
Slovenia	✓	✓	✗	✓	✗
Solomon Islands	✗	✗	✗	✓	✗
Somalia	✗	✗	✗	✓	✗
South Africa	✓	✓	✗	✓	✗
Korea, Republic of (South)	✓	✗	✗	✓	✗
South Sudan	✗	✗	✗	✓	✗

Supported Countries (S through Z) (continued)

Country	Apple Pay	Click to Pay	eCheck	Google Pay	Paze
Spain	✓	✓	✗	✓	✗
Sri Lanka	✗	✗	✗	✓	✗
Sudan	✗	✗	✗	✓	✗
Suriname	✗	✗	✗	✓	✗
Sweden	✓	✓	✗	✓	✗
Switzerland	✓	✓	✗	✓	✗
Switzerland -Italian	✗	✗	✗	✗	✗
Taiwan	✓	✗	✗	✓	✗
Tajikistan	✗	✗	✗	✓	✗
Tanzania	✗	✗	✗	✓	✗
Thailand	✗	✓	✗	✗	✗
Timor-Leste	✗	✗	✗	✓	✗
Togo	✗	✗	✗	✓	✗
Tonga	✗	✗	✗	✓	✗
Trinidad and Tobago	✗	✗	✗	✓	✗
Tunisia	✗	✗	✗	✓	✗
Turkey	✗	✗	✗	✓	✗
Turkmenistan	✗	✗	✗	✓	✗
Tuvalu	✗	✗	✗	✓	✗
Uganda	✗	✗	✗	✓	✗
Ukraine	✓	✓	✗	✓	✗
United Arab Emirates	✓	✓	✗	✓	✗

Supported Countries (S through Z) (continued)

Country	Apple Pay	Click to Pay	eCheck	Google Pay	Paze
United Kingdom	✓	✓	✗	✓	✗
United States	✓	✓	✓	✓	✓
Uruguay	✗	✓	✗	✓	✗
Uzbekistan	✗	✗	✗	✓	✗
Vanuatu	✗	✗	✗	✓	✗
Vatican City (Holy See)	✓	✗	✗	✓	✗
Venezuela	✗	✗	✗	✓	✗
Vietnam	✓	✓	✗	✓	✗
Yemen	✗	✗	✗	✓	✗
Zambia	✗	✗	✗	✓	✗
Zimbabwe	✗	✗	✗	✓	✗

Supported Locales

The locale field within the capture context request consists of an ISO 639 language code, an underscore (_), and an ISO 3166 region code. The locale controls the language in which the application is rendered. The following locales are supported:

- ar_AE
- bg_BG
- ca_ES
- cs_CZ
- da_DK
- de_AT
- de_DE
- el_GR
- en_AU
- en_CA
- en_GB
- en_IE
- en_NZ
- en_PK
- en_US
- es_AR
- es_CL
- es_CO
- es_ES
- es_MX
- es_PE
- es_US
- fi_FI

- fr_CA
- fr_FR
- he_IL
- hr_HR
- hu_HU
- id_ID
- it_IT
- ja_JP
- km_KH
- ko_KR
- lo_LA
- ms_MY
- nb_NO
- nl_NL
- pl_PL
- pt_BR
- ro_RO
- ru_RU
- sk_SK
- sl_SI
- sv_SE
- th_TH
- tl_PH
- tr_TR
- ur_PK
- vi_VN
- zh_CN

- zh_HK
- zh_MO
- zh_SG
- zh_TW

Reason Codes

A Unified Checkout request response returns one of the following reason codes:

Reason Codes

Reason Code	Description
200	Successful response.
201	Capture context created.
400	Bad request. Possible reason values: • CAPTURE_CONTEXT_EXPIRED • CAPTURE_CONTEXT_INVALID • CHECKOUT_ERROR • CLICK_TO_PAY_SDK_LOAD_ERROR • COMPLETE_AUTHENTICATION_CANCELED • COMPLETE_AUTHENTICATION_FAILED • COMPLETE_ERROR • COMPLETE_IN_PROGRESS • COMPLETE_NOT_ALLOWED • COMPLETE_TRANSACTION_CANCELLED • COMPLETE_TRANSACTION_FAILED • COMPLETE_VALIDATION_ERROR • CREATE_TOKEN_TIMEOUT • CREATE_TOKEN_XHR_ERROR • ENCRYPT_CARD_FOR_SRC_ENROLMENT_ERROR • GOOGLEPAY_CHECKOUT_ERROR • INVALID_APIKEY • LAUNCH_SRC_CHECKOUT_ERROR • SDK_XHR_ERROR

Reason Codes (continued)

Reason Code	Description
	• <code>SHOW_LOAD_CONTAINER_SELECTOR</code> • <code>SHOW_LOAD_ERROR</code> • <code>SHOW_LOAD_INVALID_CONTAINER</code> • <code>SHOW_LOAD_SIDEBAR_OPTIONS</code> • <code>SHOW_PAYMENT_TIMEOUT</code> • <code>SHOW_PAYMENT_UNAVAILABLE</code> • <code>SHOW_TOKEN_TIMEOUT</code> • <code>SHOW_TOKEN_XHR_ERROR</code> • <code>TOKENIZATION_ERROR</code> • <code>TRIGGER_PAYMENT_TYPE_NOT_SUPPORTED</code> • <code>UNIFIED_PAYMENTS_ALREADY_SHOWN</code> • <code>UNIFIED_PAYMENTS_PAYMENT_PARAMETERS</code> • <code>UNIFIED_PAYMENTS_VALIDATION_FIELDS</code> • <code>UNIFIED_PAYMENTS_VALIDATION_PARAMS</code>
<code>404</code>	The specified resource not found in the system.
<code>500</code>	Unexpected server error.

Unified Checkout Field Reference

This section includes the fields that you use for Unified Checkout.

Main Configuration Fields

The following table describes the main configuration fields for Unified Checkout:

Main Configuration Fields

Field Name	Data Type	Required?	Example	Details
allowedCardNetworks	Array	No	<code>["VISA", "MASTERCARD", "AMEX"]</code>	
allowedPaymentTypes	Array	Yes	<code>["PANENTRY", "CLICKTOPAY", "APPLEPAY", "GOOGLEPAY"]</code>	The field value can be an array of strings or an object with options.
buttonType	Enum	No		Possible values: • <code>ADD_CARD</code> • <code>SAVE_CARD</code> • <code>CARD_PAYMENT</code> • <code>CHECKOUT</code> • <code>CHECKOUT_AND_CONTINUE</code> • <code>DEBIT_CREDIT</code> • <code>DONATE</code> • <code>PAY</code> • <code>PAY_WITH_CARD</code> • <code>SUBSCRIBE_WITH_CARD</code>
clientReferenceInformation.code	String	No	<code>TAGX001</code>	
clientVersion	String	Yes	<code>0.31</code>	
country	String	Yes	<code>US</code>	
locale	String	Yes	<code>en_US</code>	
targetOrigins	Array	Yes	<code>["https://merchant.com", "https://reseller.com:8443"]</code>	Each origin must contain HTTPS and include the scheme, host, and optional port.

Order Information Fields

This table contains information about the **data.orderInformation** field object:

Order Information Fields

Field	Type	Required?	Example
data.orderInformation.amountDetails.currency	String	Yes	USD
data.orderInformation.amountDetails.discountAmount	String	No	2.00
data.orderInformation.amountDetails.servieFeeAmount	String	No	5.00
data.orderInformation.amountDetails.surcharge.amount	String	No	4.50
data.orderInformation.amountDetails.taxAmount	String	No	10.00
data.orderInformation.amountDetails.taxDetails[].taxId	String	No	1234
data.orderInformation.amountDetails.taxDetails[].type	String	No	N
data.orderInformation.amountDetails.totalAmount	String	Yes	102.21
data.orderInformation.billTo.address1	String	Required for Afterpay	1 Market St
data.orderInformation.billTo.address2	String	No	Apt B
data.orderInformation.billTo.address3	String	No	Building C
data.orderInformation.billTo.address4	String	No	Floor 2

Order Information Fields (continued)

Field	Type	Required?	Example
data.orderInformation.billTo.administrativeArea	String	Required for Afterpay	CA
data.orderInformation.billTo.buildingNumber	String	No	123
data.orderInformation.billTo.company.address1	String	No	123 Street Road
data.orderInformation.billTo.company.address2	String	No	Suite 100
data.orderInformation.billTo.company.address3	String	No	Building A
data.orderInformation.billTo.company.address4	String	No	Floor 5
data.orderInformation.billTo.company.administrativeArea	String	No	CA
data.orderInformation.billTo.company.buildingNumber	String	No	900
data.orderInformation.billTo.company.country	String	No	US
data.orderInformation.billTo.company.district	String	No	Metro
data.orderInformation.billTo.company.locality	String	No	Foster City
data.orderInformation.billTo.company.name	String	No	Company Name

Order Information Fields (continued)

Field	Type	Required?	Example
data.orderInformation.billTo.company.postalCode	String	No	12345
data.orderInformation.billTo.country	String	Conditional	US
data.orderInformation.billTo.district	String	No	Downtown
data.orderInformation.billTo.email	String	Required for Afterpay	user@example.com
data.orderInformation.billTo.firstName	String	Required for Afterpay and iDEAL	Jane
data.orderInformation.billTo.lastName	String	Required for Afterpay and iDEAL	Doe
data.orderInformation.billTo.locality	String	Required for Afterpay	San Francisco
data.orderInformation.billTo.middleName	String	No	Michael
data.orderInformation.billTo.nameSuffix	String	No	Jr.
data.orderInformation.billTo.phoneNumber	String	No	111-111-1111
data.orderInformation.billTo.phoneType	String	No	day
data.orderInformation.billTo.postalCode	String	Required for Afterpay	94105
data.orderInformation.billTo.title	String	No	Mr.
data.orderInformation.lineItems[].amountIncludesTax	Boolean	No	true

Order Information Fields (continued)

Field	Type	Required?	Example
data.orderInformation.lineItems[].commodityCode	String	No	8471
data.orderInformation.lineItems[].discountAmount	String	No	3.00
data.orderInformation.lineItems[].discountApplied	Boolean	No	true
data.orderInformation.lineItems[].discountRate	String	No	0.0500
data.orderInformation.lineItems[].fulfillmentType	String	No	physical
data.orderInformation.lineItems[].gift	Boolean	No	false
data.orderInformation.lineItems[].giftCardCurrency	String	No	2
data.orderInformation.lineItems[].invoiceDetails.productDescription	String	No	Description
data.orderInformation.lineItems[].invoiceNumber	String	No	INV-1001
data.orderInformation.lineItems[].passenger.email	String	No	user@example.com
data.orderInformation.lineItems[].passenger.firstName	String	No	Jane
data.orderInformation.lineItems[].passenger.id	String	No	1234567890

Order Information Fields (continued)

Field	Type	Required?	Example
data.orderInformation.lineItems[].passenger.lastName	String	No	Doe
data.orderInformation.lineItems[].passenger.nationality	String	No	US
data.orderInformation.lineItems[].passenger.phone	String	No	111-111-1111
data.orderInformation.lineItems[].passenger.status	String	No	standard
data.orderInformation.lineItems[].passenger.type	String	No	ADT
data.orderInformation.lineItems[].productCode	String	Conditional	electronic_good
data.orderInformation.lineItems[].productDescription	String	No	HD Receiver
data.orderInformation.lineItems[].productName	String	No	Receiver
data.orderInformation.lineItems[].productSku	String	No	SKU-12345
data.orderInformation.lineItems[].quantity	String	Conditional	2
data.orderInformation.lineItems[].referenceDataCode	String	No	REF1
data.orderInformation.lineItems[].referenceDataNumber	String	No	123

Order Information Fields (continued)

Field	Type	Required?	Example
data.orderInformation.lineItems[].shippingDestinationTypes	String	No	residential
data.orderInformation.lineItems[].taxAmount	String	No	5.40
data.orderInformation.lineItems[].taxAppliedAfterDiscount	String	No	Y
data.orderInformation.lineItems[].taxDetails[].amount	String	No	4.50
data.orderInformation.lineItems[].taxDetails[].applied	Boolean	No	true
data.orderInformation.lineItems[].taxDetails[].code	String	No	1234
data.orderInformation.lineItems[].taxDetails[].exemptionCode	String	No	1
data.orderInformation.lineItems[].taxDetails[].rate	String	No	0.0750
data.orderInformation.lineItems[].taxDetails[].taxId	String	No	TX-123456
data.orderInformation.lineItems[].taxDetails[].type	String	No	STATE_SALES_TAX
data.orderInformation.lineItems[].taxRate	String	No	0.0900
data.orderInformation.lineItems[].taxStatusIndicator	String	No	N

Order Information Fields (continued)

Field	Type	Required?	Example
data.orderInformation.lineItems[].taxTypeCode	String	No	1234
data.orderInformation.lineItems[].totalAmount	String	Conditional	59.98
data.orderInformation.lineItems[].typeOfSupply	String	No	12
data.orderInformation.lineItems[].unitOfMeasure	String	No	EA
data.orderInformation.lineItems[].unitPrice	String	Required for line-item authorizations.	29.99
data.orderInformation.lineItems[].unitTaxAmount	String	No	1.00
data.orderInformation.lineItems[].weight	String	No	500
data.orderInformation.lineItems[].weightIdentifier	String	No	"N"
data.orderInformation.lineItems[].weightUnit	String	No	mg
data.orderInformation.shipTo.address1	String	No	456 Nice Avenue
data.orderInformation.shipTo.address2	String	No	Unit 7
data.orderInformation.shipTo.address3	String	No	Warehouse B
data.orderInformation.shipTo.address4	String	No	Dock 3

Order Information Fields (continued)

Field	Type	Required?	Example
data.orderInformation.shipTo.administrativeArea	String	No	CA
data.orderInformation.shipTo.buildingNumber	String	No	
data.orderInformation.shipTo.country	String	No	US
data.orderInformation.shipTo.district	String	No	Midtown
data.orderInformation.shipTo.firstName	String	Conditional	John
data.orderInformation.shipTo.lastName	String	No	Buyer
data.orderInformation.shipTo.locality	String	No	Los Angeles
data.orderInformation.shipTo.postalCode	String	No	90010

Capture Mandate Fields

The following table describes the **captureMandate** object fields. The values in these fields determine which fields Unified Checkout displays in the UI:

Capture Mandate Fields

Field	Data Type	Required?	Example
captureMandate.billingType	Enum	No	FULL
captureMandate.comboCard	Boolean	No	true
captureMandate.CPF.required	Boolean	Conditional	true

Capture Mandate Fields (continued)

Field	Data Type	Required?	Example
captureMandate.requestEmail	Boolean	No	true
captureMandate.requestPhone	Boolean	No	true
captureMandate.requestSaveCard	Boolean	No	true
captureMandate.requestShipping	Boolean	No	true
captureMandate.shipToCountries	Array	No	["US", "GB"]
captureMandate.showAcceptedNetworkIcons	Boolean	No	true
captureMandate.showConfirmationStep	Boolean	No	true

Complete Mandate Fields

This table contains information about the **completeMandate** field object:

Complete Mandate Fields

Field	Type	Required?	Example	Details
completeMandate.consumerAuthentication	Boolean	Conditional	true	Enable 3-D Secure authentication for supported card types.
completeMandate.decisionManager	Boolean	Conditional	true	Enable Decision Manager fraud screening with device fingerprinting.
completeMandate.tms.tokenCreate	Boolean	Conditional	true	Create tokens through the Token Management Service.
completeMandate.tms.tokenTypes	Array	Conditional	["customer", "paymentInstrument",	Types of tokens to create:

Complete Mandate Fields (continued)

Field	Type	Required?	Example	Details
			<code>"instrumentIdentifier", "shippingAddress"]</code>	• Customer • Instrument identifier • Payment instrument • Shipping address When you do not set this field value to a token type, the default value is based on the vault configuration.
completeMandate.type	Enum	Conditional	<code>CAPTURE</code>	Type of transaction required.

Buyer Information Fields

This table contains information about the **buyerInformation** field object:

Buyer Information Fields

Field	Type	Required?	Example
buyerInformation.companyTaxId	String	Regional	<code>12.345.678/0001-90</code>
buyerInformation.dateOfBirth	String	No	<code>19900101</code>
buyerInformation.language	String	No	<code>en-US</code>
buyerInformation.merchantCustomerId	String	No	<code>cust_12345</code>
buyerInformation.personalIdentification.cpf	String	Required in Brazil	<code>12345678900</code>

Client Reference Information Fields

This table contains information about the **clientReferenceInformation** field object:

Client Reference Information Fields

Field	Type	Required	Example
clientReferenceInformation.code	String	No	TAGX001
clientReferenceInformation.partner.developerId	String	No	1234
clientReferenceInformation.partner.solutionId	String	No	4567

Consumer Authentication Information Fields

This table contains information about the **consumerAuthenticationInformation** field object. These fields are used only for 3-D Secure:

Consumer Authentication Information Fields

Field	Type	Required	Example
consumerAuthenticationInformation.challengeCode	String	Yes	01
consumerAuthenticationInformation.messageCategory	String	Yes	01
consumerAuthenticationInformation.acsWindowSize	String	No	10

Merchant Information Fields

The following table describes the **merchantInformation.merchantDescriptor** object fields:

Merchant Information Fields

Field	Type	Required?	Example
merchantInformation.merchantDescriptor.address1	String	No	123 Street Road

Merchant Information Fields (continued)

Field	Type	Required?	Example
merchantInformation.merchantDescriptor.administrativeArea	String	No	CA
merchantInformation.merchantDescriptor.alternateName	String	No	Susan
merchantInformation.merchantDescriptor.country	String	No	US
merchantInformation.merchantDescriptor.locality	String	No	Foster City
merchantInformation.merchantDescriptor.name	String	No	Jane Sales
merchantInformation.merchantDescriptor.phone	String	No	111-111-1111
merchantInformation.merchantDescriptor.postalCode	String	No	12345

Processing Information Fields

This table contains information about the **processingInformation** field object.

Processing Information Fields

Field	Type	Required?	Example
processingInformation.authorizationOptions.afterIndicator	Boolean	Conditional	true
processingInformation.authorizationOptions.authIndicator	Enum	No	0
processingInformation.authorizationOptions.ignoreAvsResult	Boolean	No	true

Processing Information Fields (continued)

Field	Type	Required?	Example
processingInformation.authorizationOptions.ignoreCvResult	Boolean	No	true
processingInformation.authorizationOptions.initiator.credentialStoredOnFile	Boolean	Required for stored credentials	true
processingInformation.authorizationOptions.initiator.merchantInitiatedTransaction.reason	String	Required for stored credentials	1
processingInformation.businessApplicationId	String	Required for Payouts	AA
processingInformation.commerceIndicator	String	No	retail
processingInformation.processingInstruction	String	No	NO_INSTRUCTION
processingInformation.reconciliationId	String	No	123456789012345

Recipient Information Fields

This table contains information about the **recipientInformation** field object. These fields are used only for payouts:

Recipient Information Fields

Field	Type	Required?	Example
recipientInformation.accountId	String	Conditional	acc0123567
recipientInformation.accountType	String	Conditional	01
recipientInformation.administrativeArea	String	No	GB
recipientInformation.country	String	Conditional	GB
recipientInformation.dateOfBirth	String	No	19900101

Recipient Information Fields (continued)

Field	Type	Required?	Example
recipientInformation.firstName	String	Conditional	John
recipientInformation.lastName	String	Conditional	Buyer
recipientInformation.middleName	String	No	A
recipientInformation.postalCode	String	No	12345

Merchant Defined Information Fields

This table contains information about the **merchantDefinedInformation[]** field array:

Merchant Defined Information Fields

Field	Type	Required?	Example
merchantDefinedInformation[].key	String	Required when merchantDefinedInformation[].value is included in the request.	customer_id
merchantDefinedInformation[].value	String	Required when merchantDefinedInformation[].key is included in the request.	12345

Device Information Fields

This table contains information about the **deviceInformation** field object:

Merchant Defined Information Fields

Field	Type	Required?
<code>deviceInformation.ipAddress</code>	String	No

Payment Information Fields

This table contains information about the **paymentInformation** field object:

Merchant Defined Information Fields

Field	Type	Required?	Example
<code>paymentInformation.card.typeSelectionIndicator</code>	Enum	No	<code>0</code>

Allowed Payment Types Variations

This table describes the possible values for the **allowedPaymentTypes** field object:

Payment Types Variations

Payment Type	Value	Additional Requirements in Capture Context	Details
Manual card entry	<code>PANENTRY</code>		Basic card entry.
Click to Pay	<code>CLICKTOPAY</code> or object <pre>{ "type": "CLICKTOPAY", "options": { .. } }</pre>	Include email in for autolookup.	Auto-check enrollment available is available through options.autoCheckEnrollment .
Apple Pay	<code>APPLEPAY</code>		
Google Pay	<code>GOOGLEPAY</code>		
Paze	<code>PAZE</code>		
iDEAL	<code>IDEAL</code>	Include these fields: • billTo.firstName • billTo.lastName • billTo.country	Set the completeMandate.type field to <code>CAPTURE</code> or <code>PREFER_AUTH</code> .

Payment Types Variations (continued)

Payment Type	Value	Additional Requirements in Capture Context	Details
Afterpay/Clearpay	AFTERPAY		Set the completeMandate.type field to AUTH , CAPTURE or PREFER_AUTH , as required.

Related Information

- [API field reference guide for the REST API](#)

Test Authentication

Use this table to determine how to test your authentication method.

Authentication Testing by Product

Payment Method	Authentication	Minimum Follow-On Actions	Prerequisites	Test Cards	Details
PAN Entry	Payer Authentication through Unified Checkout	Authorization and Payer Authentication	The transacting MID must be enabled for Payer Authentication and the complete mandate is used with the consumerAuthentication field set to true .	See Testing Payer Authentication in the <i>Payer Authentication Developer Guide</i> .	When the complete mandate is not used, Unified Checkout does not initiate authentication and you must perform authentication within your own environment.
Click to Pay	Payer Authentication through Unified Checkout	Authorization and Payer Authentication	The transacting MID must be enabled for Payer	See Test Payment Details (on page 94) .	When authentication is not enabled for Click to Pay or Click to Pay is not able

Authentication Testing by Product (continued)

Payment Method	Authentication	Minimum Follow-On Actions	Prerequisites	Test Cards	Details
			Authentication and the complete mandate is used with the consumerAuthentication field set to true. Authentication for Click to Pay must not be configured.		to perform authentication for Click to Pay, Unified Checkout performs authentication using Payer Authentication when the complete mandate is used with the consumerAuthentication field set to true.
Click to Pay	Visa Click to Pay	Authorization and Payer Authentication	You must configure the authentication for Click to Pay. Click to Pay performs authentication only if it is a tokenized Visa card.	See Test Payment Details (on page 94) .	When authentication is enabled for Click to Pay, authentication is attempted for all Click to Pay transactions for Visa cards that are stored in Click to Pay. For information about setting up authentication for Visa Click to Pay, see Set Up Customer Authentication for Visa Click to Pay (on page 81) .

Authentication Testing by Product (continued)

Payment Method	Authent ication	Minimum Follow-On Actions	Prerequisites	Test Cards	Details
Google Pay	Google Pay	Authorization	A Google device must be used with biometric authentication for Google authentication.		A user authenticates themselves on a Google device with a tokenized Google Pay credential – the returned payload from Google will be Authenticated
Google Pay	Payer Authentication through Unified Checkout	Authorization and Payer Authentication	You must use a device, such as a web browser, that does not authenticate the cardholder as part of the authorization process.		Google will return an un-authenticated payload to Unified Checkout . Unified Checkout will step in and process Authentication via Payer Authentication when the Complete Mandate function is used with consumerAuth entication

Click to Pay UI Guidelines

The UI that is built in Unified Checkout for Click to Pay is built based on the EMV Click to Pay XC Guidelines V1.1. Unified Checkout has simplified the integration of the UI. The only UI work that you must complete is the placement of the payment option.



Important: You must include Click to Pay as one of the presented payment methods and not as a separate payment method.

Unified Checkout captures all card details that are manually entered by the cardholder. This enables the cardholder to enroll in Click to Pay and removes the requirement for the cardholder to manually enter their card details the next time they check out.

Unified Checkout provides a standard payment label in the Unified Checkout JavaScript that is loaded in your checkout page. One of these scenarios occurs when the cardholder selects the button:

- The cardholder is recognized.
- The cardholder is not recognized but has a Click to Pay account.
- The cardholder does not have a Click to Pay account.

You can also trigger the Unified Checkout flow using a custom button. If you are using your own custom button, your payment button or widget must display the Click to Pay image for the cardholder. For information about a custom button, see [JavaScript Example: Client-Defined Trigger for Click to Pay or PAN Entry \(on page 24\)](#).



Important: Your implementation consultant will ask you for a mock-up of your payment flow for confirmation that it is compliant with the Click to Pay UI design standards.

Recognized Click to Pay Customer

The cardholder is presented with their stored Click to Pay cards in the UI when they are on a recognized device:

Recognized Click to Pay Customer UI

PAYMENT DETAILS:

 e.....e@email.com [Switch ID](#)

 **VISA** 5555

 **VISA** 5678

★ Reward points with this card

[Enter card manually](#)

Unrecognized Click to Pay Customer

When the cardholder has a Click to Pay account but is not on a registered device, they receive a one-time password to their registered email address and phone number to authenticate their identity before their stored Click to Pay credentials are shown:

Unrecognized Click to Pay Customer on a Recognized Device UI

Click to Pay has found your linked cards

To access your cards, enter the code sent to +44233 and e.....e@email.com

1 2 3 4 5 6

[Resend cod](#)

☒ Skip verification next time

Verify and continue

To access a different set of linked cards [Switch ID](#)

No Click to Pay Account

When the cardholder does not have a Click to Pay account, they can provide a new email address to perform a new lookup or they can choose to enter their card details manually. The cardholder can make a one-time payment or complete the payment and choose to create a Click to Pay account for future use:

No Click to Pay Account UI

CONTACT DETAILS

[Edit](#)

example@email.com

+44 7412 112233

PAYMENT DETAILS:

Card number

Expiry MM/YY

CVV

First name

Last name

Click to Pay didn't find linked cards for
example@email.com

[Switch ID](#)

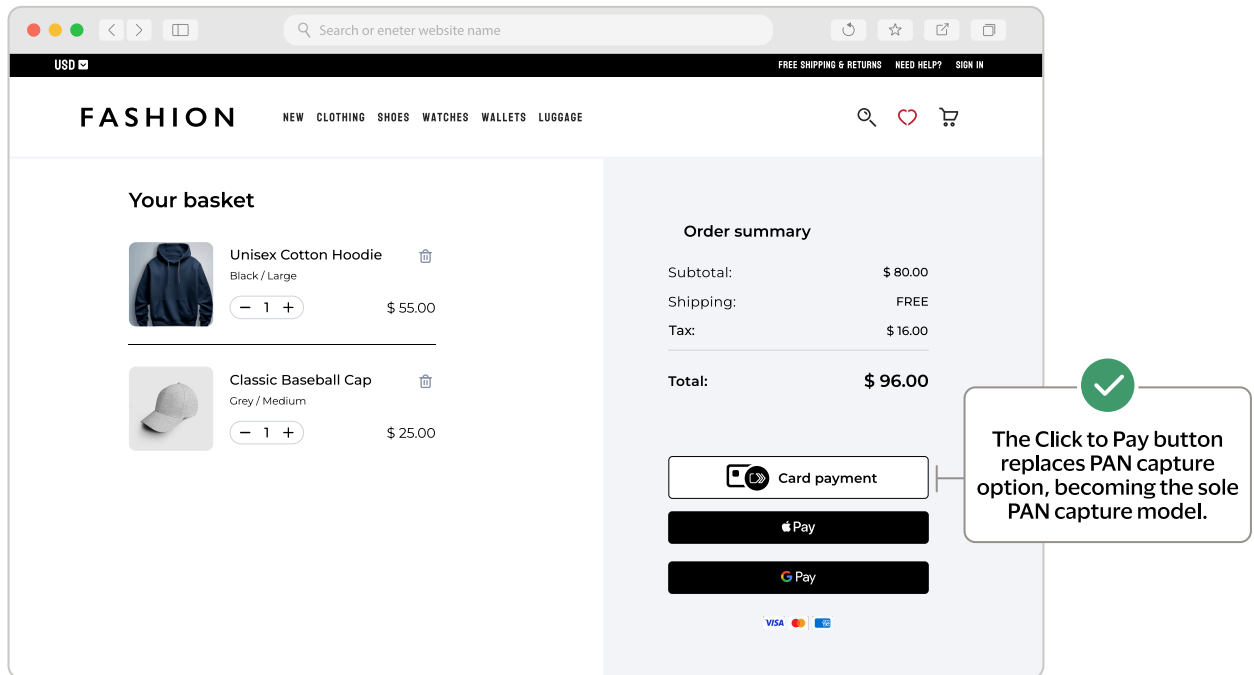
Click to Pay UI Examples

This section contains UI examples of how you should display Click to Pay on your payment page. For information about how to display the UI, see [JavaScript API Reference \(on page 65\)](#).

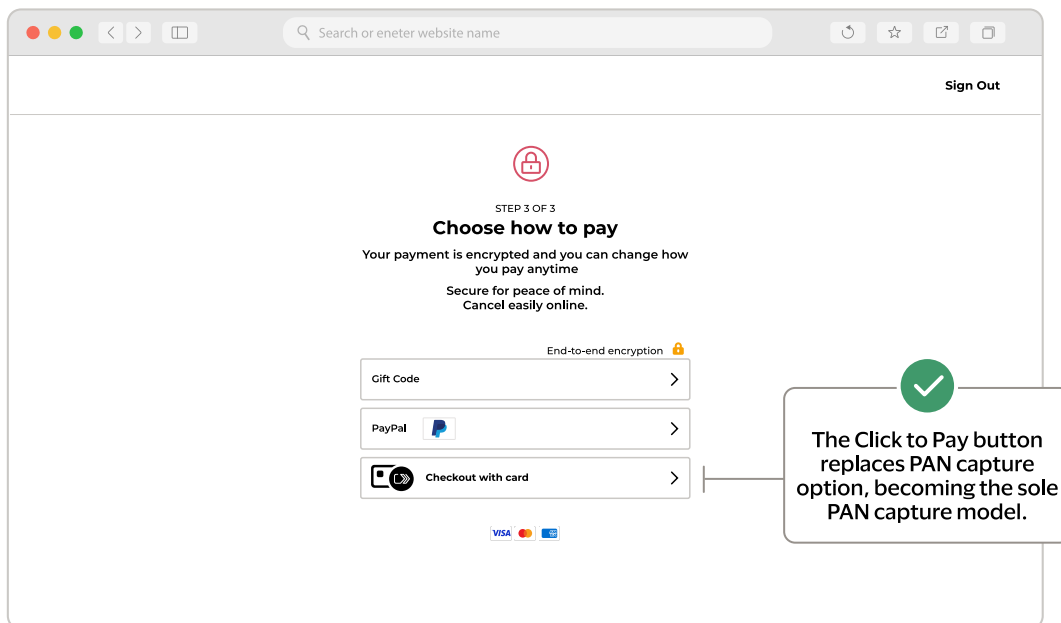
Click to Pay Replaces PAN Capture

Click to Pay is the card entry payment option within your payment page.

Click to Pay Replaces PAN Capture UI Example 1



Click to Pay Replaces PAN Capture UI Example 2

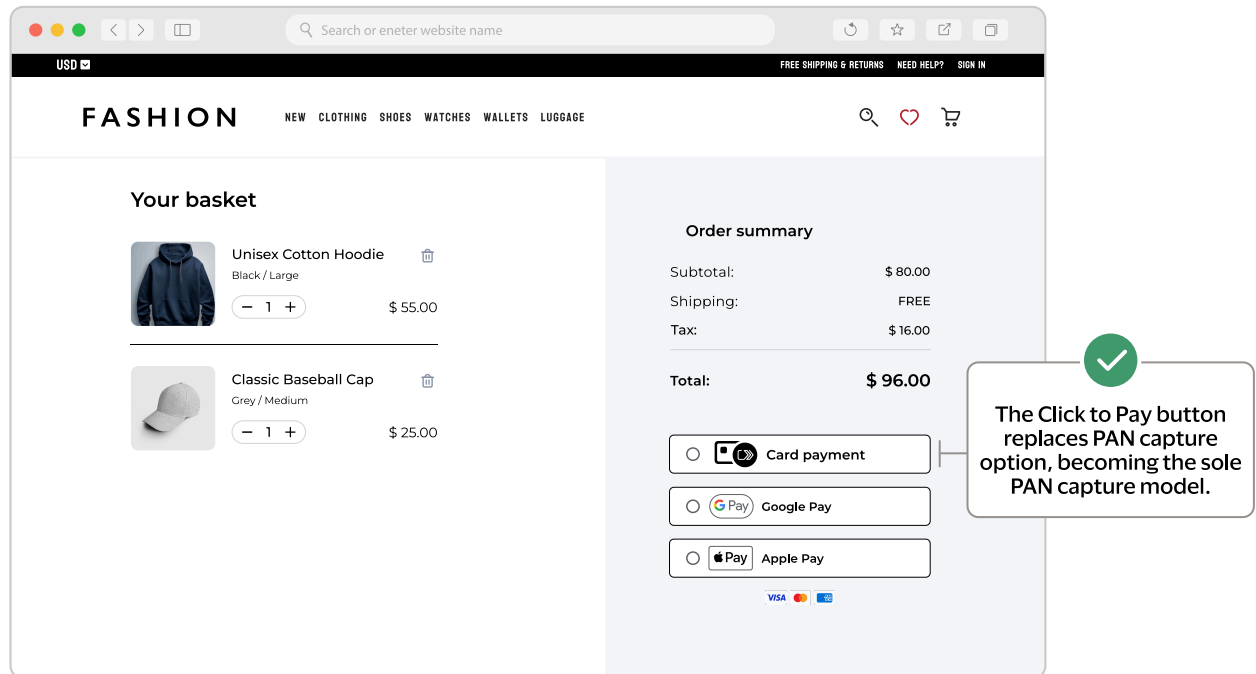


For information about how to configure this UI, see [Loading the JavaScript Library and Invoking the Accept Function \(on page 22\)](#).

Click to Pay as Radio Button

Click to Pay is a radio button for the card entry payment option within your payment page. When the cardholder selects this option, the Click to Pay payment flow is loaded.

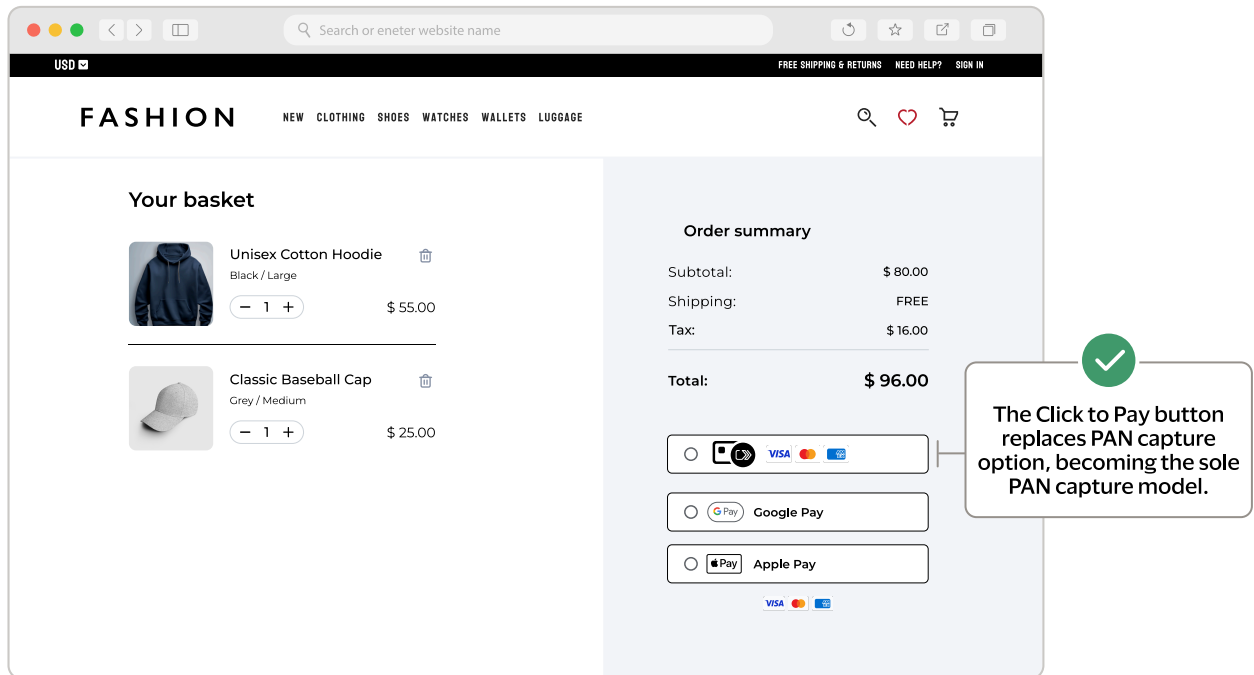
Click to Pay Radio Button Example UI



Click to Pay Icon on Radio Button

You can host the radio selection option for card payment with the Click to Pay icon displayed on the payment label. The Unified Checkout flow loads when the cardholder selects this option. For information about customizing how to trigger Unified Checkout, see [JavaScript Example: Client-Defined Trigger for Click to Pay or PAN Entry \(on page 24\)](#).

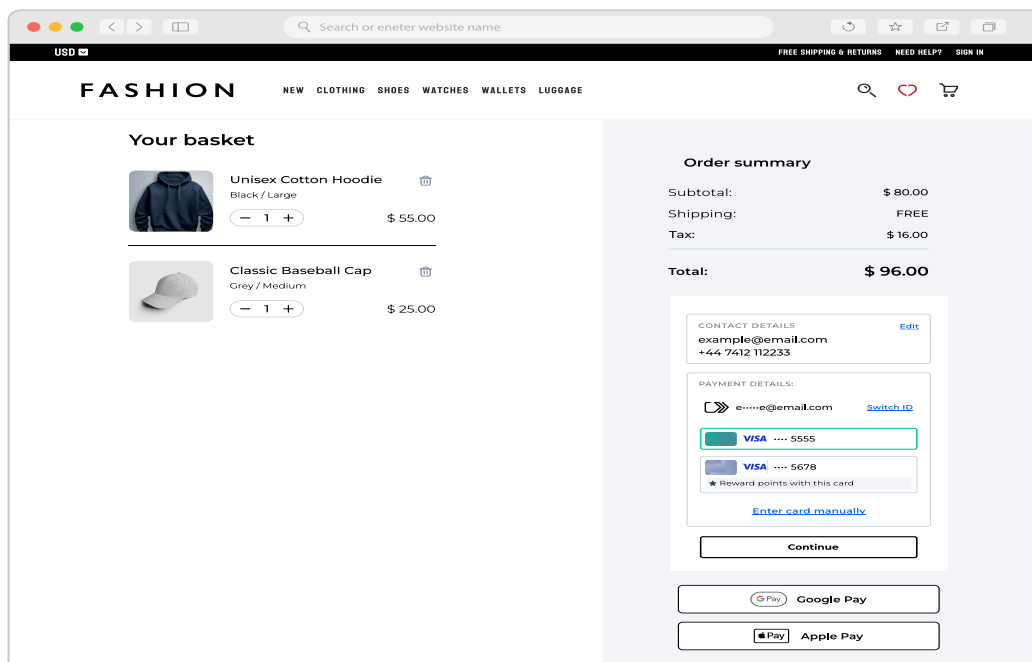
Click to Pay Icon on Radio Button Example UI



Load Click to Pay Automatically From Trigger

You can load the Unified Checkout JavaScript flow within your payment page without requiring the cardholder to select a card payment option. This example shows a recognized user payment flow where the cardholder's information is shown automatically next to the other payment methods hosted within your payment page. For information about customizing how to trigger Unified Checkout, see [JavaScript Example: Client-Defined Trigger for Click to Pay or PAN Entry \(on page 24\)](#).

Click to Pay Loaded Automatically From Trigger UI

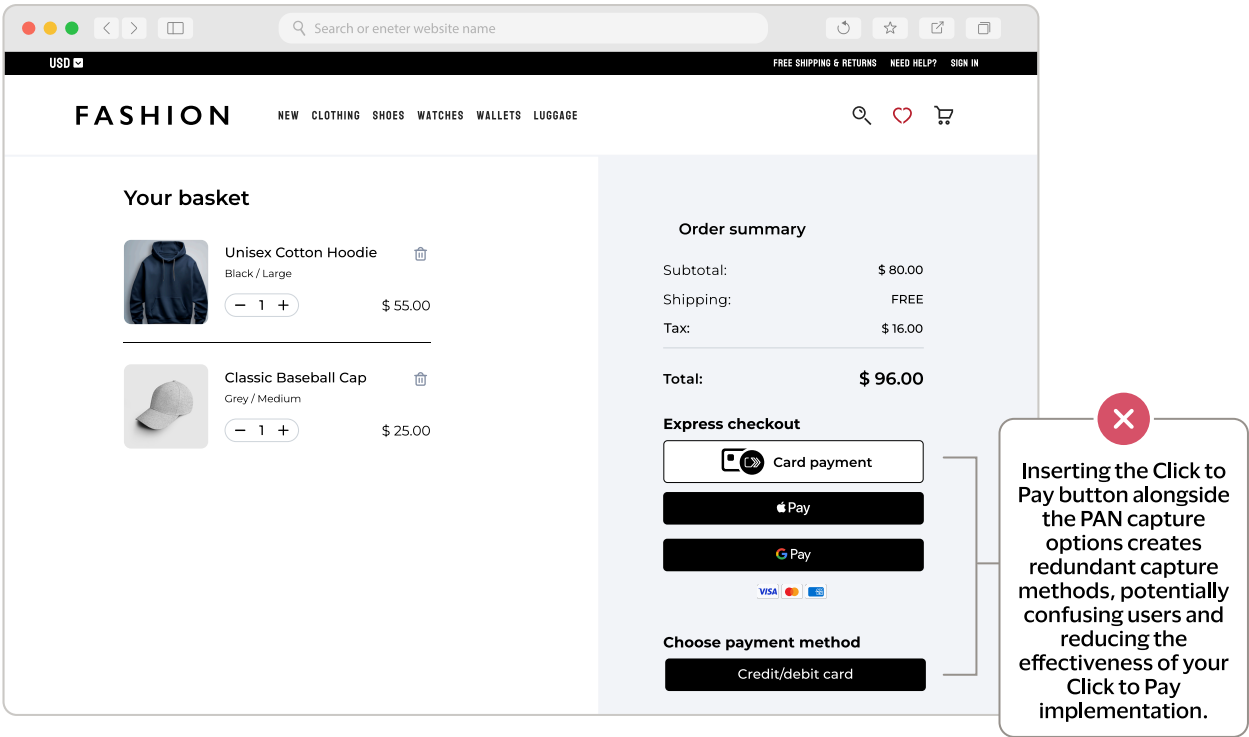


Card Payment Options with Click to Pay in UI

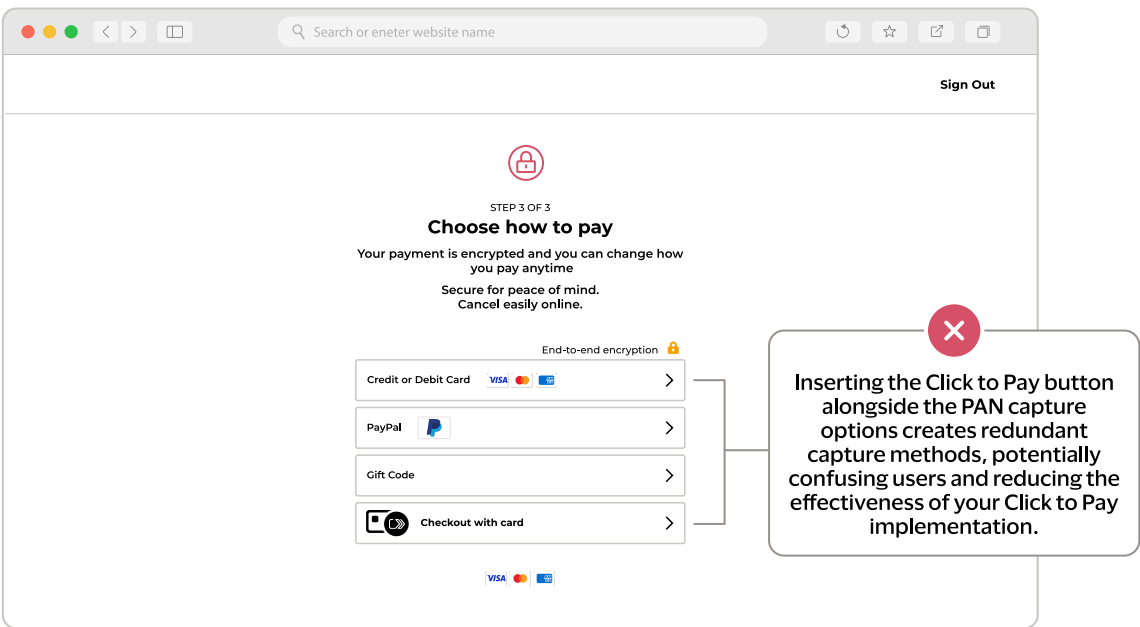
Do not present the Unified Checkout payment button as a separate payment method from the card payment button. If you do this, the cardholder is not prompted with their Click to Pay cards and must manually enter their payment details. They will also not have the option to store their card within Click to Pay for future use.

These examples show multiple card payment options and Click to Pay in a UI:

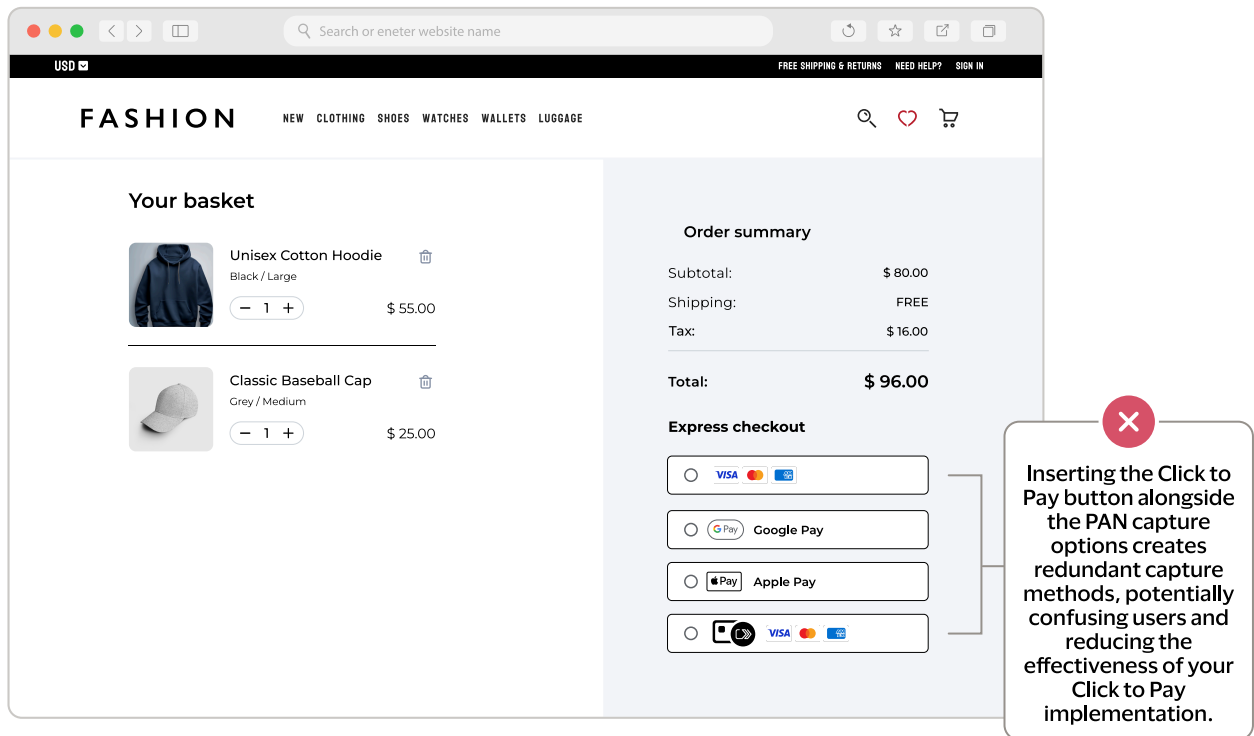
Multiple Card Payment Options in UI Example 1



Multiple Card Payment Options in UI Example 2



Multiple Card Payment Options in UI Example 3



VISA Platform Connect: Specifications and Conditions for Resellers/Partners

The following are specifications and conditions that apply to a Reseller/Partner enabling its merchants through Cybersource for Visa Platform Connect (“VPC”) processing. Failure to meet any of the specifications and conditions below is subject to the liability provisions and indemnification obligations under Reseller/Partner’s contract with Visa/Cybersource.

1. Before boarding merchants for payment processing on a VPC acquirer’s connection, Reseller/ Partner and the VPC acquirer must have a contract or other legal agreement that permits Reseller/Partner to enable its merchants to process payments with the acquirer through the dedicated VPC connection and/or traditional connection with such VPC acquirer.
2. Reseller/Partner is responsible for boarding and enabling its merchants in accordance with the terms of the contract or other legal agreement with the relevant VPC acquirer.
3. Reseller/Partner acknowledges and agrees that all considerations and fees associated with chargebacks, interchange downgrades, settlement issues, funding delays, and other processing related activities are strictly between Reseller and the relevant VPC acquirer.
4. Reseller/Partner acknowledges and agrees that the relevant VPC acquirer is responsible for payment processing issues, including but not limited to, transaction declines by network/ issuer, decline rates, and interchange qualification, as may be agreed to or outlined in the contract or other legal agreement between Reseller/Partner and such VPC acquirer.

DISCLAIMER: NEITHER VISA NOR CYBERSOURCE WILL BE RESPONSIBLE OR LIABLE FOR ANY ERRORS OR OMISSIONS BY THE Visa Platform Connect ACQUIRER IN PROCESSING TRANSACTIONS. NEITHER VISA NOR CYBERSOURCE WILL BE RESPONSIBLE OR LIABLE FOR RESELLER/PARTNER BOARDING MERCHANTS OR ENABLING MERCHANT PROCESSING IN VIOLATION OF THE TERMS AND CONDITIONS IMPOSED BY THE RELEVANT Visa Platform Connect ACQUIRER.